

Sync Toolbox: A Python Package for Efficient, Robust, and Accurate Music Synchronization

Meinard Müller¹, Yigitcan Özer¹, Michael Krause¹, Thomas Prätzlich¹, and Jonathan Driedger¹

¹ International Audio Laboratories Erlangen

DOI: [10.21105/joss.03434](https://doi.org/10.21105/joss.03434)

Software

- [Review](#) ↗
- [Repository](#) ↗
- [Archive](#) ↗

Editor: [Fabian-Robert Stöter](#) ↗

Reviewers:

- [@lutzhamel](#)
- [@magdalenafuentes](#)

Submitted: 10 June 2021

Published: 27 August 2021

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Music can be described and represented in many different ways, including as sheet music, symbolic representations, and audio recordings (Müller, 2015). For each of these representations, different versions (e.g., recordings performed by different orchestras and conductors) that correspond to the same musical work may exist. Music information retrieval (MIR) aims at developing techniques and tools for organizing, understanding, and searching this information in a robust, efficient, and intelligent manner. In this context, various alignment and synchronization procedures have been developed with the common goal of automatically linking several types of music representations, thus coordinating the multiple information sources related to a given musical work. In the design and implementation of synchronization algorithms, one has to deal with a delicate tradeoff between efficiency, robustness, and accuracy—requirements leading to various approaches with many design choices. In this contribution, we introduce a Python package called *Sync Toolbox* that provides open-source reference implementations for full-fledged music synchronization pipelines and yields state-of-the-art alignment results for a wide range of Western music. Using suitable feature representations and cost measures, the toolbox's core technology is based on *dynamic time warping* (DTW), which brings the feature sequences into temporal correspondence. To account for efficiency, robustness and, accuracy, our toolbox integrates and combines techniques such as multiscale DTW (MsDTW) (Müller et al., 2006; Salvador & Chan, 2004), memory-restricted MsDTW (MrMsDTW) (Prätzlich et al., 2016), and high-resolution music synchronization (Ewert et al., 2009). While realizing a complete system with presets that allow users to reproduce research results from the literature, our toolbox also provides well-documented functions for all basic building blocks required for feature extraction and alignment. Furthermore, the toolbox contains example code for visualizing, sonifying, and evaluating synchronization results, thus deepening the understanding of the techniques and data.

Statement of Need

The task of finding an alignment between two feature sequences has received a large amount of research interest in the past, in the context of MIR and beyond. In the music domain, alignment techniques are central for applications such as score following, content-based retrieval, automatic accompaniment, and performance analysis (Arzt, 2016; Müller, 2015). Beyond these classical applications, alignment techniques have gained in importance in view of recent data-driven machine learning techniques. In particular, music synchronization has shown the potential for facilitating data annotation, data augmentation, and model evaluation. To be more specific, for certain types of music one often has a score-like symbolic representation that explicitly encodes information such as note events, measure positions, lyrics, and other

types of metadata. Furthermore, music experts often provide their harmonic, structural, or rhythmic analyses using such symbolic reference representations. Music synchronization techniques then allow for (semi-)automatically transferring these manually generated annotations from the reference to other symbolic or audio representations. This is particularly beneficial for music, where one has many recorded performances of a given piece. Thus, using music synchronization techniques, one may simplify the annotation process and substantially increase the number of annotated training and test versions. For example, in [Zalkow et al. \(2017\)](#), a multi-version approach for transferring measure annotations between music recordings (Wagner operas) is described. The “Schubert Winterreise Dataset” provides another example where automated techniques were applied to transfer measure, chord, local key, structure, and lyrics annotations ([Weiß et al., 2021](#)). Including nine performances (versions) of Schubert’s song cycle, this cross-version dataset was used in [Weiß et al. \(2020\)](#) for training and evaluating data-driven approaches for local key estimation, where the different dataset splits across songs and performances provided new insights into the algorithms’ generalization capabilities.

Being a central task, there are a many software packages for sequence alignment of general time series. In the audio domain, the *librosa* Python package by [McFee et al. \(2015\)](#) offers a basic DTW-based pipeline for synchronizing music recordings. Since the complexity of alignment techniques such as DTW is proportional to the product of the feature sequences’ lengths, runtime and memory requirements become issues when dealing with long feature sequences. Using a fast online time warping (OLTW) algorithm as described by [Dixon & Widmer \(2005\)](#), the software¹ (Music Alignment Tool CHeat) allows for an efficient alignment of audio files. While being efficient, such online approaches are prone to local deviations in the sequences to be aligned. An efficient yet robust alternative is offered by offline procedures based on multiscale strategies such as MsDTW ([Müller et al., 2006](#); [Salvador & Chan, 2004](#)). The recent Python package *linmdtw*² contains an implementation of MsDTW as well as a linear memory DTW variant described in [Tralie & Dempsey \(2020\)](#). Another important issue in music synchronization is the temporal accuracy of the alignments, which may be achieved by considering additional local cues such as onset features ([Ewert et al., 2009](#)). Improving the accuracy, however, often goes along with an increase in computational complexity and a decrease in overall robustness.

With our *Sync Toolbox*, we offer a Python package that provides all components needed to realize a music synchronization pipeline that is robust, efficient, and accurate. First, to account for robustness and efficiency, it implements the memory-restricted MsDTW approach from [Prätzlich et al. \(2016\)](#) as its algorithmic core component. Second, to account for accuracy, it integrates the high-resolution strategy from [Ewert et al. \(2009\)](#) in the finest MsDTW layer. Third, the toolbox contains all feature extractions methods (including chroma and onset features) needed to reproduce the results from the research literature. Fourth, we also provide functions required for quantitative and qualitative evaluations (including visualization and sonification methods). Even though it overlaps the previously mentioned software (e.g., *librosa* and *linmdtw*), the *Sync Toolbox* provides, for the first time, an open-source Python package for offline music synchronization that produces state-of-the-art alignment results regarding efficiency and accuracy. Thus, with the publicly available and well-documented *Sync Toolbox*, we hope to fill a gap between theory and practice for an important MIR task, while providing a useful pre-processing, annotation, and evaluation tool for data-driven machine learning.

Design Choices

When we designed the *Sync Toolbox*, we had a number of different objectives in mind. First, we tried to keep a close connection to the research articles by [Ewert et al. \(2009\)](#) and [Prätzlich](#)

¹<http://www.eecs.qmul.ac.uk/~simond/match/>

²<https://github.com/ctralie/linmdtw>

et al. (2016). Second, we reimplemented and included all required components (e.g., feature extractors, DTW), even though such basic functionality is also covered by other packages such as librosa and linmdtw. This way, along with a specification of meaningful variable presets, the Sync Toolbox provides reference implementations that can exactly reproduce previously published research results and experiments. Third, we followed many of the design principles suggested by librosa (McFee et al., 2015), which allows users to easily combine the different Python packages. The Sync Toolbox code along with API documentation is hosted in a publicly available GitHub repository.³ Finally, we included the synctoolbox package in the Python package index PyPI, which makes it possible to install synctoolbox with the standard Python package manager pip.⁴

Acknowledgements

The synctoolbox package builds on results, material, and insights that have been obtained in close collaboration with different people. We would like to express our gratitude to former and current students, collaborators, and colleagues who have influenced and supported us in creating this package, including Vlora Arifi-Müller, Michael Clausen, Sebastian Ewert, Christian Fremerey, and Frank Kurth. We also thank the German Research Foundation (DFG) for various research grants that allowed us for conducting fundamental research in music processing (in particular, MU 2686/7-2, DFG-MU 2686/14-1). The International Audio Laboratories Erlangen are a joint institution of the Friedrich-Alexander-Universität Erlangen-Nürnberg (FAU) and Fraunhofer Institute for Integrated Circuits IIS.

References

- Arzt, A. (2016). *Flexible and robust music tracking* [PhD thesis]. Universität Linz.
- Dixon, S., & Widmer, G. (2005). MATCH: A music alignment tool chest. *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 492–497. <https://doi.org/10.5281/zenodo.1416951>
- Ewert, S., Müller, M., & Grosche, P. (2009). High resolution audio synchronization using chroma onset features. *Proceedings of IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, 1869–1872. <https://doi.org/10.1109/icassp.2009.4959972>
- McFee, B., Raffel, C., Liang, D., Ellis, D. P. W., McVicar, M., Battenberg, E., & Nieto, O. (2015). Librosa: Audio and music signal analysis in Python. *Proceedings the Python Science Conference*, 18–25. <https://doi.org/10.25080/Majora-7b98e3ed-003>
- Müller, M. (2015). *Fundamentals of music processing* [Monograph]. Springer Verlag. <https://doi.org/10.1007/978-3-319-21945-5>
- Müller, M., Mattes, H., & Kurth, F. (2006). An efficient multiscale approach to audio synchronization. *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 192–197.
- Prätzlich, T., Driedger, J., & Müller, M. (2016). Memory-restricted multiscale dynamic time warping. *Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, 569–573. <https://doi.org/10.1109/ICASSP.2016.7471739>

³<https://github.com/meinardmueller/synctoolbox>

⁴<https://pypi.org/project/synctoolbox>

- Salvador, S., & Chan, P. (2004). FastDTW: Toward accurate dynamic time warping in linear time and space. *Proceedings of the KDD Workshop on Mining Temporal and Sequential Data*.
- Tralie, C. J., & Dempsey, E. (2020). Exact, parallelizable dynamic time warping alignment with linear memory. *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 462–469. <https://doi.org/10.5281/zenodo.4245470>
- Weiß, C., Schreiber, H., & Müller, M. (2020). Local key estimation in music recordings: A case study across songs, versions, and annotators. *IEEE/ACM Transactions on Audio, Speech & Language Processing*, 28, 2919–2932. <https://doi.org/10.1109/TASLP.2020.3030485>
- Weiß, C., Zalkow, F., Arifi-Müller, V., Müller, M., Koops, H. V., Volk, A., & Grohgan, H. G. (2021). Schubert Winterreise dataset: A multimodal scenario for music analysis. *ACM Journal on Computing and Cultural Heritage (JOCCH)*, 14(2), 25:1–18. <https://doi.org/10.1145/3429743>
- Zalkow, F., Weiß, C., Prätzlich, T., Arifi-Müller, V., & Müller, M. (2017). A multi-version approach for transferring measure annotations between music recordings. *Proceedings of the AES International Conference on Semantic Audio*, 148–155.

An Efficient Multiscale Approach to Audio Synchronization

Meinard Müller Henning Mattes Frank Kurth

Department of Computer Science, University of Bonn

Römerstraße 164, 53117 Bonn, Germany

meinard@cs.uni-bonn.de, henning1000@gmx.de, frank@cs.uni-bonn.de

Abstract

We present an efficient and robust multiscale DTW (Ms-DTW) approach to music synchronization for time-aligning CD recordings of different interpretations of the same piece. The general strategy is to recursively project an alignment path computed at a coarse resolution level to the next higher level and then to refine the projected path. As main contributions, we address several crucial issues including the design and specification of robust and scalable audio features, suitable local cost measures, MsDTW levels, constraint regions, as well as sampling rate adaptation and structural enhancement strategies. Extensive experiments on Western classical music show that our MsDTW-based algorithm yields the same alignment result as the classical DTW-based strategy while significantly reducing the running time and memory requirements. Even for pieces of a duration of 10 to 15 minutes, the alignment (based on previously extracted feature sequences) can be computed in less than a second.

Keywords: audio synchronization, alignment, multiscale, chroma feature

1. Introduction

For one and the same piece of music, there often exists a large number of CD recordings representing different interpretations by various musicians. In particular for Western classical music, these interpretations may exhibit considerable deviations in tempo, note realizations, dynamics, and instrumentation. For example, a music database may contain for Beethoven's Fifth Symphony some interpretations by Karajan and Bernstein, some historical recordings by Furthwängler and Toscanini, Liszt's piano transcription of Beethoven's Fifth played by Sherrakov and Glenn Gould, or some synthesized version of a corresponding MIDI file. Then, the goal of *audio synchronization* is to automatically generate *alignments* between corresponding note events between different interpretations. These alignments can be used to jump freely between different audio recordings, thus affording efficient and convenient music browsing.

In the last few years, several alignment strategies have been proposed, see, e.g., [1, 2, 3, 4, 5, 6] and the references therein. Most of these approaches rely on some variant of dynamic time warping (DTW). However, due to the quadratic time and space complexity, DTW-based strategies become infeasible for long pieces. To reduce the computational cost, Salvador et al. [7] propose for general time series a multiscale DTW (MsDTW) approach that recursively projects an alignment path computed at a coarse resolution level (using coarse features, e.g., obtained by averaging and downsampling) to the next higher level and then refines the projected path. One hazard with this approach is that an incorrect alignment on a low resolution level propagates to higher levels resulting in erroneous alignment results. This hazard is fostered by the fact that coarsening the features can lead to heavily deteriorated cost matrices, as is also illustrated by Fig. 4 (a)-(c). Dixon et al. [1] describe a linear time DTW approach based on forward path estimation. Further related work will be discussed in the respective sections.

In this paper, we describe an MsDTW approach to efficient as well as robust audio synchronization. In Sect. 2, the general ideas of MsDTW are summarized. In Sect. 3, we then propose solutions to several crucial issues of MsDTW including the design of robust and scalable audio features, specification of suitable local cost measures, adaptation strategies for the feature sampling rate, determination of MsDTW resolution levels and constraint regions, as well as enhancement strategies of DTW cost matrices. We then report on our extensive experiments based on a wide range of classical music demonstrating the practicability of our algorithm. Furthermore, the synchronization results have been integrated in our audio player [8] and sonified for evaluation, see Sect. 4. For some audio demos we refer to www-mmdb.iai.uni-bonn.de/projects/MsDTWsync.

2. General Multiscale Approach

In this section, we summarize the main ideas of classical DTW and MsDTW. Let $X := (x_1, x_2, \dots, x_N)$ and $Y := (y_1, y_2, \dots, y_M)$ be two feature sequences with $x_n, y_m \in \mathcal{F}$, $n \in [1 : N]$, $m \in [1 : M]$, where $\mathcal{F}$ denotes a suitable feature space. Furthermore, let $c : \mathcal{F} \times \mathcal{F} \rightarrow \mathbb{R}$ denote a *local cost measure* on $\mathcal{F}$. The resulting $(N \times M)$ -cost matrix C is defined by $C(n, m) := c(x_n, y_m)$.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page.

© 2006 University of Victoria

2.1. Classical DTW

DTW is a well-known technique to align X and Y with respect to the cost measure c . Recall that a *warping path* is a sequence $p = (p_1, \dots, p_L)$ with $p_\ell = (n_\ell, m_\ell) \in [1 : N] \times [1 : M]$ for $\ell \in [1 : L]$ satisfying $1 = n_1 \leq n_2 \leq \dots \leq n_L = N$ and $1 = m_1 \leq m_2 \leq \dots \leq m_L = M$ (boundary and monotonicity condition), as well as $p_{\ell+1} - p_\ell \in \{(1, 0), (0, 1), (1, 1)\}$ (step size condition). The total cost of p is defined as $\sum_{\ell=1}^L c(x_{n_\ell}, y_{m_\ell})$. Then, an optimal warping path between X and Y is given by a warping path p^* having minimal total cost among all possible warping paths. To determine such an optimal path, one recursively computes an $(N \times M)$ -matrix D , where the matrix entry $D(n, m)$ is the total cost of an optimal path between $(x_1, \dots, x_n)$ and $(y_1, \dots, y_m)$. In the following, we denote a matrix entry $D(n, m)$ as *cell*. Introducing an additional weight vector $(w_d, w_h, w_v) \in \mathbb{R}^3$ yields the recursive definition

$$D(n, m) := \min \begin{cases} D(n-1, m-1) + w_d \cdot c(x_n, y_m), \\ D(n-1, m) + w_h \cdot c(x_n, y_m), \\ D(n, m-1) + w_v \cdot c(x_n, y_m), \end{cases}$$

for $n, m > 1$. Furthermore, $D(n, 1) := \sum_{k=1}^n w_h \cdot c(x_k, y_1)$ for $n > 1$, $D(1, m) := \sum_{k=1}^m w_v \cdot c(x_1, y_k)$ for $m > 1$, and $D(1, 1) := c(x_1, y_1)$. Note that in the unweighted case $(w_d, w_h, w_v) = (1, 1, 1)$ one has a preference of the diagonal alignment direction, since one diagonal step (cost of one cell) corresponds to the combination of one horizontal and one vertical step (cost of two cells). To counterbalance this preference, we chose $(w_d, w_h, w_v) = (2, 1.5, 1.5)$ (still slightly favoring the diagonal direction). Now, p^* can be derived from D in some linear fashion, see [9] for details. The overall computational cost is proportional to the number of cells $D(n, m)$ to be computed during this process leading to a time and space complexity of $O(NM)$.

2.2. Multiscale DTW (MsDTW)

To speed up DTW computations, one common strategy is to impose a global *constraint region* $R \subseteq [1 : N] \times [1 : M]$ on the possible warping paths (e.g., Itakura parallelogram, Sakoe-Chiba band), thus limiting the number of cells needed to be computed, see [9]. However, the usage of global constraint regions is problematic, since the optimal warping path may leave the specified region. In other words, let p_R^* denote the optimal warping path with respect to R , then p_R^* may differ from p^* , see Fig. 1. A second strategy is to reduce the feature sampling rate (also referred to as dimensionality reduction or data abstraction), thus reducing the lengths N and M of the sequences to be synchronized. However, the resulting optimal warping path becomes increasingly inaccurate or even completely useless as the resolution decreases, see Fig. 4 for an illustration. MsDTW employs these two strategies in some iterative fashion to generate data-dependent constraint regions. We summarize

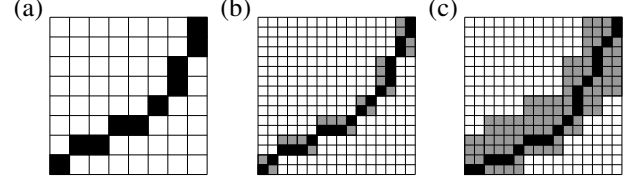


Figure 1. (a) Optimal warping path p_2^* on Level 2. (b) Optimal warping path p_R^* with respect to the constraint region R obtained by projecting path p_2^* to Level 1. (Here, p_R^* does not coincide with the (unconstrained) optimal warping path p^* .) (c) Optimal warping path $p_{R^\delta}^*$ using an increased constraint region $R^\delta \supset R$ with $\delta = 2$. Here, $p_{R^\delta}^* = p^*$.

the main ideas and refer to [7] for details. For a similar approach, applied to melody alignment we refer to [10].

Let $X_1 := X$ and $Y_1 := Y$ be the sequences to be synchronized having lengths $N_1 := N$ and $M_1 = M$, respectively. It is the objective to compute an optimal warping path p^* between X_1 and Y_1 . The highest resolution level will also be referred to as Level 1. By reducing the feature sampling rate by a factor $f_2 \in \mathbb{N}$, one obtains sequences X_2 of length $N_2 := N_1/f_2$ and Y_2 of length $M_2 := M_1/f_2$. (Here, we assume that f_2 divides N_1 and M_1 , which can be achieved, e.g., by suitably padding X_1 and Y_1 .) Next, one computes an optimal warping path p_2^* between X_2 and Y_2 on the resulting resolution level (Level 2). This path is projected onto Level 1 and there defines a constraint region R . Note that R consists of $L_2 \times f_2^2$ cells, where L_2 denotes the length of p_2^* . Finally, an optimal alignment path p_R^* relative to R is computed. We say that this procedure is *successful*, if $p^* = p_R^*$. The overall number of cells to be computed in this procedure is $N_2 M_2 + L_2 \cdot f_2^2$, which is generally much smaller than the total number $N_1 M_1$ of cells on Level 1. In an obvious fashion, this procedure can be recursively applied by introducing further levels of decreasing resolution. For a complexity analysis, we refer to [7].

The constraint path p_R^* may not coincide with the optimal path p^* . To alleviate this problem, one can increase the constraint region R —at the expense of efficiency—by adding δ cells to the left, right, top, and bottom of every cell in R for some parameter $\delta \in \mathbb{N}$. The resulting region R^δ will be referred to as δ -neighborhood of R , see Fig. 1.

3. MsDTW Audio Synchronization

The multiscale approach to DTW constitutes a general framework to speed up computations. In view of robustness, efficiency, and practicability of the resulting synchronization procedure, however, one has to specify several important parameters. In this section, we describe and discuss the design choices for our audio synchronization algorithm and report on our experiments. To avoid incorrect alignments even in extreme situations we sketch a strategy for structural enhancement of the cost matrix, which works even at very low resolution levels, see Sect. 3.5.

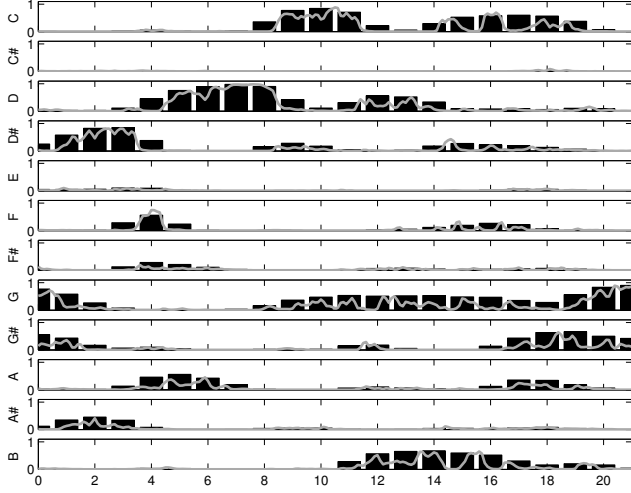


Figure 2. The first 21 seconds of Bernstein’s interpretation of Beethoven’s Fifth Symphony. The light curves represent the local chroma energy distributions (10 features per second). The dark bars represent the CENS features (1 feature per second).

3.1. Audio Features

Previous studies have shown that chroma-based audio features are well suited to characterize harmony-based music, see [11, 2, 12]. Representing the spectral energy of each of the 12 traditional pitch classes of the equal-tempered scale, chroma features do not only account for the close octave relationship in both melody and harmony as it is prominent in Western music, but also introduce a high degree of robustness to variations in timbre and articulation [11]. Since different interpretations typically exhibit significant variations in such parameters, chroma-based features are well-suited for audio synchronization leading to robust and accurate alignments, see [2].

There are various ways to compute chroma features, e. g., by suitably pooling spectral coefficients obtained from some short-time Fourier transform [11] or by suitably summing up pitch subbands obtained as output after applying some pitch-based filter bank [12]. For details, we refer to the literature. In our implementation, we convert an audio signal into a sequence $X = (x_1, x_2, \dots, x_N)$ of normalized 12-dimensional feature vectors $x_n \in [0, 1]^{12}$, $1 \leq n \leq N$, which express the local energy distribution in the 12 chroma classes. Here, we use a feature sampling rate of 10 Hz, where each feature vector x_n covers 200 ms of audio with an overlap of 100 ms. This rate, which will constitute the finest resolution level, turns out to be sufficient in view of our intended applications.

For our multiscale approach, we need a computationally inexpensive way to adjust the feature resolution. Instead of simply modifying the analysis window in the chroma computation, we introduce a second, much larger statistics window (covering w consecutive chroma vectors) and consider *short-time statistics* of the chroma energy distribution

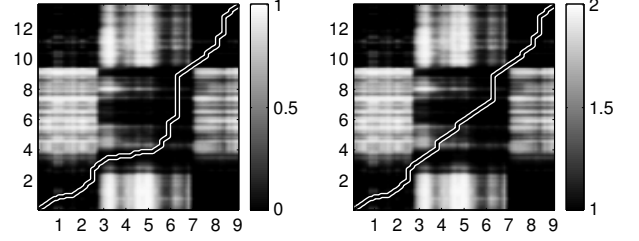


Figure 3. Optimal warping path based on the cost measure c_α with $\alpha = 0$ (left) and $\alpha = 1$ (right).

over this window. This again results in a sequence of 12-dimensional vectors, which is then downsampled by a factor of q and renormalized with respect to the Euclidean norm (hence being invariant towards variations in dynamics). For example, $w = 41$ and $q = 10$ yields a feature sampling rate of 1 Hz, where each feature vector represents information of the audio signal within a window of 4200 ms. The resulting feature sequence will be referred to as CENS(w, q) sequence (**C**hroma **E**nergy **N**ormalized **S**tatistics). Similar features have been applied in the audio matching scenario and are described in detail in [12]. Note that by modifying the parameters w and q , one can adjust the feature granularity and sampling rate without repeating the cost-intensive chroma computations. Fig. 2 shows the resulting chroma and CENS(41, 10) features sequences extracted from a Bernstein interpretation of Beethoven’s Fifth.

3.2. Local Cost Measure

The normalized chroma and CENS features are elements in $\mathcal{F} := [0, 1]^{12}$. To compare two features $x, y \in \mathcal{F}$, we use the cost measure $c_\alpha : \mathcal{F} \times \mathcal{F} \rightarrow [0, 1] + \alpha$ defined by $c_\alpha(x, y) := 1 - \langle x, y \rangle + \alpha$ for some offset $\alpha \in \mathbb{R}_{\geq 0}$. (Note that $\langle x, y \rangle$ is the cosine of the angle between x and y , since x and y are normalized.) The offset α is introduced for the following reason. Audio recordings often contain long segments of little variance such as pauses or sustained chords. This leads to rectangular regions in the cost matrix, also referred to as *plateaus*. If $\alpha = 0$, all cells within a plateau reveal some cost close to zero. Being close to zero, there may be large, more or less random *relative* differences among the costs of these cells (e. g., one cell has cost 0.01 and another one 0.001). As a result, one obtains an uncontrollable run of the optimal warping path within a plateau. By increasing α , the relative differences decrease, whereas the absolute differences are retained unchanged (e. g., for $\alpha = 1$, one cell has cost 1.01 and another one 1.001). As a consequence, the effect of the weight vector (w_d, w_h, w_v) as introduced in Sect. 2.1 becomes more dominant. For the parameters $(w_d, w_h, w_v) = (2, 1.5, 1.5)$ and $\alpha = 1$, which have turned out to be suitable in our experiments, the diagonal direction receives a slight but stable preference in plateau regions, see Fig. 3. In the following, we set $c := c_1$.

Table 1. Specification of the features used in our MsDTW audio synchronization system.

Level	Feature	Resolution	Factor
1	Chroma	10 Hz	-
2	CENS(41, 10)	1 Hz	10
3	CENS(121, 30)	1/3 Hz	3
4	CENS(271, 90)	1/9 Hz	3

Table 2. Total running time (for 363 synchronization pairs) against the number of levels used in the MsDTW algorithm based on the features of Table 1.

Run time \ Levels	1	1 - 2	1 - 3	1 - 4
t_{Cells} [s]	1434.0	78.5	67.6	67.2
t_{CENS} [s]	0.0	16.5	24.7	29.8
t_{MsDTW} (absolute) [s]	1434.0	95.0	92.3	97.0
t_{MsDTW} (relative) [%]	100	6.62	6.44	6.76

3.3. Resolution Levels and δ -Neighborhood

As described in Sect. 3.1, the chroma features at a time resolution of 10 Hz constitute the basic resolution (Level 1) of our audio synchronization. The CENS features, which can be efficiently derived from the chroma features, are used at the lower resolution levels. To determine a suitable number of levels as well as the resolutions at each level used in the MsDTW, we conducted comprehensive experiments. We report on some experiments that are based on the features indicated by Table 1. For our tests, we used 363 pairs of CD recordings, where each pair corresponds to two different interpretations of the same piece. The recordings have a duration of 3 to 20 minutes and cover a wide range of classical music, see Table 5 for examples. In a preprocessing step, we computed and stored the chroma features of all recordings. For each test series, we performed an audio synchronization for all 363 pairs. The algorithms have been implemented in C/C++ and tests were run on an Intel Pentium M, 1.7 GHz, 1 GByte RAM, under Windows XP.

In one test series, we used classical DTW on Level 1 and MsDTW based on the first two, three, and four levels. In all these cases we used $\delta = 30$, see the discussion below. The resulting running times are shown in Table 2. The DTW-based strategy required 1434.0 seconds to synchronize all of the 363 pairs. In contrast, the two-level MsDTW-based strategy required $t_{\text{CENS}} = 16.5$ seconds to compute the CENS(41, 10)-features used for Level 2, and $t_{\text{Cells}} = 78.5$ seconds to compute all required cells on the two levels, amounting to a total running time of $t_{\text{MsDTW}} = 95.0$ seconds—only 6.62% of the running time of classical DTW. Similarly, it took 92.3 and 97.0 seconds when using the three-level and four-level MsDTW-based strategy, respectively. In particular, we obtained the lowest total running time for three levels. Using a fourth level indeed further decreased the total number of cells to be evaluated, but the computational overhead due to the additional CENS

Table 3. Running time, absolute and relative error against the size of the δ -neighborhood and adaptive neighborhood based on a three-level MsDTW.

Run time \ δ	0	10	20	30
t_{Cells} [s]	29.3	41.6	54.6	67.6
t_{CENS} [s]	25.0	24.7	24.6	24.7
t_{MsDTW} (absolute) [s]	54.3	66.3	79.2	92.3
t_{MsDTW} (relative) [%]	3.79	4.62	5.52	6.44
Error (absolute)	92	27	6	0
Error (relative) [%]	25.34	7.44	1.65	0

features needed at Level 4 deteriorated the overall result. Also introducing additional intermediate levels had only a marginal effect on the overall running time. We therefore use three multiscale levels as default setting in our audio synchronization system.

For our evaluation, we use the DTW-alignment (unconstrained warping path) as ground truth and check whether the MsDTW-alignment (constrained warping path) entirely coincides with the DTW-alignment (then MsDTW is called successful, see Sect. 2.2) or not (then MsDTW produces an error). In another test series, we evaluated different δ -neighborhoods for a three-level MsDTW. Table 3 shows the performance for $\delta = 0, 10, 20, 30$. For $\delta = 0$, in 92 of the 363 cases the MsDTW-based strategy was not successful (corresponding to an error rate of 25.34%). Increasing δ leads to an increase of the running time and a decrease of the error rate. For $\delta = 30$, all 363 pairs have been successfully aligned by the MsDTW strategy. The running time to evaluate the cells has increased from $t_{\text{Cells}} = 29.3$ seconds ($\delta = 0$) to $t_{\text{Cells}} = 67.6$ seconds ($\delta = 30$)—a moderate increase with regard to the total running time. For our audio synchronization system, we therefore use $\delta = 30$ as default.

3.4. Experimental Results

In this section, we discuss some representative experimental results based on the three-level MsDTW with $\delta = 30$. For these parameters, as was reported above, the audio synchronization has been successful for all of the 363 pairs of audio recordings. Table 5 shows a selection of these recordings including complex orchestral pieces having a duration of 3 to 20 minutes. Some of the interpretations significantly differ in tempo, instrumentation, and articulation. For example, Sacchi’s interpretation of Schubert’s Unfinished is much faster (817 seconds) than Solti’s interpretation (951 seconds). Or, there is an orchestral as well as a piano version (piano transcription) of Beethoven’s Fifth and Wagner’s Prelude, respectively. Furthermore, the Mae interpretation of Vivaldi’s spring includes many additional ornamentations, which can not be found in the Zukerman interpretation. In view of such significant variations, the CENS(41, 10) features as used on Level 2 constitute a good

Table 4. Performance of the implementation of our MsDTW-based audio synchronization algorithm for a representative selection of recordings using three-levels and $\delta = 30$. For each level, the total number of cells (DTW) as well as the number of cells to be evaluated by MsDTW are indicated. The last three columns show a comparison of the DTW and MsDTW running times.

Synchronization				Number of cells to be evaluated by DTW and MsDTW in each level						Total run time[s]			
Recording 1		Recording 2		Level 1			Level 2			Level 3	Levels 1-3		
Id 1	Length [s]	Id 2	Length [s]	DTW	MsDTW	[%]	DTW	MsDTW	[%]	DTW	DTW	MsDTW	[%]
Beet9Bern	1144.9	Beet9Kar	1054.8	120808050	2117929	1.75	1209030	17657	1.46	134464	31.18	1.08	3.46
RavBolAbb	862.5	RavBolOza	901.0	77737897	1694610	2.18	778426	14121	1.81	86688	20.04	0.80	3.99
Schub8Sac	817.3	Schub8Sol	950.8	77736075	1704150	2.19	777918	14322	1.84	86541	20.01	0.80	3.99
Dvo9Maaz	704.2	Dvo9Franc	710.7	50068752	1356308	2.71	501255	11295	2.25	55695	12.85	0.60	4.67
WagPreArm	595.0	WagPreGould	576.9	34337270	1124029	3.27	343892	9387	2.73	38407	8.85	0.48	5.42
Beet5Bern	519.0	BeLi5Sher	444.1	23068056	923444	4.00	231400	7721	3.34	25926	5.84	0.37	6.34
Beet5Bern	519.0	Beet5Kar	443.9	23052480	923028	4.00	230880	7716	3.34	25752	5.87	0.38	6.47
SchosJazzCha	223.6	SchosJazzYab	193.6	4335306	394259	9.09	43456	3291	7.57	4875	1.11	0.15	13.51
VivSpringMae	192.5	VivSpringZuk	218.9	4217940	389034	9.22	42267	3261	7.72	4745	1.07	0.13	12.15
ElgEnigDel	91.8	ElgEnigSino	93.1	856508	167699	19.58	8648	1404	16.23	992	0.22	0.06	27.27

Table 5. Some audio recordings (with identifier) contained in our test database comprising 33 hours of audio.

Composer / piece / interpreter	Identifier
Beethoven / Symph. 5, Op. 67, 1st mov. / Bernstein	Beet5Bern
Beethoven / Symph. 5, Op. 67, 1st mov. / Karajan	Beet5Kar
Beeth. (Liszt) / Symph. 5, Op. 67, 1st mov. (piano) / Sherbakov	BeLi5Sher
Beethoven / Symph. 9, Op. 125, 4th mov. / Bernstein	Beet9Bern
Beethoven / Symph. 9, Op. 125, 4th mov. / Karajan	Beet9Kar
Dvorak / Symph. 9, Op. 95, 1st mov. / Francis	Dvo9Franc
Dvorak / Symph. 9, Op. 95, 1st mov. / Maazel	Dvo9Maaz
Elgar / Op. 36, Andante (Enigma) / Del Mar	ElgEnigDel
Elgar / Op. 36, Andante (Enigma) / Sinopoli	ElgEnigSino
Ravel / Bolero / Abbado	RavBolAbb
Ravel / Bolero / Ozawa	RavBolOza
Schubert / Symph. 8, D759, 1st mov. (Unfinished) / Sacchi	Schub8Sac
Schubert / Symph. 8, D759, 1st mov. (Unfinished) / Solti	Schub8Sol
Shostakovich / Jazz Suite No. 2, 6th mov. (Waltz) / Chailly	SchosJazzCha
Shostakovich / Jazz Suite No. 2, 6th mov. (Waltz) / Yablonsky	SchosJazzYab
Vivaldi / RV 269 (Spring), 1st mov. / Mae	VivSpringMae
Vivaldi / RV 269 (Spring), 1st mov. / Zukerman	VivSpringZuk
Wagner / Meistersinger Prelude / Armstrong	WagPreArm
Wagner / Meistersinger Prelude (piano) / Gould	WagPreGould

compromise between reasonable feature resolution, computational efficiency and robustness of the alignment result. (Actually, in situations where one has significant differences in note realizations—e. g., one interpreter plays a sustained chord whereas another interpreter plays an ornamentation—the alignment on a finer resolution level may not even be semantically meaningful.)

The increase in performance of our MsDTW-based audio synchronization in comparison to a classical DTW-based approach is illustrated by Table 4. For example, to synchronize ‘Beet9Bern’ and ‘Beet9Kar’ only 1.75% of the cells on Level 1 have to be evaluated by MsDTW, decreasing the memory requirements roughly by a factor of 57. Here, note that the overall memory requirement is proportional to the maximal number of cells needed to be evaluated at some level. The number of additional cells needed to be computed at Levels 2 and 3 is comparatively small. The overall running time (including the CENS feature computation) to synchronize the two recordings (each version having a duration of almost 20 minutes) was 1.08 seconds—nearly thirty

times faster than classical DTW. Obviously, the relative savings increase with the lengths of the pieces.

3.5. Enhancing Cost Matrices

We have also conducted experiments with manually distorted and highly repetitive audio material, which constitutes an extreme scenario for MsDTW. As one example, we used the first 22 seconds of an Cabrera interpretation of Bach’s Toccata BWV 565, where the theme is repeated three times at three different octaves. We generated an audio file, referred to as ‘Bach12’, by concatenating four copies of this segment, resulting in 12 repetitions of the theme. We then generated a time-warped version of ‘Bach12’, referred to as ‘Bach12Warp’, by locally increasing and decreasing the tempo up to 50 percent. The cost matrix using CENS(41, 10) and the resulting optimal warping between ‘Bach12’ and ‘Bach12Warp’ is shown in Fig. 4 (a). Now, reducing the feature sampling rate leads to a heavily deteriorated cost matrix, where the repetitions cannot be resolved any longer. This, in turn, results in absurd optimal warping paths on the lower resolution levels, see Fig. 4 (b) and (c).

To alleviate this problem, we improve the structural properties of the cost matrix by incorporating contextual information into the local cost measure, see [13]. Intuitively, the idea is to enhance the diagonal path structure of the 2D cost matrix by applying a local 1D low-pass filter along the diagonals (having gradient (1, 1)). Note that this process only works if corresponding segments of the two audio recordings reveal the same tempo progression. To account for tempo differences in the two interpretations, the idea is to simultaneously filter along different directions (in our implementation we used eight different gradients in a neighborhood of (1, 1), which cover tempo variations of roughly −30 to +40 percent) and then to take the minimum over the filter outputs. The technical details of this approach are described in [13]. The effect of this enhancement strategy, which allows to reduce the feature sampling rate without completely destroying the structural properties of the cost

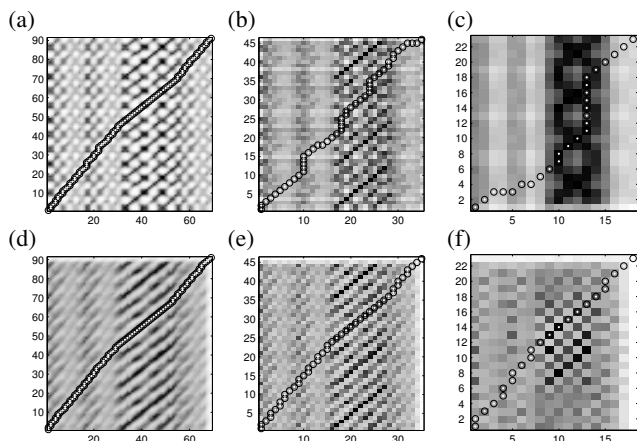


Figure 4. Optimal warping path between ‘Bach12’ and ‘Bach12Warp’ using CENS(41, 10) (a), CENS(81, 20) (b), and CENS(161, 40) (c). The paths are semantically incorrect for the two lower resolution levels. Locally filtering the cost matrix based on CENS(41, 10) (using eight different gradients) and downsampling by a factor 1 (d), 2 (e), and 4 (f) leads to correct optimal warping paths even on the lower resolution levels.

matrix, is illustrated by Fig. 4 (d)-(f). Even at the lowest resolution level (obtained by filtering with an averaging filter of length 8 and downsampling by a factor of 4), the optimal warping path leads to the ‘correct’ alignment. In practice, one has to assess the trade-off between increased computational complexity caused by the additional filtering step and the boost of robustness and confidence due to the structural enhancement.

4. Conclusions and Future Work

In this paper, we presented an efficient MsDTW-based approach to audio synchronization yielding stable alignments even in the presence of significant variations. One main application of audio synchronization is to allow for efficient music browsing. To this end, we integrated the alignment results into the SyncPlayer framework [8] (an audio player with additional retrieval and browsing functionality). During playback of a CD recording, SyncPlayer allows the user to directly jump to the corresponding position of any other interpretation of the same piece. In view of this application, an alignment at a resolution of 10 Hz is more than sufficient. To evaluate the absolute alignment quality achieved by our system, we conducted the following experiment. Based on the alignment result of two recordings, we time-warped the second recording to run synchronously to the first recording. For the warping, we used an overlap-add technique based on waveform similarity (WSOLA) as described in [14]. We then produced a stereo audio file containing the mono version of the original first recordings in one channel and a mono version of the time-warped second recording in the other channel. Listening to this stereo audio file exhibits even small temporal deviations of less than 100 ms be-

tween corresponding note events. Some representative audio examples can be found at www-mmdb.iai.uni-bonn.de/projects/MsDTWsync. Based on the synchronization result, it is possible to continuously blend from one interpretation to another one. It would be an interesting task to employ synchronization techniques for mixing and morphing different interpretations. As another task, based on our chroma-based alignment, one can apply more refined techniques and features to further enhance the alignment. Finally, we plan to develop strategies to automatically detect critical audio segments (e. g., segments leading to plateaus), where one can then locally switch between various synchronization strategies (e. g., locally activating and deactivating the enhancement strategy).

References

- [1] Simon Dixon and Gerhard Widmer. Match: A music alignment tool chest. In *Proc. ISMIR, London, GB, 2005*.
- [2] Ning Hu, Roger Dannenberg, and George Tzanetakis. Polyphonic audio matching and alignment for music retrieval. In *Proc. IEEE WASPAA, New Paltz, NY, October 2003*.
- [3] Meinard Müller, Frank Kurth, and Tido Röder. Towards an efficient algorithm for automatic score-to-audio synchronization. In *Proc. ISMIR, Barcelona, Spain, 2004*.
- [4] Christopher Raphael. A hybrid graphical model for aligning polyphonic audio with musical scores. In *Proc. ISMIR, Barcelona, Spain, 2004*.
- [5] Ferréol Soulez, Xavier Rodet, and Diemo Schwarz. Improving polyphonic and poly-instrumental music to score alignment. In *Proc. ISMIR, Baltimore, USA, 2003*.
- [6] Robert J. Turetsky and Daniel P.W. Ellis. Force-Aligning MIDI Syntheses for Polyphonic Music Transcription Generation. In *Proc. ISMIR, Baltimore, USA, 2003*.
- [7] S. Salvador and P. Chan. FastDTW: Toward accurate dynamic time warping in linear time and space. In *Proc. KDD Workshop on Mining Temporal and Sequential Data, 2004*.
- [8] Frank Kurth, Meinard Müller, David Damm, Christian Fremer, Andreas Ribbrock, and Michael Clausen. Syncplayer - an advanced system for content-based audio access. In *Proc. ISMIR, London, GB, 2005*.
- [9] Lawrence Rabiner and Biing-Hwang Juang. *Fundamentals Of Speech Recognition*. Prentice Hall, 1993.
- [10] Norman Adams, Daniela Marquez, and Gregory H. Wakefield. Iterative deepening for melody alignment and retrieval. In *Proc. ISMIR, London, GB, 2005*.
- [11] Mark A. Bartsch and Gregory H. Wakefield. Audio thumbnailing of popular music using chroma-based representations. *IEEE Trans. on Multimedia*, 7(1):96–104, 2005.
- [12] Meinard Müller, Frank Kurth, and Michael Clausen. Audio Matching via Chroma-based Statistical Features. In *Proc. ISMIR, London, GB, 2005*.
- [13] Meinard Müller and Frank Kurth. Enhancing similarity matrices for music audio analysis. In *Proc. ICASSP, 2006*.
- [14] W. Verhelst and M. Roelands. An overlap-add technique based on waveform similarity (WSOLA) for high quality time-scale modification of speech. In *Proc. ICASSP, 1993*.

ASAP: A DATASET OF ALIGNED SCORES AND PERFORMANCES FOR PIANO TRANSCRIPTION

Francesco Foscarin¹ Andrew McLeod² Philippe Rigaux¹
Florent Jacquemard^{1,3} Masahiko Sakai³

¹ CNAM, Paris, France

² EPFL, Lausanne, Switzerland

³ INRIA, Paris, France

⁴ Nagoya University, Nagoya, Japan

francesco.foscarin@cnam.fr, andrew.mcleod@epfl.ch, philippe.rigaux@cnam.fr
florent.jacquemard@inria.fr, sakai@i.nagoya-u.ac.jp

ABSTRACT

In this paper we present Aligned Scores and Performances (ASAP): a new dataset of 222 digital musical scores aligned with 1068 performances (more than 92 hours) of Western classical piano music. The scores are provided as paired MusicXML files and quantized MIDI files, and the performances as paired MIDI files and partially as audio recordings. Scores and performances are aligned with downbeat, beat, time signature, and key signature annotations. ASAP has been obtained thanks to a new annotation workflow that combines score analysis and alignment algorithms, with the goal of reducing the time for manual annotation. The dataset itself is, to our knowledge, the largest that includes an alignment of music scores to MIDI and audio performance data. As such, it is a useful resource for a wide variety of MIR applications, from those that target the complete audio-to-score Automatic Music Transcription task, to others that target more specific aspects (e.g., key signature estimation and beat or downbeat tracking from both MIDI and audio representations).

1. INTRODUCTION

As data-hungry deep learning models have become more ubiquitous in the field of MIR in recent years (e.g., [19, 22, 37]), large, well-annotated datasets have become increasingly important. Similar trends towards deep learning methods have been seen in related fields such as natural language processing [42] and computer vision [39]. For many tasks in these fields, large datasets can be automatically scraped from the web, and annotated quickly by non-experts (e.g., [34]). Unfortunately, the same can often not be said for tasks in MIR, for many reasons.

First, producing high-quality audio or MIDI data is a non-trivial task, requiring expert performers and expensive equipment that is not always available. Second, the availability of a high-quality digital ground truth is not guaranteed in many cases, particularly for tasks which require a

musical score¹, e.g., for the complete audio-to-score Automatic Music Transcription (AMT) task (see [1] for a recent overview of AMT). Finally, even in the case when both audio/MIDI data and high-quality digital ground truth scores are available, acquiring an alignment between the two is non-trivial. Automatic alignment methods (e.g., [30]) are often not robust enough to deliver highly reliable results and require time-consuming post-processing and cleaning by expert musicians for advanced data usage.

We introduce the Aligned Scores and Performances (ASAP) dataset², containing digital musical scores of Western classical piano pieces as both MusicXML and MIDI, aligned at the beat level with audio (from MAESTRO [17]) and MIDI recordings of over 1000 human performances. Regarding the three difficulties outlined above (data creation, digital ground truth availability, and recording-ground truth alignment), we use (1) publicly available MIDI and audio from expert human performances; (2) publicly-available musical scores scraped from the web; and (3) a new workflow to efficiently produce aligned beat, downbeat, key, and time signature annotations with minimal human correction required.

Although ASAP can be used for many tasks, it was designed with two categories specifically in mind: (1) complete audio-to-score AMT and (2) metrical structure-based tasks (e.g., beat and downbeat tracking, tempo estimation, metrical structure alignment, and rhythm quantization) of classical piano performance, from both audio and MIDI.

While a major goal of AMT is to convert an input audio recording into a form of human-readable music notation, the vast majority of AMT systems fall short of such an output. Rather, they convert the input audio recording into some sort of time-frequency representation: either a frame-based multi-pitch detection, where the presence of each pitch is estimated at each point in the input recording (e.g., [21]); or a note-based output such as a piano-roll or MIDI file, where notes are detected each with a pitch, an onset time, and an offset time (e.g., [16]).³ For these purposes, the ground truth only needs to contain some aligned



© Francesco Foscarin, Andrew McLeod, Philippe Rigaux, Florent Jacquemard, Masahiko Sakai. Licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0). **Attribution:** Francesco Foscarin, Andrew McLeod, Philippe Rigaux, Florent Jacquemard, Masahiko Sakai. “ASAP: a dataset of aligned scores and performances for piano transcription”, 21st International Society for Music Information Retrieval Conference, Montréal, Canada, 2020.

¹ Although PDF scores are sometimes available, and the field of Optical Music Recognition (see [3] for a recent overview) involves converting these into digital format, this conversion can add errors, and starting from a clean, digital score is generally better if available.

² <https://github.com/fosfrancesco/asap-dataset>

³ This can be post-processed into a musical score, as in the pipeline approach of [29], but such pipelines tend to add noise at each step.

pitch or note presence data—not a full musical score. Existing datasets such as MAPS [8] and the larger MAE-STRO [17] contain appropriate ground truths for this level of transcription (see Section 2).

In recent years, a few systems have been designed to output a more complete musical score directly (e.g., [5, 35]). While each of these works shows promise on this difficult task, neither includes time signatures or key signatures in their output. One [5] uses synthetically generated scores (and synthesized audio), and the other [35] uses audio synthesized from real scores. Since the eventual goal of AMT is a full transcription from human performance to musical score, a large dataset of non-synthetic human performances (containing the tempo deviations and timing intricacies of human performance, as well as real audio) is needed: synthetic data can be useful in the initial phase of model design, but human performance data is required for a truly reliable evaluation. ASAP provides such a dataset.

To our knowledge, no audio-to-score transcription system exists that requires fine-grained alignments between recordings and ground truths, perhaps due to a lack of available data. However, the use of data with even a coarse alignment has been shown to improve performance on the related task of monophonic note-based transcription [31], suggesting that the same should be true for audio-to-score AMT, if a large enough dataset of aligned recordings and scores was available. ASAP provides this time-alignment between the included recordings and ground truth scores.

Regarding metrical tasks, large audio datasets exist, especially for beat and downbeat tracking, enabling sophisticated systems to be trained (e.g., [2]). However, there is an absence of similarly-sized annotated datasets consisting of MIDI data, resulting in a noted lack of training data for metrical tasks from MIDI input (e.g., [25]). Even in the case of audio, much of the existing data is dance music or from other genres with relatively steady tempi compared to the classical piano music contained in ASAP (see Section 4.3 for an analysis of ASAP’s tempo changes).

We produce beat and downbeat annotations for every performance in ASAP with a novel workflow that exploits the precise metrical structure information available in a musical score. Projecting this onto each corresponding performance guarantees a robust means to identify the beat and downbeat positions. Working with MIDI allows us to overcome many difficulties found in similar approaches using only audio data (e.g., [32]). The workflow allows us to drastically reduce the time required for manual annotation.

2. RELATED WORK

Some public datasets similar to ASAP—containing combinations of musical scores, MIDI performances, and audio recordings, for AMT and/or beat tracking—already exist. In this Section, we describe those existing datasets in comparison to ASAP, highlighting specifically where ASAP addresses their deficiencies. Table 1 contains a summary of the largest of these datasets in comparison to ASAP. We first describe those containing musical performances (useful for AMT), and follow that with a brief discussion of available datasets for metrical structure-based tasks.

2.1 Performance datasets

There are two datasets which contain multiple performances of many different pieces. The Vienna 4x22 Piano Corpus [11] consists of 22 different performances of each of 4 different pieces, in both audio and MIDI format, aligned to a metrical grid. The CHARM Chopin Mazurka Project⁴ dataset contains many recordings of each of 49 different Mazurkas composed by Frédéric Chopin, although the original audio recordings are only referenced, and not available online (instead, many pre-calculated features are provided). While these datasets are valuable for investigating live performance deviations and comparisons between different performances of the same piece, they are not as useful for AMT, since they each consist of a small number of different pieces, leading to likely model overfitting (only 4 different pieces for Vienna 4x22, and only pieces by a single composer in the Mazurka dataset).

The Saarland Music Data (SMD) dataset [28] contains 50 synchronized audio and MIDI recordings of human performers playing a Disklavier piano. The files are not aligned with any musical scores or beat annotations, and the dataset’s size is somewhat small compared to other similar datasets. Likely because of its size, SMD has not been used for AMT in recent work to our knowledge.

CrestMuse PEDB [15] is a dataset based on audio recordings of multiple piano performances of around 100 unique pieces. However, the original audio recordings are not included. Rather, references to commercial CDs which can be purchased, and on which the recordings can be found are given. After a PDF application and pledge are filled out and submitted, access is granted to download the database in about a week. Provided in the dataset are MIDI files, whose notes have been hand-aligned to the referenced recordings; and digital musical scores in an XML-based format, to which the notes and beats of the MIDI files are aligned (using “deviation” XML tags in the score files). Since its initial release, some audio files have been added. However, these are different from the original score-aligned audio recordings, and in some cases are synthesized from MIDI performance. The difficulty of acquiring the audio recordings makes this database rarely used for audio-based tasks such as AMT.

The piano-midi dataset⁵ contains 324 quantized MIDI files whose tempo curves have been manually altered with the goal of becoming more human-like. The MAPS dataset [8] contains 210 of these MIDI files (of 119 different pieces), without key and time signatures, each paired with an audio file—some synthesized and some actual recordings. A-MAPS [41] later augmented MAPS with MIDI files containing key signature and time signature annotations. Since the MIDI data is generated from tempo-varied quantized MIDI, rather than actual performance, the MIDI files and recordings do not contain all of the timing variance that would be present in real performance: note onsets and offsets which lie on the same beat in the original musical score also occur simultaneously in the corresponding

⁴ <http://www.mazurka.org.uk/>

⁵ www.piano-midi.de

Dataset	Size		Performance		Quantized		Annotations		
	Total	Unique	Audio	MIDI	MIDI	Score	Alignment	Metrical	Key
MAPS [8]	269	52	Pseudo [†]	Pseudo	✓		Full	✓ [41]	✓ [41]
CrestMuse-PEDB [15]	411	≈100	Partial [†]	✓	✓	✓	Full	✓	✓
SUPRA [36]	478	≈430	Pseudo [†]	✓					
MAESTRO [17]	1282	≈430	✓	✓					
GTZAN [38]	1000	1000	✓					✓ [24]	Global
Ballroom [12]	685	685	✓					Beat [23]	
Hainsworth [14]	222	222	✓					Beat	
SMC [18]	217	217	✓					Beat	
ASAP	1068	222	520	✓	✓	✓	Beat	Beat	✓

Table 1. An overview of the most relevant datasets for AMT (top section) and metrical tasks (middle section), compared to ASAP (bottom). Alignment refers to the level of alignment between the quantized and performance data. MAPS consists of pseudo-live performance (quantized MIDI with manually altered tempo curves). [†]MAPS, SUPRA (fully) and CrestMuse-PEDB (partially) include synthesized audio (not real recordings).

MIDI and audio files. In real performance, such events only rarely occur simultaneously. Rather, small timing deviations introduce gaps, overlaps, and other timing variation (see e.g. [26], Figure 4), which are therefore missing (along with ornamentation such as trills, as well as performance errors). Although these datasets contain perfectly-aligned ground truth annotations (which has made MAPS a standard for AMT evaluation since its release), their modest size and the fact that they are not real live performance are drawbacks that we hope to address with ASAP.

The SUPRA dataset [36] contains 478 MIDI files of around 430 different pieces generated from an archive of piano performances in the form of physical piano rolls. SUPRA also contains synthesized audio recordings of each MIDI file, and labels each with a composer and title, but provides no metrical alignment of the pieces.

The MAESTRO dataset [17] contains 1282 real performances of around 430 different pieces from the Yamaha piano e-competition⁶. Each performance is available as a MIDI file and an audio recording with a fine alignment of around 3 ms. Metadata are available for each performance, including the composer and title of each. MAESTRO’s size, fine alignment with ground truth, and the fact that it is real performance have made it an excellent source of training and evaluation data for AMT from recording to piano-roll. However, MAESTRO does not contain any note-, beat-, or even piece-level alignment with digital musical scores, required for the complete audio-to-score AMT task (and which ASAP does contain).

2.2 Metrical structure datasets

For metrical structure-based tasks, from live performance MIDI data, annotated datasets from the previous section (particularly piano-midi and CrestMuse-PEDB) are typically used. However, they are relatively small (especially in terms of unique pieces), and piano-midi files in particular do not contain real live performance, as mentioned. For the same tasks from audio data, large annotated datasets exist, enabling sophisticated models to be designed and trained (e.g., [2]). The largest and most widely used (where

audio files are publicly available, including at least beat annotations) are: GTZAN [38] (1000 audio recordings of various genres with beat, downbeat, and 8th-note annotations), Ballroom [12] (685 audio recordings of ballroom dancing music with beat and downbeat annotations), Hainsworth [14] (222 audio recordings of Western music), and SMC [18] (217 audio recordings of Western music, specifically selected to be difficult for beat tracking). However, the music contained in these datasets tend to have a much steadier tempo than those contained in ASAP (even SMC; see Section 4.3 for a comparison).

3. PRODUCING MUSIC ANNOTATIONS

Annotating a dataset the size of ASAP with ground truth for AMT and related problems such as beat tracking is a time consuming task that can, in principle, only be performed by expert musicians. This severely limits the ease with which one can produce a reliable and finely crafted dataset at the required scale.

For piano music, MIDI and audio performances can be automatically aligned if they are recorded at the same time using an acoustic piano fitted with the proper sensors such as a Disklavier. The main problem then becomes annotating the performances with metrical markings (such as beats and downbeats) and aligning those with a musical score. Holzapfel et al. [18] describe the process used to annotate the SMC dataset in detail, showing how much manual work was required for its beat annotations. ASAP contains more than 92 hours of performance data, and even a skilled musician would need to listen to each multiple times with the proper annotation software in order to annotate it fully (and this time can increase dramatically for complicated pieces with multiple time and key signature changes). Moreover, as highlighted in [40], the manual annotations would still be affected by human subjectivity, requiring a system with multiple annotators and a reconciliation mechanism between them (e.g., [10, 18]). We believe that without a large budget and/or the ability to involve a community of expert users willing to spend significant time on the task, producing a large ground truth dataset cannot be achieved through a purely manual approach.

⁶<http://piano-e-competition.com/>

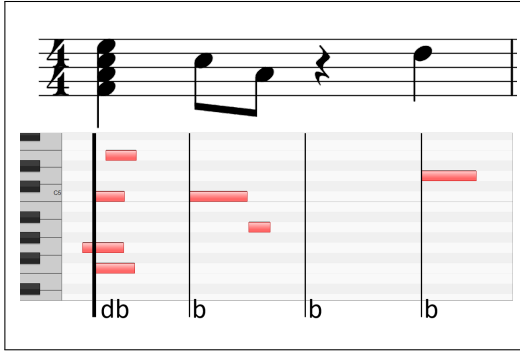


Figure 1. Beat and downbeat annotations produced by our workflow in different cases. The 1st (db) is the median of the onsets of the corresponding notes, the 2nd and the 4th (b) are the onset of the corresponding note, and the 3rd (b) is the mean between the two neighbor annotations.

We therefore propose a workflow (Section 3.1) that allows for the automatic production of annotations from digital musical scores and MIDI performances. The precision of the results is much higher than what would be expected from human-based annotation with a single annotator. Moreover, when the quality of the available scores is high, the workflow does not require any human intervention. Manual intervention is sometimes required to fix problems related to either the digital encoding of the scores or performance errors (Section 3.2).

3.1 Annotation Workflow

The annotation workflow (Figure 3) takes a MIDI performance and a MusicXML score as input, and produces beat, downbeat, key signature change and time signature change annotations for each. Each annotation is aligned to a position (in seconds) in both the MIDI performance and a MIDI score (generated automatically from the MusicXML score), and each downbeat is further aligned with a measure number from the MusicXML score. The workflow is:

1. Expand any repetitions present in the MusicXML score and extract time and key signature changes using music21 [7].
2. Generate the MIDI score using the MuseScore3 MIDI export function.
3. Extract the times of beats, downbeats, and key and time signature changes from the generated MIDI using pretty_midi [33].
4. Align every downbeat from the MIDI score with a measure number in the XML score.
5. Produce a Score2Performance mapping from each note in the MIDI score to each note in the MIDI performance using the algorithm presented in [30].
6. The performance annotations can then be obtained. For each annotation in the MIDI score (from step 3):
 - (a) Take the notes with an onset within 20ms of the annotation (there can be multiple notes, e.g. for the downbeat annotation in Figure 1).
 - (b) Use the Score2Performance mapping, to obtain the onset times of the corresponding notes in the performance file.

- (c) Compute the median of the onset times of those notes, and use it as the time of the annotation in the performance.
- (d) If no notes are within 20ms of the MIDI score annotation (e.g., in case of a rest), the position is interpolated from neighboring annotations (e.g., the 3rd annotation in Figure 1)

As shown in [13], annotations on rests or multiple non-simultaneous notes (grace-notes, arpeggios) are inherently problematic, even for human annotators. An inspection of our annotations reveals that our workflow generally produces good results. In particular, our use of the median increases robustness while handling non-simultaneous notes.

3.2 Practical issues with erroneous input

While the mapping between the scores and performances in ASAP is such that they have a very good correspondence in general, local problems can still occur in specific cases. These can be caused either by encoding issues in the XML score, or by performance errors in the MIDI.

For our automatic workflow to produce good results, the *content* level of the XML score must be correctly encoded, although we can ignore problems at the *graphical* level (we refer to the model of the multiple levels of information in a musical score proposed in [9]). Many of the problems that we encountered during the automatic creation of ASAP’s annotations are tricks used by editors to fix problems at the graphical level at the expense of correctness at the content level: for example, grace notes entered as regular notes with a smaller font, invisible barlines inserted mid-bar, invisible notes or rests, or note heads changed for visual reasons inconsistent with their actual duration.

In some cases, editors are forced to use such tricks because the original score itself does not follow standard notation rules. Figure 2 shows two examples of incorrect measure duration: one from Ravel’s *Ondine* (top) and another from Mozart’s *Fantasia in C minor* (in 4/4; bottom left). There are many ways to handle such cases. Possibilities include having invisible tuplet markings, having a different time signature than the one displayed, and having an overflowing measure. The latter two techniques create problems for automatic beat extraction. Figure 2 (bottom right) is an example of a key change in the middle of a bar in Beethoven’s *Sonata No.29, Op.106*, 2nd movement. One way to encode this situation is to split the bar into 2 separate bars, but this also creates problems for automatic beat extraction in the form of an extra downbeat.

Fortunately, such problems are generally easy to detect since they often result in a measure where the sum of the events does not match the duration defined by the time signature. To be able to produce correct annotations even in the case of these “faulty” measures (around 3% of measures in ASAP’s musical scores), we introduce a manual correction step for the MIDI score annotations (Figure 3). This prevents the propagation of such problems down to the performance annotations.

The alignment that we use to generate the Score2Performance mapping [30] is robust against small errors and

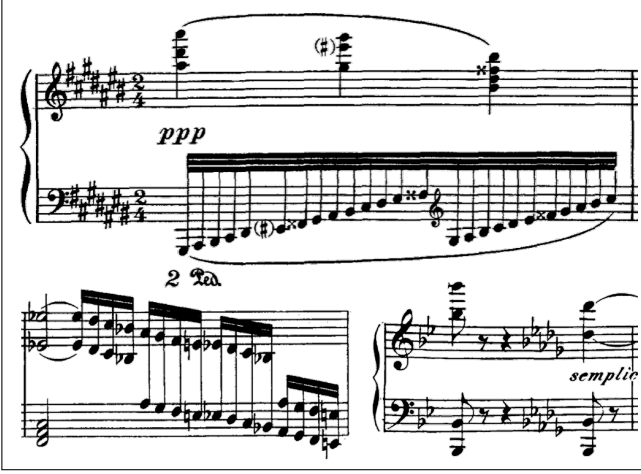


Figure 2. Examples of problems in musical scores, including incorrect bar length (Ravel’s *Ondine* (top) and Mozart’s *Fantasia in C minor* (in 4/4; bottom left)) and a mid-bar key change (Beethoven’s *Sonata No. 29, Op. 106*, 2nd movement (bottom right)).

note inversions. Nonetheless, small errors in its alignment exist. As the difference between a performance and MIDI score increases, the chance of having an incorrect alignment also increases. This can occur in the case of embellishments (e.g. long trills, or mordents that can be played from the printed note or from the note above) or major performance mistakes. We try to detect these problems automatically by measuring the inter-beat-intervals of each performance and marking the outliers as possible problems. On these outliers (which occurred in around 400 of ASAP’s performances), we introduce a final manual correction step. 43 performances contained significant alignment errors that were corrected, and around 2% of annotations had to be moved by less than 1 second.

4. DATASET OVERVIEW

4.1 Dataset content

ASAP contains 222 distinct musical scores and 1068 unique performances of Western classical piano music from 15 different composers (see Table 2 for a breakdown). 548 of the recordings are available as MIDI only, and all the others (520) are provided as MIDI and audio recordings aligned with approximately 3 ms precision. Each score corresponds with at least one performance (and usually more). Every score and performance in ASAP is labeled with metadata including the composer and the title of the piece. We took care to ensure that any two performances of the same piece are labeled with the same title and composer, and no two distinct pieces in ASAP share both.

Each musical score is provided in both MusicXML⁷ and MIDI formats. In the MIDI score, the position of all MIDI events are quantized to a metrical grid according to their position in the MusicXML score. Grace notes are represented in MIDI as notes of very short duration. Repetitions in the score are “unfolded” in the MIDI file such

⁷ <https://www.musicxml.com/>

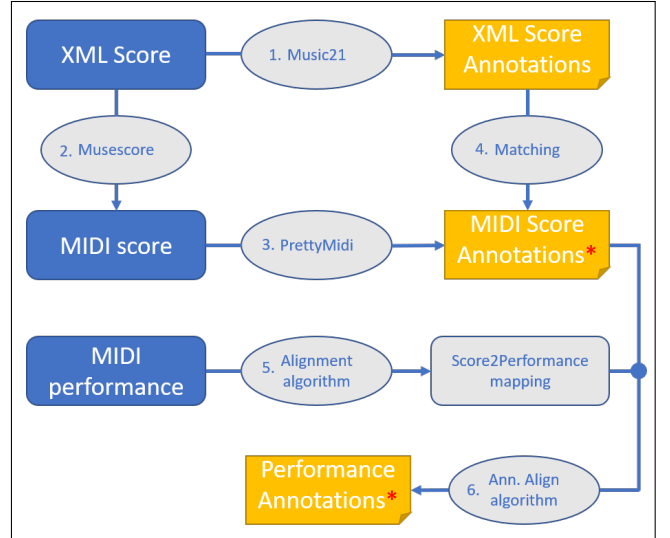


Figure 3. Our annotation workflow; “*” indicates manual correction (see Section 3.2)

that some sections of the MusicXML score may be duplicated in the MIDI score. Except for performance mistakes, there is a one-to-one correspondence between the notes in a MIDI performance and its associated MIDI score.

For each performance and MIDI score, ASAP provides the positions (in seconds) of all beats, downbeats, time signature changes and key signature changes. Time signature changes are annotated only on downbeats. In the case of pickup measures (and pickup measures in the middle of the score) we delay the position of the time signature change annotation to the following downbeat. Similarly, key signature changes are annotated only on beats. Each downbeat is also mapped to a specific measure number in the MusicXML score, which allows for a clear score alignment, even in the case of repetitions.

The dataset and the code used to generate annotations, along with a detailed description of the specific formatting of the dataset and usage examples are available online⁸.

4.2 Origin of the files

The files in ASAP are drawn from multiple sources. The MusicXML scores are from the MuseScore online library⁹, created and uploaded by the users of the MuseScore music notation software. They were first collected and associated to MIDI performances from the Yamaha e-piano competition by the authors of [20]. We manually edited the MusicXML scores using MuseScore3 to correct significant notation errors, and generated quantized MIDI files using MuseScore3’s MIDI export utility. The paired audio and MIDI performances come from the MAESTRO dataset [17]. We automatically matched as many of the MAESTRO performances as we could to ones collected by [20], thus associating them with musical scores. The unmatched performances from MAESTRO are not included in ASAP. Finally, we modified 5 of the MIDI performances

⁸ <https://github.com/fosfrancesco/asap-dataset>

⁹ <https://musescore.com/sheetmusic>

Composer	XML/MIDI Score	MIDI Perf.	Audio Perf.
Bach	59	169	152
Balakirev	1	10	3
Beethoven	57	271	120
Brahms	1	1	0
Chopin	34	290	109
Debussy	2	3	3
Glinka	1	2	2
Haydn	11	44	16
Liszt	16	121	48
Mozart	6	16	5
Prokofiev	1	8	0
Rachmaninoff	4	8	4
Ravel	4	22	0
Schubert	13	62	44
Schumann	10	28	7
Scriabin	2	13	7
Total	222	1068	520

Table 2. The composition of ASAP.

by inserting a missing note at the beginning, and 277 more have been cut to obtain more homogeneous pieces (e.g., the Bach Preludes are separated from the Fugues, even though they sometimes come from the same performance).

4.3 Dataset Statistics

ASAP contains performances of classical piano music, a style that can be challenging for beat tracking systems due to the large tempo variations that are often present. Here, we compare the tempo variation of the pieces in ASAP to that of the pieces of datasets commonly used for beat tracking from audio [12, 14, 18, 38]. To quantify the tempo variation for each piece, we first compute the BPM at each beat based on the amount of time between consecutive beats. Then, we compute Δ_{BPM} at each beat as the difference between consecutive BPMs. Finally, we compute the standard deviation of the set of all Δ_{BPM} values in a particular piece, which we call $\sigma(\Delta_{BPM})$. Figure 4 presents the distribution of these standard deviations for each dataset as a Cumulative Distribution Function, which shows the probability that a randomly chosen piece from each dataset has a $\sigma(\Delta_{BPM})$ less than the given value. From the plot, it can be seen that Ballroom and SMC have generally steadier tempos than the other datasets, and that ASAP’s steadiest 40% and 50% of pieces roughly match those of Hainsworth and SMC respectively. However, a large portion of the pieces in ASAP have significantly larger tempo variation than any of the compared datasets.

In ASAP, differences can be observed between composers. Even though the pieces in the dataset fall under the broad term “classical music”, ASAP is very diverse, containing pieces of various styles written across different periods. This can be observed from differences in average $\sigma(\Delta_{BPM})$ for each composer, as is shown in Figure 5. The values in this plot generally match musicological intuitions about tempo variation for the different composers.

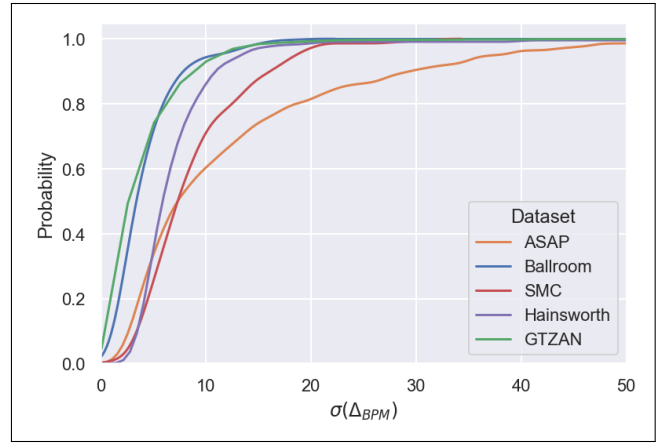


Figure 4. Cumulative Distribution Function of tempo variation $\sigma(\Delta_{BPM})$ of each piece in the compared datasets.

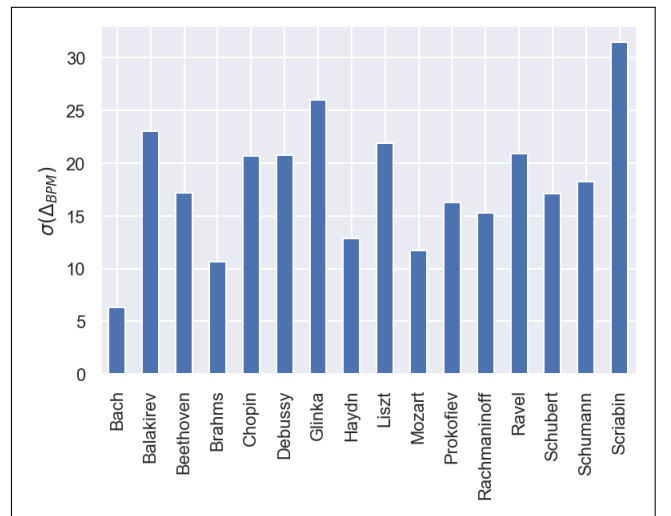


Figure 5. Average tempo variation $\sigma(\Delta_{BPM})$ of the pieces by each composer in ASAP.

5. CONCLUSION

This paper presented ASAP: a new dataset of aligned musical scores and performances of classical piano music. Downbeat, beat, time signature, and key signature annotations are produced using a novel workflow that exploits information present in the musical score to drastically reduce manual annotation time compared to fully manual annotation. ASAP contains over 1000 annotated MIDI performances of classical piano music, over 500 of which are paired with audio from the MAESTRO dataset. To our knowledge, it is the largest dataset of that contains such a fine-grained alignment between scores and performances.

This work has only scratched the surface of what can be done with ASAP. Future work will present further statistical analyses on the data and baseline model performance on tasks for which it can be used: complete AMT and beat tracking as presented, as well as others such as expressive performance analysis and rendering [4]. For complete AMT in particular, the evaluation method is still an open problem, although proposals have been made (e.g., [6, 27]).

6. ACKNOWLEDGMENTS

This work was partially funded by ANR France-Quebec MUNIR Project. The first author conducted his research as International Research Fellow of the Japan Society for the Promotion of Science.

7. REFERENCES

- [1] Emmanouil Benetos, Simon Dixon, Zhiyao Duan, and Sebastian Ewert. Automatic music transcription: An overview. *IEEE Signal Processing Magazine*, 36(1):20–30, jan 2019.
- [2] Sebastian Böck, Matthew E.P. Davies, and Peter Knees. Multi-task learning of tempo and beat: Learning one to improve the other. In *ISMIR*, 2019.
- [3] Jorge Calvo-Zaragoza, Jan Hajic Jr, and Alexander Pacha. Understanding optical music recognition. *Computer Research Repository*, abs/1908.03608, 2019.
- [4] Carlos E. Cancino-Chacón, Maarten Grachten, Werner Goebel, and Gerhard Widmer. Computational models of expressive music performance: A comprehensive and critical review. *Frontiers in Digital Humanities*, 5:25, 2018.
- [5] Ralf Gunter Correa Carvalho and Paris Smaragdis. Towards end-to-end polyphonic music transcription: Transforming music audio directly to a score. In *WASPAA*, pages 151–155, 2017.
- [6] Andrea Cogliati and Zhiyao Duan. A metric for music notation transcription accuracy. In *ISMIR*, pages 407–413, 2017.
- [7] Michael Scott Cuthbert, Christopher Ariza, and Lisa Friedland. Feature extraction and machine learning on symbolic music using the music21 toolkit. In *ISMIR*, pages 387–392, 2011.
- [8] Valentin Emiya, Roland Badeau, and Bertrand David. Multipitch estimation of piano sounds using a new probabilistic spectral smoothness principle. *IEEE Transactions on Audio, Speech, and Language Processing*, 18(6):1643–1654, aug 2010.
- [9] Francesco Foscarin, David Fiala, Florent Jacquemard, Philippe Rigaux, and Virginie Thion. Gioqoso, an online Quality Assessment Tool for Music Notation. In *4th International Conference on Technologies for Music Notation and Representation (TENOR'18)*, Montreal, Canada, May 2018.
- [10] Thassilo Gadermaier and Gerhard Widmer. A study of annotation and alignment accuracy for performance comparison in complex orchestral music. *arXiv preprint arXiv:1910.07394*, 2019.
- [11] Werner Goebel. Numerisch-klassifikatorische interpretationsanalyse mit dem "Bösendorfer Computerflügel". Master's thesis, Universität Wien, 1999.
- [12] Fabien Gouyon, Anssi Klapuri, Simon Dixon, Miguel Alonso, George Tzanetakis, Christian Uhle, and Pedro Cano. An experimental comparison of audio tempo induction algorithms. *IEEE Transactions on Audio, Speech, and Language Processing*, 14(5):1832–1844, 2006.
- [13] Peter Grosche, Meinard Müller, and Craig Stuart Sapp. What makes beat tracking difficult? a case study on chopin mazurkas. In *ISMIR*, pages 649–654, 2010.
- [14] Stephen W Hainsworth and Malcolm D Macleod. Particle filtering applied to musical tempo tracking. *EURASIP Journal on Advances in Signal Processing*, 2004(15):927847, 2004.
- [15] Mitsuyo Hashida, Toshie Matsui, and Haruhiro Katayose. A new music database describing deviation information of performance expressions. In *ISMIR*, pages 489–494, 2008.
- [16] Curtis Hawthorne, Erich Elsen, Jialin Song, Adam Roberts, Ian Simon, Colin Raffel, Jesse Engel, Sageev Oore, and Douglas Eck. Onsets and frames: Dual-objective piano transcription. In *ISMIR*, 2018.
- [17] Curtis Hawthorne, Andriy Stasyuk, Adam Roberts, Ian Simon, Cheng-Zhi Anna Huang, Sander Dieleman, Erich Elsen, Jesse Engel, and Douglas Eck. Enabling factorized piano music modeling and generation with the MAESTRO dataset. In *International Conference on Learning Representations*, 2019.
- [18] Andre Holzapfel, Matthew EP Davies, José R Zapata, João Lobato Oliveira, and Fabien Gouyon. Selective sampling for beat tracking evaluation. *IEEE Transactions on Audio, Speech, and Language Processing*, 20(9):2539–2548, 2012.
- [19] Cheng-Zhi Anna Huang, Ashish Vaswani, Jakob Uszkoreit, Ian Simon, Curtis Hawthorne, Noam Shazeer, Andrew M Dai, Matthew D Hoffman, Monica Dinulescu, and Douglas Eck. Music transformer: Generating music with long-term structure. In *International Conference on Learning Representations*, 2018.
- [20] Dasaem Jeong, Taegyun Kwon, Yoojin Kim, Kyogu Lee, and Juhan Nam. VirtuosoNet: A hierarchical RNN-based system for modeling expressive piano performance. In *ISMIR*, pages 908–915, 2019.
- [21] Rainer Kelz, Matthias Dorfer, Filip Korzeniowski, Sebastian Böck, Andreas Arzt, and Gerhard Widmer. On the potential of simple framewise approaches to piano transcription. In *ISMIR*, pages 475–481, 2016.
- [22] Jong Wook Kim and Juan Pablo Bello. Adversarial learning for improved onsets and frames music transcription. In *ISMIR*, pages 670–677, 2019.
- [23] Florian Krebs, Sebastian Böck, and Gerhard Widmer. Rhythmic pattern modeling for beat and downbeat tracking in musical audio. In *ISMIR*, pages 227–232, 2013.

- [24] Ugo Marchand and Geoffroy Peeters. Swing ratio estimation. In *Digital Audio Effects (DAFx)*, pages 423–428, 2015.
- [25] Andrew McLeod, Eita Nakamura, and Kazuyoshi Yoshii. Improved metrical alignment of MIDI performance based on a repetition-aware online-adapted grammar. In *IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, pages 186–190, 2019.
- [26] Andrew McLeod and Mark Steedman. HMM-based voice separation of MIDI performance. *Journal of New Music Research*, 45(1):17–26, 2016.
- [27] Andrew McLeod and Mark Steedman. Evaluating automatic polyphonic music transcription. In *ISMIR*, pages 42–49, 2018.
- [28] Meinard Müller, Verena Konz, Wolfgang Bogler, and Vlora Arifi-Müller. Saarland music data (SMD). In *Late-Breaking and Demo Session of ISMIR*, 2011.
- [29] Eita Nakamura, Emmanouil Benetos, Kazuyoshi Yoshii, and Simon Dixon. Towards complete polyphonic music transcription: Integrating multi-pitch detection and rhythm quantization. In *IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, pages 101–105. IEEE, 2018.
- [30] Eita Nakamura, Kazuyoshi Yoshii, and Haruhiro Katayose. Performance error detection and post-processing for fast and accurate symbolic music alignment. In *ISMIR*, pages 347–353, 2017.
- [31] Ryo Nishikimi, Eita Nakamura, Satoru Fukayama, Masataka Goto, and Kazuyoshi Yoshii. Automatic singing transcription based on encoder-decoder recurrent neural networks with a weakly-supervised attention mechanism. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 161–165. IEEE, 2019.
- [32] Adriana Olmos, Nicolas Bouillot, Trevor Knight, Nordhal Mabire, Josh Redel, and Jeremy R Cooperstock. A high-fidelity orchestra simulator for individual musicians’ practice. *Computer Music Journal*, 36(2):55–73, 2012.
- [33] Colin Raffel and Daniel P. W. Ellis. Intuitive analysis, creation and manipulation of midi data with pretty_midi. In *ISMIR Late Breaking and Demo Papers*, 2014.
- [34] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. Squad: 100,000+ questions for machine comprehension of text. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392, 2016.
- [35] Miguel A. Román, Antonio Pertusa, and Jorge Calvo-Zaragoza. A holistic approach to polyphonic music transcription with neural networks. In *ISMIR*, pages 731–737, 2019.
- [36] Zhengshan Shi, Craig Stuart Sapp, Kumaran Arul, Jerry McBride, and Julius O. Smith. SUPRA: Digitizing the stanford university piano roll archive. In *ISMIR*, pages 517–523, 2019.
- [37] Daniel Stoller, Sebastian Ewert, and Simon Dixon. Wave-U-Net: A multi-scale neural network for end-to-end audio source separation. In *ISMIR*, pages 334–340, 2018.
- [38] George Tzanetakis and Perry Cook. Musical genre classification of audio signals. *IEEE Transactions on speech and audio processing*, 10(5):293–302, 2002.
- [39] Athanasios Voulodimos, Nikolaos Doulamis, Anastasios Doulamis, and Eftychios Protopapadakis. Deep learning for computer vision: A brief review. *Computational intelligence and neuroscience*, 2018, 2018.
- [40] Christof Weiß, Vlora Arifi-Müller, Thomas Prätzlich, Rainer Kleinertz, and Meinard Müller. Analyzing measure annotations for western classical music recordings. In *ISMIR*, pages 517–523, 2016.
- [41] Adrien Ycart and Emmanouil Benetos. A-MAPS: Augmented MAPS dataset with rhythm and key annotations. In *ISMIR Late Breaking and Demo Papers*, 2018.
- [42] Tom Young, Devamanyu Hazarika, Soujanya Poria, and Erik Cambria. Recent trends in deep learning based natural language processing. *IEEE Computational Intelligence Magazine*, 13(3):55–75, 2018.

HIGH RESOLUTION AUDIO SYNCHRONIZATION USING CHROMA ONSET FEATURES

Sebastian Ewert*

Universität Bonn, Informatik III
Römerstr. 164, D-53117 Bonn

Meinard Müller, Peter Grosche†

Saarland University and MPI Informatik
Campus E1 4, D-66123 Saarbrücken

ABSTRACT

The general goal of music synchronization is to automatically align the multiple information sources such as audio recordings, MIDI files, or digitized sheet music related to a given musical work. In computing such alignments, one typically has to face a delicate tradeoff between robustness and accuracy. In this paper, we introduce novel audio features that combine the high temporal accuracy of onset features with the robustness of chroma features. We show how previous synchronization methods can be extended to make use of these new features. We report on experiments based on polyphonic Western music demonstrating the improvements of our proposed synchronization framework.

Index Terms— Music synchronization, onset features, chroma features, audio alignment

1. INTRODUCTION

In digital music libraries and private music collections, there is an increasing number of documents available for a given musical work. These documents may comprise various audio recordings, MIDI files or score representations. Music information retrieval (MIR) aims at developing techniques and tools for organizing, understanding and searching this multimodal information in a robust, efficient and intelligent manner. In this context, various alignment and synchronization procedures have been proposed with the common goal to automatically link several types of music representations, thus coordinating the multiple information sources related to a given musical work [4, 5, 6].

In general terms, *music synchronization* denotes a procedure which, for a given position in one representation of a piece of music, determines the corresponding position within another representation. Depending upon the respective data formats, one distinguishes between various synchronization tasks [5]. For example, *audio-audio* synchronization refers to the task of time aligning two different audio recordings of a piece of music. These alignments can be used to jump freely

between different interpretations, thus affording efficient and convenient audio browsing. The goal of *MIDI-audio* synchronization is to coordinate MIDI note events with audio data. The result can be regarded as an automated annotation of the audio recording with available MIDI data.

In the design of synchronization algorithms, one has to deal with a delicate tradeoff between robustness and temporal accuracy. As first contribution, we introduce a novel class of 12-dimensional onset features, which combine the robustness of conventional chroma features [1] with the accuracy of conventional one-dimensional onset features [2]. These features are obtained by identifying pitch-based onset information on the chroma level (Sect. 2). As second contribution, we introduce a synchronization framework that allows for improving the overall synchronization accuracy without losing robustness (Sect. 3). Here, the idea is to making the best of each feature type when combining the various information. Our experiments show that our synchronization procedure, which integrates conventional chroma features as well as our novel onset features, significantly improves the accuracy in particular for piano music while not collapsing for music that does not contain clear note attacks (Sect. 4). Further related work will be discussed in the respective sections.

2. AUDIO FEATURES

In order to synchronize different music representations, one needs to find suitable feature representations being robust towards those variations that are to be left unconsidered in the comparison. In this context, chroma-based features have turned out to be a powerful tool for synchronizing harmony-based music, see [4, 5]. In summary, chroma features encode the short-time energy distribution over the 12 traditional pitch classes of the equal-tempered scale encoded by the attributes C, C[♯], D, . . . , B. Furthermore, chroma features can be made invariant to dynamic variations by normalization. For details we refer to the literature [5]. In the following, the first six measures of the Etude No. 2, Op. 100, by Friedrich Burgmüller will serve us as our running example, see Fig. 1a, denoted by the identifier **Burg2**. Fig. 1b shows a normalized chroma representation of an audio recording of **Burg2**.

In the following, we describe another class of highly expressive audio features that indicate note onsets along with

*The author is funded by the German Research Foundation (DFG CL 64/6-1)

†The authors are funded by the Cluster of Excellence on Multimodal Computing and Interaction (MMCI)

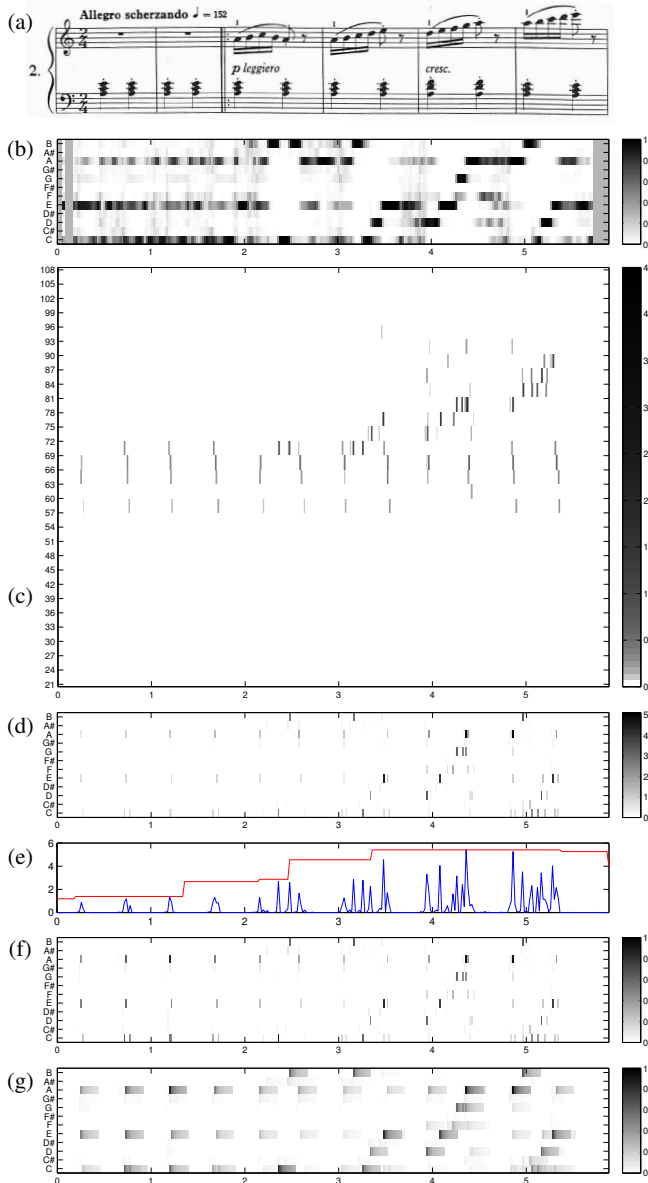


Fig. 1. (a) First six measures of Burgmüller, Op. 100, Etude No. 2. (b) - (g) feature representations of a corresponding audio recording (see Sect. 2 for a description).

their respective pitch affiliation [5]. The feature extraction procedure is motivated by the observation that for many instruments such as the piano or the guitar, playing a note results in a sudden energy increase (attack phase).

First, the audio signal is decomposed into 88 subbands corresponding to the musical notes A0 to C8 (MIDI pitches $p = 21$ to $p = 108$) of the equal-tempered scale, as in the chroma feature calculation. Then, 88 local energy curves are computed by convolving each of the squared subbands with a suitable window function. Finally, for each energy curve the discrete temporal derivative is calculated and half-wave rectified (positive part of the function remains). The signif-

icant peaks of the resulting curves indicate positions of significant energy increase in the respective pitch subband. An onset feature is specified by the pitch of its subband and by the time position and height of the corresponding peak. Fig. 1c shows the resulting onset representation obtained for our running example **Burg2**. Note that the set of onset features is sparse while providing information of very high temporal accuracy. On the downside, the extraction of onset features is a delicate problem involving fragile peak picking operations.

To enhance the robustness of the pitch-based onset features, we add up the features belonging to pitches of the same pitch class, as motivated by the chroma features. We first evenly split up the time axis into segments of a fixed length and take a suitable logarithm of the onset values, which accounts for the logarithmic sensation of sound intensity. For each segment, we add up the logarithmic values over all pitches that correspond to the same chroma. The resulting 12-dimensional features will be referred to as *CO (chroma onset) features*, see Fig. 1d. To make the CO feature invariant to dynamic variations while keeping low level onsets we employ a locally adaptive normalization strategy. First, we compute the norm of each 12-dimensional CO feature vector, see Fig. 1e (blue curve). Then, for each time frame, we assign the local maxima of the sequence of norms over a time window that ranges one second to the left and one second to the right, see Fig. 1e (red curve). Finally, we divide the sequence of CO features by the sequence of local maxima in a pointwise fashion, see Fig. 1f. The resulting features are referred to as *LNCO (locally adaptive normalized CO) features*. Intuitively, LNCO features account for the fact that onsets of low energy are less relevant in musical passages of high energy than in passages of low energy. In summary, the octave identification makes LNCO features robust to timbre and extraction errors while still encoding 12-dimensional highly accurate onset information. At this point, we emphasize that the opposite variant of first computing chroma features and then computing onsets from the resulting chromagrams is not as successful as our strategy. The major reason for this is that by first changing to the coarser chroma representation one may already loose valuable onset information. For example, suppose a clear onset in the C3 pitch band and some smearing in the C4 band. Then, the smearing may overlay the onset on the chroma level, which may result in missing the onset information. However, by first computing onsets for all pitches separately and then merging this information, the onset of the C3 pitch band will be clearly visible on the chroma level.

In view of synchronization applications, we further process the LNCO feature representation by introducing an additional temporal decay. To this end, each LNCO feature vector is copied n times and the copies are multiplied by decreasing positive weights (in our experiments we chose $n = 10$ with weights $(1, \sqrt{0.9}, \sqrt{0.8}, \dots, \sqrt{0.1})$). Then, the n copies are arranged to form short sequences of n consecutive feature vectors of decreasing norm starting at the time position of the

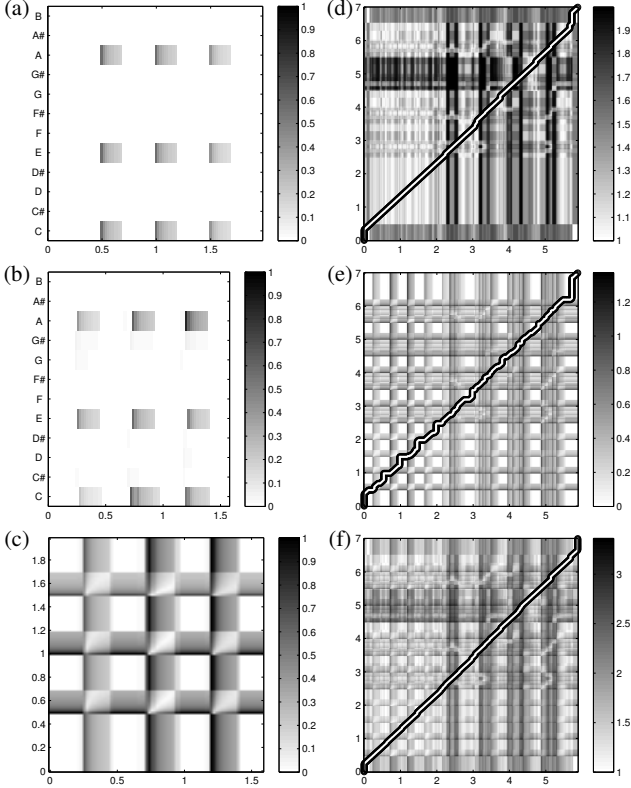


Fig. 2. (a)-(c) Illustration of the effect of the decay operation on the cost matrix level. (d) C_{chroma} , (e), C_{DLNCO} (f) $C_{\text{chroma}} + C_{\text{DLNCO}}$ for **Burg2**.

original vector. The overlay of all these decaying sequences results in a feature representation, which we refer to as *DLNCO (decaying LNCO) feature* representation, see Fig. 1g. The benefit of these additional temporal decays will become clear in the synchronization context described in Sect. 3. Note that in the DLNCO feature representation, one does not lose the temporal accuracy of the LNCO features—the onset positions still appear as sharp left edges in the decays.

3. SYNCHRONIZATION ALGORITHM

In this section, we show how our novel DLNCO features can be used to significantly improve the temporal accuracy of previous chroma-based synchronization strategies without sacrificing their robustness. While we consider the case of MIDI-audio synchronization in the following, other cases such as audio-audio synchronization may be handled in the same fashion. Most synchronization algorithms [4, 6] rely on some variant of dynamic time warping (DTW) and can be summarized as follows. First, the two music data streams to be aligned are converted into feature sequences, say $V := (v_1, v_2, \dots, v_N)$ and $W := (w_1, w_2, \dots, w_M)$, respectively. Then, an $N \times M$ cost matrix C is built up by evaluating a local cost measure c for each pair of features,

i.e., $C(n, m) = c(v_n, w_m)$ for $1 \leq n \leq N, 1 \leq m \leq M$. Finally, an optimum-cost alignment path is determined from this matrix via dynamic programming, which encodes the synchronization result. See [5] for a detailed account on DTW in the music context. For an illustration, we refer to Figs. 2d-2f, which show various cost matrices along with optimal alignment paths for our **Burg2** example.

We now introduce three different cost matrices, where the third one is a simple combination of the first and second one. The first matrix C_{chroma} is a conventional cost matrix based on normalized chroma features and the cosine distance [4, 5], see Fig. 2d. The second cost matrix C_{DLNCO} is based on DLNCO features as introduced in Sect. 2. To compare two DLNCO feature vectors, v and w , we now use the Euclidean distance $c_{\text{DLNCO}}(v, w) := \|v - w\|$, see Fig. 2e. At this point, we need to make some explanations. First, recall that each onset has been transformed into a short vector sequence of decaying norm. As an example, Figs. 2a and 2b show DLNCO features for the very beginning of **Burg2** for an audio and a MIDI version, respectively. Using the Euclidean distance to compare two such decaying sequences leads to a diagonal corridor of low cost in C_{DLNCO} in the case that the directions (i.e., the relative chroma distributions) of the onset vectors are similar, see Fig. 2c. This corridor is tapered to the lower left and starts at the precise time positions of the two onsets to be compared. Second, note that C_{DLNCO} reveals a grid like structure of an overall high cost, where each beginning of a corridor forms a small needle’s eye of low cost. Third, sections in the feature sequences with no onsets lead to regions in C_{DLNCO} having zero cost. In other words, only significant events in the DLNCO feature sequences take effect on the cost matrix level and the structure of C_{DLNCO} regulates the course of a cost-minimizing alignment path in event-based regions to run through the needle’s eyes of low cost.

The cost matrix C_{chroma} accounts for the rough harmonic flow of the two representations, whereas C_{DLNCO} exhibits onsets of the same chroma class. The sum $C = C_{\text{chroma}} + C_{\text{DLNCO}}$ yields a cost matrix that accounts for both types of information. Note that in regions with no onsets, C_{DLNCO} is zero and the combined matrix C is dominated by C_{chroma} . Contrary, in regions with significant onsets, C is dominated by C_{DLNCO} . Therefore, the component C_{chroma} regulates the overall course of the cost-minimizing alignment path and accounts for a robust synchronization, whereas the component C_{DLNCO} locally adjusts the alignment path and accounts for high temporal accuracy.

4. EXPERIMENTS

In this section, we report on some of our synchronization experiments, which have been conducted on a corpus of harmony-based Western music. To allow for a reproduction of our experiments, we used pieces from the RWC music database [3]. In the following, we consider 16 representa-

RWC ID (Comp./Interp., Instr.)	Chroma		DLNCO		Chroma+DLNCO	
	mean	std	mean	std	mean	std
Burg2 (Burgmüller, piano)	50	48	21	17	18	14
C025 (Bach, piano)	27	33	18	27	14	12
C028 (Beethoven, piano)	54	58	131	318	29	40
C031 (Chopin, piano)	57	64	22	68	22	33
C032 (Chopin, piano)	30	47	12	9	13	21
C029 (Schumann, piano)	46	72	94	264	15	36
Average over piano	44	54	50	117	19	26
C003 (Beethoven, orchestra)	116	96	241	338	116	98
C015 (Borodin, strings)	79	68	268	356	82	56
C022 (Brahms, orchestra)	50	54	26	52	17	20
C044 (Rimski-Korsakov, flute/piano)	41	17	22	19	27	15
C048 (Schubert, voice/piano)	55	50	70	173	31	34
Average over non-piano	68	57	125	188	55	45
J001 (Nakamura, piano)	34	59	17	37	14	15
J038 (HH Band, big band)	45	46	85	204	31	64
J041 (Umitsuki Quart., sax/bass/perc.)	39	67	37	117	23	55
P031 (Nagayama, electronic)	68	50	124	217	46	43
P093 (Burke, voice/guitar)	91	95	71	103	40	58
Average over jazz/pop	55	63	67	136	31	47
Average over all	55	58	79	145	34	38

Table 1. Alignment accuracy for the three different synchronization procedures (Chroma, DLNCO, Chroma+DLNCO) on the test database obtained from the RWC database [3]. All values are given in milliseconds.

tive pieces, which are listed in Table 1. These pieces are divided into three groups: six classical piano pieces, five classical pieces of various instrumentations, and five jazz pieces and pop songs. Note that while pure piano music typically comprises the concise note attacks the DLNCO features are designed for, such information is often missing especially in string and general orchestral music. We now show that our extended synchronization framework leads to significant improvements for piano music, while not losing on accuracy for music lacking in clear note attacks.

In the following, we use three different synchronization procedures based on chroma features only, on DLNCO features only, and a combination of these features (Chroma+DLNCO), see Sect. 3. In each experiment we use 50 features per second, i.e., the features have a temporal resolution of 20ms. To automatically determine the accuracy of our synchronization procedures, we used pairs of MIDI and audio versions of each of the 16 pieces listed in Table 1. Here, the audio versions were generated from the MIDI files using a high-quality synthesizer. Thus, for each synchronization pair, the note onset times in the MIDI file are perfectly aligned with the onset times in the respective audio recording. We randomly distorted the MIDI files by splitting up the MIDI files into N segments of equal length (in our experiment we used $N = 20$) and stretching or compressing each segment by a random factor within an allowed distortion range (in our experiments we used a range of $\pm 30\%$). We refer to the resulting MIDI file as the *distorted MIDI file* in contrast to the original *annotation MIDI file*. We synchronized the distorted MIDI file and the associated audio recording and used the resulting alignment path to adjust the note onset times in the distorted MIDI file and to obtain a third MIDI file referred to as *realigned MIDI file*. The accuracy of the synchronization result is then determined by comparing on-

set times of corresponding notes in the realigned MIDI file and the annotation MIDI file. For each of the 16 pieces and for each synchronization procedure Table 1 shows the mean value and the standard deviation over all absolute onset differences. Note that using a combination of chroma and DLNCO features significantly improves the synchronization accuracy: the average onset error for piano music drops from 44ms (Chroma) to 19ms (Chroma+DLNCO). For orchestral or pure string music without clear note attacks, the DLNCO features do not yield any valuable information. For example, in the case of Borodin’s String Quartet (C015), the onset error increases from 79ms (Chroma) to 269ms (DLNCO) when using only the onset features. However, in the combined case, the chroma features overrule the corrupt DLNCO features leading to an onset error of 82ms (Chroma+DLNCO) that is comparable to the chroma only case.

In conclusion, our experiments show that the combination of using chroma and DLNCO onset features significantly improve the synchronization accuracy for music with clear note attacks and does not degrade for music which lacks this information. At this point, one may object that one typically obtains better absolute synchronization results for synthetic audio material (which was used to completely automate our evaluation) than for real audio recordings. We also tested our synchronization on real audio recordings of all 16 pieces, which actually led to similar results as the synthesized examples. Sonifications of the MIDI-audio synchronization results for the real audio files of the 16 pieces have been made available on the website <http://www-mmdb.iain.uni-bonn.de/projects/syncDLNCO/>.

For the future, we will incorporate other types of features that capture local rhythmic information and smooth note transitions for orchestral, string, or brass music [7]. Here, our synchronization framework allows for making the best of each feature type when combining the various information.

5. REFERENCES

- [1] M. Bartsch and G. Wakefield, “Audio thumbnailing of popular music using chroma-based representations,” *IEEE Trans. on Multimedia*, vol. 7, no. 1, pp. 96–104, Feb. 2005.
- [2] J. P. Bello, L. Daudet, S. Abdallah, C. Duxbury, M. Davies, and M. B. Sandler, “A tutorial on onset detection in music signals,” in *IEEE Trans. on Speech and Audio Processing*, vol. 13, no. 5, Sept. 2005.
- [3] M. Goto, H. Hashiguchi, T. Nishimura, and R. Oka, “RWC music database: Popular, classical and jazz music databases,” in *Proc. ISMIR, Paris, France*, 2002.
- [4] N. Hu, R. Dannenberg, and G. Tzanetakis, “Polyphonic audio matching and alignment for music retrieval,” in *Proc. IEEE WASPAA, New Paltz, NY*, October 2003.
- [5] M. Müller, *Information Retrieval for Music and Motion*, Springer, 2007.
- [6] R. J. Turetsky and D. P. W. Ellis, “Force-aligning MIDI syntheses for polyphonic music transcription generation,” in *Proc. ISMIR, Baltimore, USA*, 2003.
- [7] W. You and R. Dannenberg, “Polyphonic music note onset detection using semi-supervised learning,” in *Proc. ISMIR, Vienna, Austria*, 2007.

HIGH RESOLUTION AUDIO SYNCHRONIZATION USING CHROMA ONSET FEATURES

Sebastian Ewert*

Universität Bonn, Informatik III
Römerstr. 164, D-53117 Bonn

Meinard Müller, Peter Grosche†

Saarland University and MPI Informatik
Campus E1 4, D-66123 Saarbrücken

ABSTRACT

The general goal of music synchronization is to automatically align the multiple information sources such as audio recordings, MIDI files, or digitized sheet music related to a given musical work. In computing such alignments, one typically has to face a delicate tradeoff between robustness and accuracy. In this paper, we introduce novel audio features that combine the high temporal accuracy of onset features with the robustness of chroma features. We show how previous synchronization methods can be extended to make use of these new features. We report on experiments based on polyphonic Western music demonstrating the improvements of our proposed synchronization framework.

Index Terms— Music synchronization, onset features, chroma features, audio alignment

1. INTRODUCTION

In digital music libraries and private music collections, there is an increasing number of documents available for a given musical work. These documents may comprise various audio recordings, MIDI files or score representations. Music information retrieval (MIR) aims at developing techniques and tools for organizing, understanding and searching this multimodal information in a robust, efficient and intelligent manner. In this context, various alignment and synchronization procedures have been proposed with the common goal to automatically link several types of music representations, thus coordinating the multiple information sources related to a given musical work [4, 5, 6].

In general terms, *music synchronization* denotes a procedure which, for a given position in one representation of a piece of music, determines the corresponding position within another representation. Depending upon the respective data formats, one distinguishes between various synchronization tasks [5]. For example, *audio-audio* synchronization refers to the task of time aligning two different audio recordings of a piece of music. These alignments can be used to jump freely

between different interpretations, thus affording efficient and convenient audio browsing. The goal of *MIDI-audio* synchronization is to coordinate MIDI note events with audio data. The result can be regarded as an automated annotation of the audio recording with available MIDI data.

In the design of synchronization algorithms, one has to deal with a delicate tradeoff between robustness and temporal accuracy. As first contribution, we introduce a novel class of 12-dimensional onset features, which combine the robustness of conventional chroma features [1] with the accuracy of conventional one-dimensional onset features [2]. These features are obtained by identifying pitch-based onset information on the chroma level (Sect. 2). As second contribution, we introduce a synchronization framework that allows for improving the overall synchronization accuracy without losing robustness (Sect. 3). Here, the idea is to making the best of each feature type when combining the various information. Our experiments show that our synchronization procedure, which integrates conventional chroma features as well as our novel onset features, significantly improves the accuracy in particular for piano music while not collapsing for music that does not contain clear note attacks (Sect. 4). Further related work will be discussed in the respective sections.

2. AUDIO FEATURES

In order to synchronize different music representations, one needs to find suitable feature representations being robust towards those variations that are to be left unconsidered in the comparison. In this context, chroma-based features have turned out to be a powerful tool for synchronizing harmony-based music, see [4, 5]. In summary, chroma features encode the short-time energy distribution over the 12 traditional pitch classes of the equal-tempered scale encoded by the attributes C, C[♯], D, . . . , B. Furthermore, chroma features can be made invariant to dynamic variations by normalization. For details we refer to the literature [5]. In the following, the first six measures of the Etude No. 2, Op. 100, by Friedrich Burgmüller will serve us as our running example, see Fig. 1a, denoted by the identifier **Burg2**. Fig. 1b shows a normalized chroma representation of an audio recording of **Burg2**.

In the following, we describe another class of highly expressive audio features that indicate note onsets along with

*The author is funded by the German Research Foundation (DFG CL 64/6-1)

†The authors are funded by the Cluster of Excellence on Multimodal Computing and Interaction (MMCI)

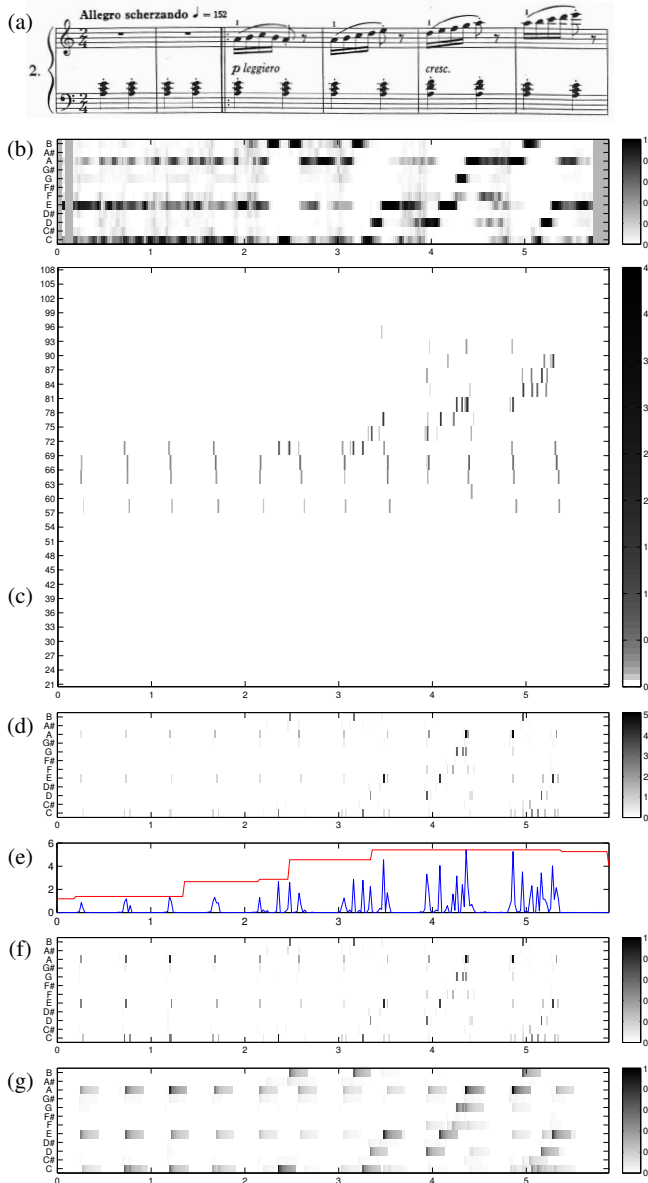


Fig. 1. (a) First six measures of Burgmüller, Op. 100, Etude No. 2. (b) - (g) feature representations of a corresponding audio recording (see Sect. 2 for a description).

their respective pitch affiliation [5]. The feature extraction procedure is motivated by the observation that for many instruments such as the piano or the guitar, playing a note results in a sudden energy increase (attack phase).

First, the audio signal is decomposed into 88 subbands corresponding to the musical notes A0 to C8 (MIDI pitches $p = 21$ to $p = 108$) of the equal-tempered scale, as in the chroma feature calculation. Then, 88 local energy curves are computed by convolving each of the squared subbands with a suitable window function. Finally, for each energy curve the discrete temporal derivative is calculated and half-wave rectified (positive part of the function remains). The signif-

icant peaks of the resulting curves indicate positions of significant energy increase in the respective pitch subband. An onset feature is specified by the pitch of its subband and by the time position and height of the corresponding peak. Fig. 1c shows the resulting onset representation obtained for our running example **Burg2**. Note that the set of onset features is sparse while providing information of very high temporal accuracy. On the downside, the extraction of onset features is a delicate problem involving fragile peak picking operations.

To enhance the robustness of the pitch-based onset features, we add up the features belonging to pitches of the same pitch class, as motivated by the chroma features. We first evenly split up the time axis into segments of a fixed length and take a suitable logarithm of the onset values, which accounts for the logarithmic sensation of sound intensity. For each segment, we add up the logarithmic values over all pitches that correspond to the same chroma. The resulting 12-dimensional features will be referred to as *CO (chroma onset) features*, see Fig. 1d. To make the CO feature invariant to dynamic variations while keeping low level onsets we employ a locally adaptive normalization strategy. First, we compute the norm of each 12-dimensional CO feature vector, see Fig. 1e (blue curve). Then, for each time frame, we assign the local maxima of the sequence of norms over a time window that ranges one second to the left and one second to the right, see Fig. 1e (red curve). Finally, we divide the sequence of CO features by the sequence of local maxima in a pointwise fashion, see Fig. 1f. The resulting features are referred to as *LNCO (locally adaptive normalized CO) features*. Intuitively, LNCO features account for the fact that onsets of low energy are less relevant in musical passages of high energy than in passages of low energy. In summary, the octave identification makes LNCO features robust to timbre and extraction errors while still encoding 12-dimensional highly accurate onset information. At this point, we emphasize that the opposite variant of first computing chroma features and then computing onsets from the resulting chromagrams is not as successful as our strategy. The major reason for this is that by first changing to the coarser chroma representation one may already loose valuable onset information. For example, suppose a clear onset in the C3 pitch band and some smearing in the C4 band. Then, the smearing may overlay the onset on the chroma level, which may result in missing the onset information. However, by first computing onsets for all pitches separately and then merging this information, the onset of the C3 pitch band will be clearly visible on the chroma level.

In view of synchronization applications, we further process the LNCO feature representation by introducing an additional temporal decay. To this end, each LNCO feature vector is copied n times and the copies are multiplied by decreasing positive weights (in our experiments we chose $n = 10$ with weights $(1, \sqrt{0.9}, \sqrt{0.8}, \dots, \sqrt{0.1})$). Then, the n copies are arranged to form short sequences of n consecutive feature vectors of decreasing norm starting at the time position of the

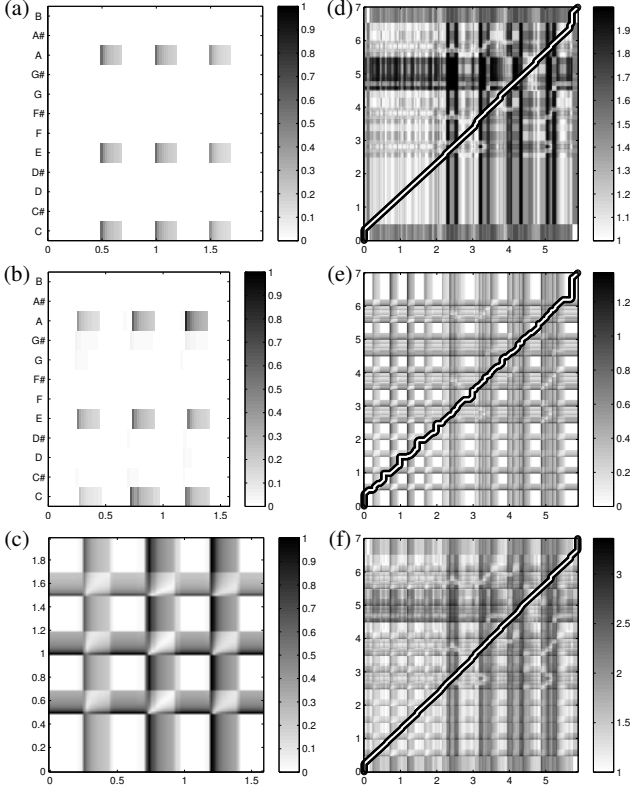


Fig. 2. (a)-(c) Illustration of the effect of the decay operation on the cost matrix level. (d) C_{chroma} , (e), C_{DLNCO} (f) $C_{\text{chroma}} + C_{\text{DLNCO}}$ for **Burg2**.

original vector. The overlay of all these decaying sequences results in a feature representation, which we refer to as *DLNCO (decaying LNCO) feature* representation, see Fig. 1g. The benefit of these additional temporal decays will become clear in the synchronization context described in Sect. 3. Note that in the DLNCO feature representation, one does not lose the temporal accuracy of the LNCO features—the onset positions still appear as sharp left edges in the decays.

3. SYNCHRONIZATION ALGORITHM

In this section, we show how our novel DLNCO features can be used to significantly improve the temporal accuracy of previous chroma-based synchronization strategies without sacrificing their robustness. While we consider the case of MIDI-audio synchronization in the following, other cases such as audio-audio synchronization may be handled in the same fashion. Most synchronization algorithms [4, 6] rely on some variant of dynamic time warping (DTW) and can be summarized as follows. First, the two music data streams to be aligned are converted into feature sequences, say $V := (v_1, v_2, \dots, v_N)$ and $W := (w_1, w_2, \dots, w_M)$, respectively. Then, an $N \times M$ cost matrix C is built up by evaluating a local cost measure c for each pair of features,

i.e., $C(n, m) = c(v_n, w_m)$ for $1 \leq n \leq N, 1 \leq m \leq M$. Finally, an optimum-cost alignment path is determined from this matrix via dynamic programming, which encodes the synchronization result. See [5] for a detailed account on DTW in the music context. For an illustration, we refer to Figs. 2d-2f, which show various cost matrices along with optimal alignment paths for our **Burg2** example.

We now introduce three different cost matrices, where the third one is a simple combination of the first and second one. The first matrix C_{chroma} is a conventional cost matrix based on normalized chroma features and the cosine distance [4, 5], see Fig. 2d. The second cost matrix C_{DLNCO} is based on DLNCO features as introduced in Sect. 2. To compare two DLNCO feature vectors, v and w , we now use the Euclidean distance $c_{\text{DLNCO}}(v, w) := \|v - w\|$, see Fig. 2e. At this point, we need to make some explanations. First, recall that each onset has been transformed into a short vector sequence of decaying norm. As an example, Figs. 2a and 2b show DLNCO features for the very beginning of **Burg2** for an audio and a MIDI version, respectively. Using the Euclidean distance to compare two such decaying sequences leads to a diagonal corridor of low cost in C_{DLNCO} in the case that the directions (i.e., the relative chroma distributions) of the onset vectors are similar, see Fig. 2c. This corridor is tapered to the lower left and starts at the precise time positions of the two onsets to be compared. Second, note that C_{DLNCO} reveals a grid like structure of an overall high cost, where each beginning of a corridor forms a small needle’s eye of low cost. Third, sections in the feature sequences with no onsets lead to regions in C_{DLNCO} having zero cost. In other words, only significant events in the DLNCO feature sequences take effect on the cost matrix level and the structure of C_{DLNCO} regulates the course of a cost-minimizing alignment path in event-based regions to run through the needle’s eyes of low cost.

The cost matrix C_{chroma} accounts for the rough harmonic flow of the two representations, whereas C_{DLNCO} exhibits onsets of the same chroma class. The sum $C = C_{\text{chroma}} + C_{\text{DLNCO}}$ yields a cost matrix that accounts for both types of information. Note that in regions with no onsets, C_{DLNCO} is zero and the combined matrix C is dominated by C_{chroma} . Contrary, in regions with significant onsets, C is dominated by C_{DLNCO} . Therefore, the component C_{chroma} regulates the overall course of the cost-minimizing alignment path and accounts for a robust synchronization, whereas the component C_{DLNCO} locally adjusts the alignment path and accounts for high temporal accuracy.

4. EXPERIMENTS

In this section, we report on some of our synchronization experiments, which have been conducted on a corpus of harmony-based Western music. To allow for a reproduction of our experiments, we used pieces from the RWC music database [3]. In the following, we consider 16 representa-

RWC ID (Comp./Interp., Instr.)	Chroma		DLNCO		Chroma+DLNCO	
	mean	std	mean	std	mean	std
Burg2 (Burgmüller, piano)	50	48	21	17	18	14
C025 (Bach, piano)	27	33	18	27	14	12
C028 (Beethoven, piano)	54	58	131	318	29	40
C031 (Chopin, piano)	57	64	22	68	22	33
C032 (Chopin, piano)	30	47	12	9	13	21
C029 (Schumann, piano)	46	72	94	264	15	36
Average over piano	44	54	50	117	19	26
C003 (Beethoven, orchestra)	116	96	241	338	116	98
C015 (Borodin, strings)	79	68	268	356	82	56
C022 (Brahms, orchestra)	50	54	26	52	17	20
C044 (Rimski-Korsakov, flute/piano)	41	17	22	19	27	15
C048 (Schubert, voice/piano)	55	50	70	173	31	34
Average over non-piano	68	57	125	188	55	45
J001 (Nakamura, piano)	34	59	17	37	14	15
J038 (HH Band, big band)	45	46	85	204	31	64
J041 (Umitsuki Quart., sax/bass/perc.)	39	67	37	117	23	55
P031 (Nagayama, electronic)	68	50	124	217	46	43
P093 (Burke, voice/guitar)	91	95	71	103	40	58
Average over jazz/pop	55	63	67	136	31	47
Average over all	55	58	79	145	34	38

Table 1. Alignment accuracy for the three different synchronization procedures (Chroma, DLNCO, Chroma+DLNCO) on the test database obtained from the RWC database [3]. All values are given in milliseconds.

tive pieces, which are listed in Table 1. These pieces are divided into three groups: six classical piano pieces, five classical pieces of various instrumentations, and five jazz pieces and pop songs. Note that while pure piano music typically comprises the concise note attacks the DLNCO features are designed for, such information is often missing especially in string and general orchestral music. We now show that our extended synchronization framework leads to significant improvements for piano music, while not losing on accuracy for music lacking in clear note attacks.

In the following, we use three different synchronization procedures based on chroma features only, on DLNCO features only, and a combination of these features (Chroma+DLNCO), see Sect. 3. In each experiment we use 50 features per second, i.e., the features have a temporal resolution of 20ms. To automatically determine the accuracy of our synchronization procedures, we used pairs of MIDI and audio versions of each of the 16 pieces listed in Table 1. Here, the audio versions were generated from the MIDI files using a high-quality synthesizer. Thus, for each synchronization pair, the note onset times in the MIDI file are perfectly aligned with the onset times in the respective audio recording. We randomly distorted the MIDI files by splitting up the MIDI files into N segments of equal length (in our experiment we used $N = 20$) and stretching or compressing each segment by a random factor within an allowed distortion range (in our experiments we used a range of $\pm 30\%$). We refer to the resulting MIDI file as the *distorted MIDI file* in contrast to the original *annotation MIDI file*. We synchronized the distorted MIDI file and the associated audio recording and used the resulting alignment path to adjust the note onset times in the distorted MIDI file and to obtain a third MIDI file referred to as *realigned MIDI file*. The accuracy of the synchronization result is then determined by comparing on-

set times of corresponding notes in the realigned MIDI file and the annotation MIDI file. For each of the 16 pieces and for each synchronization procedure Table 1 shows the mean value and the standard deviation over all absolute onset differences. Note that using a combination of chroma and DLNCO features significantly improves the synchronization accuracy: the average onset error for piano music drops from 44ms (Chroma) to 19ms (Chroma+DLNCO). For orchestral or pure string music without clear note attacks, the DLNCO features do not yield any valuable information. For example, in the case of Borodin’s String Quartet (C015), the onset error increases from 79ms (Chroma) to 269ms (DLNCO) when using only the onset features. However, in the combined case, the chroma features overrule the corrupt DLNCO features leading to an onset error of 82ms (Chroma+DLNCO) that is comparable to the chroma only case.

In conclusion, our experiments show that the combination of using chroma and DLNCO onset features significantly improve the synchronization accuracy for music with clear note attacks and does not degrade for music which lacks this information. At this point, one may object that one typically obtains better absolute synchronization results for synthetic audio material (which was used to completely automate our evaluation) than for real audio recordings. We also tested our synchronization on real audio recordings of all 16 pieces, which actually led to similar results as the synthesized examples. Sonifications of the MIDI-audio synchronization results for the real audio files of the 16 pieces have been made available on the website <http://www-mmdb.iai.uni-bonn.de/projects/syncDLNCO/>.

For the future, we will incorporate other types of features that capture local rhythmic information and smooth note transitions for orchestral, string, or brass music [7]. Here, our synchronization framework allows for making the best of each feature type when combining the various information.

5. REFERENCES

- [1] M. Bartsch and G. Wakefield, “Audio thumbnailing of popular music using chroma-based representations,” *IEEE Trans. on Multimedia*, vol. 7, no. 1, pp. 96–104, Feb. 2005.
- [2] J. P. Bello, L. Daudet, S. Abdallah, C. Duxbury, M. Davies, and M. B. Sandler, “A tutorial on onset detection in music signals,” in *IEEE Trans. on Speech and Audio Processing*, vol. 13, no. 5, Sept. 2005.
- [3] M. Goto, H. Hashiguchi, T. Nishimura, and R. Oka, “RWC music database: Popular, classical and jazz music databases,” in *Proc. ISMIR, Paris, France*, 2002.
- [4] N. Hu, R. Dannenberg, and G. Tzanetakis, “Polyphonic audio matching and alignment for music retrieval,” in *Proc. IEEE WASPAA, New Paltz, NY*, October 2003.
- [5] M. Müller, *Information Retrieval for Music and Motion*, Springer, 2007.
- [6] R. J. Turetsky and D. P. W. Ellis, “Force-aligning MIDI syntheses for polyphonic music transcription generation,” in *Proc. ISMIR, Baltimore, USA*, 2003.
- [7] W. You and R. Dannenberg, “Polyphonic music note onset detection using semi-supervised learning,” in *Proc. ISMIR, Vienna, Austria*, 2007.

MATCHMAKER: AN OPEN-SOURCE LIBRARY FOR REAL-TIME PIANO SCORE FOLLOWING AND SYSTEMATIC EVALUATION

Jiyun Park^{1*} Carlos Cancino-Chacón^{2*}

Suhit Chiruthapudi² Juhan Nam¹

¹ Graduate School of Culture Technology, KAIST, South Korea

² Institute of Computational Perception, Johannes Kepler University Linz, Austria

{june, juhan.nam}@kaist.ac.kr, {carlos.cancino_chacon, suhit.chiruthapudi}@jku.at

ABSTRACT

Real-time music alignment, also known as *score following*, is a fundamental MIR task with a long history and is essential for many interactive applications. Despite its importance, there has not been a unified open framework for comparing models, largely due to the inherent complexity of real-time processing and the language- or system-dependent implementations. In addition, low compatibility with the existing MIR environment has made it difficult to develop benchmarks using large datasets available in recent years. While new studies based on established methods (e.g., dynamic programming, probabilistic models) have emerged, most evaluations compare models only within the same family or on small sets of test data. This paper introduces *Matchmaker*, an open-source Python library for real-time music alignment that is easy to use and compatible with modern MIR libraries. Using this, we systematically compare methods along two dimensions: music representations and alignment methods. We evaluated our approach on a large test set of solo piano music from the (n)ASAP, Batik, and Vienna4x22 datasets with a comprehensive set of metrics to ensure robust assessment. Our work aims to establish a benchmark framework for score-following research while providing a practical tool that developers can easily integrate into their applications.

1. INTRODUCTION

Real-time music alignment, also known as *score following*, is the task of aligning performance data to the corresponding position in the musical score in real-time. Ever since it was first introduced independently by Roger Dannenberg [1] and Barry Vercoe [2] over 40 years ago, music alignment has become one of the fundamental MIR tasks. Score following is a necessary component of many inter-

active applications (e.g., automatic accompaniment systems [3–6], automatic page turning [7, 8], lyrics alignment or tracking singing voice [9–11], audiovisual/multimodal [6, 12] and visualizations [13]. Music alignment began as real-time score following [1, 2, 14–17] but, by the mid-90s, had diverged into online and offline methods (see, e.g., early offline work by Desain et al. [18]).

From its early use on monophonic sources like voice [17] and wind instruments, score following has grown to support polyphonic instruments such as piano, ensemble, and even full orchestral performances [17, 19–21]. Research has also expanded across input modalities of the performance, with systems operating on audio or MIDI, and score representations including string format, symbolic score, and sheet image [22].

The score following challenge [23] in MIREX laid the foundation to formalize the evaluation framework, introducing important metrics that include considerations in real-time. However, many subsequent studies have been developed in different environments—ranging from system-dependent [24, 25] to language-dependent [26, 27] implementations—often tailored to specific use cases and without publicly shared source code. As a result, implementations became fragmented across platforms, making it difficult to extend, reproduce, or compare methods in a unified setting. This has hindered the development of a unified evaluation framework and comparison over methods or features on shared datasets remain rare, limiting the generalizability and reproducibility.

In this paper, we address these challenges by proposing a unified, open framework for the evaluation and benchmarking of real-time audio-based score following. Considering public datasets that offer a range of difficulty levels, multiple renditions, and precise beat-level annotations, we base our evaluation on three representative piano performance datasets. We implement this framework as an open-source Python package called *Matchmaker*,¹ that allows real-time execution of representative baselines of score following algorithms. In addition to benchmarking, it supports audio device input and has been validated in application contexts through a standalone demo system.

*Equal contribution.



© J. Park, C. Cancino-Chacón, S. Chiruthapudi and J. Nam. Licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0). **Attribution:** J. Park, C. Cancino-Chacón, S. Chiruthapudi and J. Nam, “Matchmaker: An Open-Source Library for Real-Time Piano Score Following and Systematic Evaluation”, in *Proc. of the 26th Int. Society for Music Information Retrieval Conf.*, Daejeon, South Korea, 2025.

¹ <https://github.com/pymatchmaker/matchmaker>

2. A CONCEPTUAL FRAMEWORK FOR SCORE FOLLOWING

As a way to organize and compare the components of systems for score following, we follow the structure proposed by Müller [28]. This framework consists of three core components: (1) input music representations, (2) features, and (3) online alignment algorithms.

2.1 Music Representation

Score following aligns a fixed reference derived from musical scores with a time-evolving input from a performance. The score can take various symbolic formats (e.g., MIDI, MusicXML) or sheet images, and is typically converted into an intermediate representation such as synthesized audio or event sequences. The performance input may be given as either audio or MIDI, each with distinct representational and computational characteristics. Audio input is continuous and latency-sensitive, while MIDI is discrete and event-based. Instrumental factors also affect alignment design: polyphonic or discrete-pitch instruments (e.g., piano) differ from continuous-pitch sources (e.g., violin, voice). Multi-instrument recordings pose further challenges due to timbral overlap and source ambiguity.

2.2 Features

Chroma features are the most commonly used in music synchronization, with many variants for their computation [29–32]. Other works also use various spectral features such as constant-Q transforms (CQT) [27, 33], non-negative matrix factorization (NMF)-based [34] or spectral template [35] for improved polyphonic alignment. Beyond spectral representations, context-aware features such as onset-based feature [36] or beat-synchronous frames have been introduced to capture temporally salient events useful for alignment. Later work explored learned features, including feedforward mappings [27], semi-supervised decompositions like NMF, and more recent neural approaches [37]. While these offer richer contextual information, they often rely on fixed-length inputs and introduce latency, making real-time usage more challenging.

2.3 Alignment Algorithms

Two major families of alignment algorithms have been used in score following: dynamic programming and probabilistic models.

The dynamic programming approach, especially dynamic time warping (DTW), aligns two sequences by minimizing cumulative cost. Its online variant, On-Line Time Warping (OLTW) [38], enables causal alignment within a fixed-size of window. Variants include windowed [39], parallel [40], and constrained DTW [40, 41], as well as tempo-aware extensions [21, 42].

Probabilistic state-space models offer an alternative by treating alignment as latent state inference under uncertainty [24, 29, 43]. HMM-based systems model each note as a sequence of states (e.g., attack–steady–release), with

extensions including semi-Markov [44], hybrid [19], and Bayesian variants [45]. Kalman filter models and switching state-space systems [46, 47] further incorporate tempo dynamics, while particle filters [12, 29] handle multimodal uncertainty in real time.

Other paradigms include early string-matching algorithms [1] and reinforcement learning-based approaches for multimodal or visual score alignment [48].

3. IMPLEMENTATION

3.1 Python Package Structure

Matchmaker is an open source Python package that implements representative real-time music alignment algorithms within a modular, extensible framework. Figure 1 illustrates the overview of the package and the whole pipeline. The current version of *Matchmaker* provides two types of algorithms: 1) online time warping, with two variants: OLTWDixon, based on the methods proposed in [38, 49], and OLTWArzt, based on [21, 50]; and 2) an HMM-based algorithm, similar to the one used in [3, 47]. A full description of the algorithms and their parameters can be found in the supplementary Appendix.²

Matchmaker supports two main usage scenarios: (1) live streaming mode using the audio device and (2) simulation mode, which processes a performance file as input. Figure 2 shows an example of running live streaming mode with the default setting. The `AudioStream` object handles the input stream by chunking the audio with overlapping windows to avoid padding artifacts. Both the synthesized score audio and the performance audio are passed to a `Processor` object that performs feature extraction. The extracted features are pushed into a queue and consumed by the `OnlineAlignment` object, which runs the alignment methods in real time. *Matchmaker* takes a musical score with all symbolic music formats (MusicXML, MIDI, MEI, etc.) available by *partitura*.³ The returned output is the current position in the score, represented in beats as a musical unit according to the time signature in the piece. More detailed description and API documentation of the package are available here.⁴

3.2 Design and Implementation Details

We provide a simple and user-friendly interface to run the score following with minimal setup. As shown in Figure 2, users can instantiate a `Matchmaker` object with a score file and execute a run that iterates over the estimated score position for each step. To streamline real-time processing, the `AudioStream` class is implemented as a context manager that automatically handles stream initialization and teardown. Furthermore, the alignment process is designed as a generator, enabling users to receive score positions concurrently while the alignment is in progress.

² https://pymatchmaker.github.io/ismir2025_supplementary_materials/

³ <https://github.com/CPJKU/partitura>

⁴ <https://pymatchmaker.readthedocs.io/>

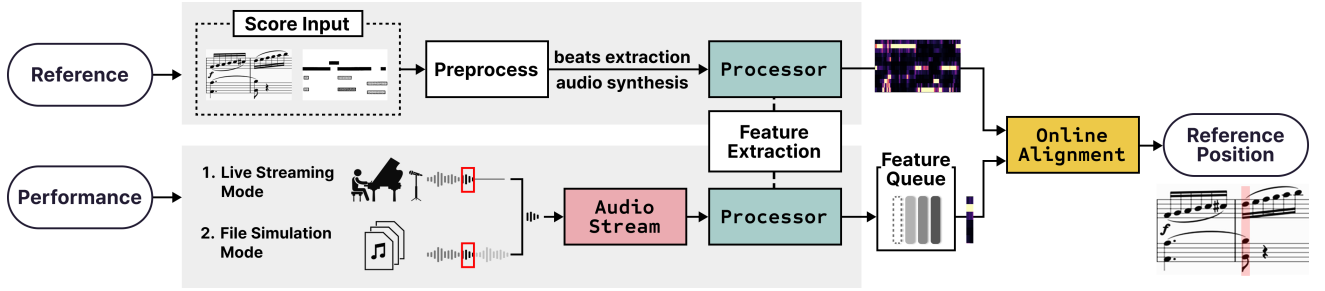


Figure 1. Overview of the score following package

```

1 from matchmaker import Matchmaker
2
3 mm = Matchmaker(
4     score_file="path/to/score.musicxml",
5     input_type="audio",
6 )
7 for current_position in mm.run():
8     print(current_position)

```

Figure 2. A code example for running the *Matchmaker* in a live streaming mode.

This design allows for efficient real-time integration without requiring users to manage multiple threads, buffers, or callbacks explicitly.

While the online mode uses a multi-threaded queue for asynchronous audio buffering, the simulation mode processes audio chunks in advance within a single-threaded setup. By decoupling real-time I/O concerns from core alignment evaluation, it is intended to avoid variability from Python version, OS-level threading, or queuing delays, ensuring a consistent and reproducible benchmarking environment. In addition, OLTWARzt is implemented in Cython [51] for efficiency, a superset of Python designed for C-like performance by incorporating C data types and optimizing the execution of Python code.

4. EXPERIMENTS

4.1 Datasets

We use three public piano performance datasets: (n)ASAP [52], Batik [53] and Vienna 4x22 [54], each of them offering complementary characteristics for benchmarking score following. (n)ASAP, a subset of the MAESTRO dataset including note-level score alignments, includes expressive performances of technically demanding solo piano pieces, offering high difficulty and stylistic diversity. We use only the pieces in the MAESTRO v2 test split. Vienna4x22 provides 22 distinct renditions for each of four relatively easy pieces, which is suitable to test robustness to interpretive variation. Batik dataset contains recordings of 12 Mozart sonatas by a single pianist with the longest average piece duration among the three datasets, enabling evaluation across long-form classical repertoire.

We use ground-truth beat-level annotations provided with the (n)ASAP dataset, and extract equivalent annota-

Dataset	#Pieces	#Perf	#Beats	#Notes	Dur (h)	Difficulty
(n)ASAP	43	59	26,329	100,958	2.65	6.53
Batik	30	30	18,789	102,421	2.85	5.67
Vienna	4	88	13,728	43,656	2.24	4.88
Total	77	177	58,846	247,035	7.74	6.11

Table 1. Datasets used in the evaluation.

tions for Batik and Vienna4x22 from the *.match* files [55], which contain note-wise score-performance alignments. In addition, we incorporate the difficulty levels of each piece based on G. Henle Publishers,⁵ which provides a 1-to-9 grading scale. The pieces used in our experiments span levels 4 through 9, representing a diverse set of works above intermediate level. Table 1 provides the detailed statistics of the datasets.

We only included performances in the experiment that recorded an MAE of less than 100 ms in the offline test, using the *synctoolbox*⁶ with Chroma & DLNCO features. The evaluation was conducted on 184 performances across 93 pieces, totaling over 58,000 beats and 247,000 notes, with an overall duration of 7.74 hours of performances and a piece-wise average difficulty of 6.11.

4.2 Experiment Settings

We conducted all evaluations under simulation-based conditions to ensure reproducibility. Live testing was avoided due to variability introduced by room acoustics and hardware setup, which complicates fair comparison across systems. The accuracy tests were carried out on an Intel i9-9900K CPU (16 cores @ 3.6 GHz), Python 3.9, with a sample rate of 44.1 kHz and a frame rate of 30, chosen to balance latency and alignment accuracy. We tested chromagram, mel-spectrogram, constant-Q transform (CQT), mel-frequency cepstral coefficients (MFCCs) [56] and a simple STFT-based onset-sensitive representation similar to the one used in Dixon [38], which we name log-spectral energy (LSE). While results for all features were evaluated, we report detailed latency and accuracy metrics for the best-performing configuration of each model. To account for hardware variability, latency was measured in multiple setups: an Intel i9-9900K, an Apple M4 MacMini, and an

⁵ <https://www.henle.de/Levels-of-Difficulty/>

⁶ <https://github.com/meinardmueller/synctoolbox>

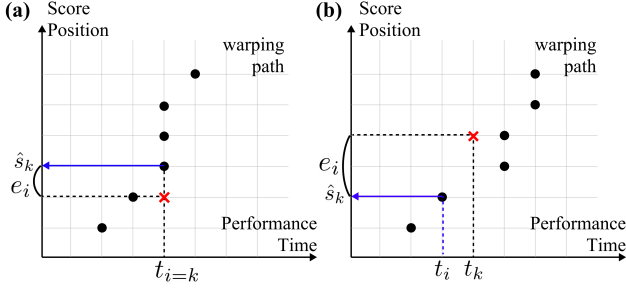


Figure 3. Two examples of error calculation using the mapping function. (a) shows a one-to-many alignment at the evaluation point, while (b) illustrates a skipped alignment.

Apple M2 Pro MacBook, with the reported latency values averaged across these devices.

4.3 Preprocessing

In the preprocessing step (see Fig. 1), the symbolic scores are synthesized to audio using FluidSynth, provided by *partitura*. Since MusicXML often lacks tempo markings, we set the synthesis tempo to each performance’s average—rounded to the nearest 20 BPM—assuming performers follow approximate tempo indications.

To generate beat annotations for the synthesized score audio, we computed beat positions using the synthesis tempo and the score’s time signature. For compound meters (e.g., 6/8, 9/8, and 12/8), we adopted (n)ASAP’s beat annotation rules—counting them as two, three, and four beats per measure, respectively—across all datasets to align score-side annotations with performance annotations. Based on the synthesized audio, we then extract the feature using the same `Processor` used in the online phase, but precompute them offline for the entire score sequence.

5. EVALUATION

Evaluating score following is challenging due to causality, timing precision, and output latency. Since the MIREX challenge [23] provided foundational metrics, later studies introduced alternative evaluation strategies including beat-level evaluations or asynchrony [3], reflecting the task’s frequent integration with automatic accompaniment systems.

In this work, we adopt two complementary evaluation perspectives. First, we evaluate in the performance domain, where errors are measured in milliseconds based on ground-truth annotations aligned to the audio. This approach is commonly used in audio-to-score alignment research and enables precise, frame-level evaluation, since the annotations directly reflect the actual timing of the performance. Second, we also evaluate in the score domain measured in beat units as suggested in [29, 57], which better reflects the nature of score following as a task of predicting the corresponding score position at each moment of the performance.

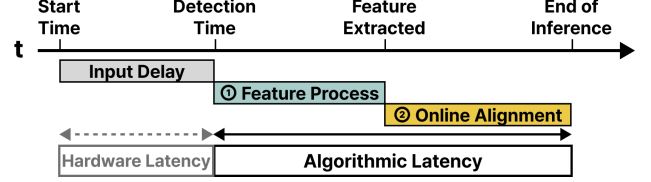


Figure 4. Defined delay types of the system. Only system delay is considered in the experiment.

5.1 Evaluation Metrics

We select evaluation metrics mostly adapted from score following MIREX benchmark [23] and audio-to-score alignment (ASA) metrics [57]. We use **Alignment Rate** (AR) within a tolerance range of $|\theta_e|$, varying from 50 ms to 2000 ms. We also compute **Absolute Errors** (AE), both in milliseconds and in beats, from which we derive the **Average Absolute Error** (AAE) and **Median Absolute Error** (MAE), along with the standard deviation σ_e . To further characterize the distribution of errors, we report **kurtosis** and **skewness** which capture the peakedness and asymmetry of the non-absolute error distribution, respectively. In addition, we report the **average latency** μ_{lat} , defined as the *system delay* from the detection time to the end of inference. Unlike total latency, this excludes *hardware latency* and is composed of two parts: (i) feature processing and (ii) execution of the online alignment algorithm for each frame step (see Fig. 4). Errors exceeding 2 seconds (or 2 beats in the score domain) are excluded from AE calculations, including both AAE and MAE, to avoid distortion from unbounded tracking failures. We report AR in two ways. The averaged piece-wise AR is a common measure, while the total AR reflects the proportion of successfully aligned beat events across the entire dataset. The latter avoids overrepresentation of shorter pieces and provides a more balanced view of overall performance.

To evaluate runtime latency under simulation, we measure two components: the average duration for extracting features from incoming audio frames, and the time taken by the alignment process to consume features and predict score positions. Specifically, the latency was computed from the moment audio was read to the time the score position was predicted—excluding hardware I/O delays. This two-step measurement allows for standardized latency reporting independent of the hardware setup.

5.2 Alignment Mapping Function

Given the alignment path, the alignment mapping function is applied to transfer the beat positions on one axis (either performance or score) to another axis to compute the alignment error. Due to the local, stepwise nature of real-time alignment, the resulting path is not necessarily monotonic and may contain multiple correspondents or skipped positions, depending on the implementation and purpose of the methods. Unlike linear interpolation methods commonly used in offline audio-to-score alignment, which assume continuous mappings, our evaluation relies only on

Dataset	Method	AAE(ms)↓ ± σ	MAE(ms)↓	Skew.	Kurt.	Piece-wise AR (%) ↑					Total AR↑ (≤2000ms, %)
						≤50ms	≤100ms	≤500ms	≤1000ms	≤2000ms	
(n)ASAP	OLTWDixon	189.55 ± 281.55	97.09	3.20	17.97	40.3	58.5	82.5	88.3	92.0	89.4
	OLTWARzt	183.56 ± 263.95	91.18	0.75	11.79	44.1	58.3	84.8	92.0	95.1	92.8
	HMM	487.73 ± 423.27	346.01	0.18	3.33	15.6	22.2	37.5	43.8	43.8	43.8
Batik	OLTWDixon	186.97 ± 262.55	<u>104.40</u>	3.75	24.70	28.2	51.7	82.1	85.2	87.6	<u>89.4</u>
	OLTWARzt	193.36 ± 269.13	<u>107.15</u>	1.00	12.63	35.9	53.0	82.2	87.4	90.3	<u>89.7</u>
	HMM	693.63 ± 376.58	641.77	0.11	0.98	4.5	10.8	34.0	46.2	64.2	61.9
Vienna4x22	OLTWDixon	285.43 ± 390.82	132.73	1.57	5.90	26.6	43.2	72.4	80.0	85.5	82.5
	OLTWARzt	300.41 ± 368.70	152.51	0.50	3.93	33.2	44.5	73.3	84.3	86.7	86.7
	HMM	439.64 ± 427.02	319.13	0.15	3.79	23.5	33.3	51.1	57.1	63.0	75.9

Table 2. Evaluation results on three datasets using different score-following methods. The piece-wise alignment rate (AR) is measured as the average over pieces, while the total AR indicates the global proportion of aligned beat events across the entire dataset. All tests were conducted with STFT-based Chroma as features.

predictions made prior to or at each evaluation time point. To reflect this, we define the mapping function as follows:

$$\hat{u}_k = \min\{u_i \mid (u_i, v_i) \in \mathcal{W}, v_i = \max\{v_j \mid v_j \leq k\}\},$$

where $\mathcal{W} = \{(u_i, v_i)\}$ is the warping path expressed in the frame indices: u_i is the score-rendered-audio frame index and v_i is the performance-audio frame index. The inner max finds the latest performance frame v_i not exceeding the current frame k , and the outer min selects the smallest score frame u_i among those alignments. This mapping relies solely on past or current frames to maintain causality. It handles skipped or one-to-many mappings and avoids any interpolation methods that depend on future frames.

6. RESULTS

Table 2 presents a comparison of alignment methods based on performance-domain evaluation, measured in milliseconds. All methods exhibit positive skewness in error distribution, reflecting the expected lag of the beat estimates in real-time alignment. The overall results show that the OLTW-based method outperforms the HMM baseline across all datasets in both alignment accuracy and coverage. While OLTWDixon and OLTWARzt show comparable MAE depending on the dataset, OLTWARzt consistently achieves higher coverage (*Total AR*), suggesting that it is more robust against overall failures. The difference likely stems from OLTWDixon skipping uncertain regions, while OLTWARzt’s “backward-forward” strategy corrects early misalignments and enhances coverage. Despite having the lowest AR, the HMM shows the lowest skewness and kurtosis primarily because significant errors (>2 s) are excluded from the summary statistics and its “sticky” behavior to linger in the same state in local regions tends to narrow the error distribution.

Table 3 presents an evaluation comparison in beat units, offering a tempo-normalized perspective. The overall trend mirrors the performance-domain results in milliseconds, but these results are standardized across tempi. AAE remains around 0.3 beats, with median values typically below 0.2. Total AR is consistently lower than the 2000 ms-

Dataset	Method	AAE↓(beats) ± σ	MAE↓(beats)	AR↑ (%)
(n)ASAP	OLTWDixon	0.22 ± 0.27	0.13	83.4
	OLTWARzt	0.27 ± 0.30	0.16	85.2
	HMM	0.80 ± 0.54	0.66	76.9
Batik	OLTWDixon	0.20 ± 0.27	0.11	88.9
	OLTWARzt	0.29 ± 0.34	0.18	88.8
	HMM	0.80 ± 0.38	0.67	59.3
Vienna4x22	OLTWDixon	0.31 ± 0.33	0.19	78.3
	OLTWARzt	0.37 ± 0.38	0.24	84.0
	HMM	0.76 ± 0.78	0.51	70.3

Table 3. Beat-level evaluation results including total alignment rate (AR) (%).

Feature Process			Online Alignment	
Type	MAE (ms)	Latency (ms)	Method	Latency (ms)
Chroma	265.50	3.05	OLTWDixon	1.22
mel	297.92	3.40	OLTWARzt	0.07
CQT	341.25	42.58	HMM	3.59
LSE	241.85	0.91		
MFCC	931.81	2.58		

Table 4. Comparison of feature types and alignment methods in terms of alignment error (MAE) and latency. LSE is log-spectral energy feature that was adopted in [38]. Latency values are averaged over the hardware setups evaluated in Section 4.

based metric, reflecting that most pieces have tempi above 60BPM, where two beats span less than two seconds.

In addition, a comparison of various feature types and latencies of the alignment methods are reported in Table 4. Among the features, log-spectral energy (LSE) shows the lowest MAE (241.85 ms) and delay (0.91 ms), indicating strong performance with minimal overhead. In contrast, CQT and MFCC yield higher MAE, with CQT also requiring considerable extraction time (42.58 ms), which limits its real-time suitability. For alignment methods, OLTWARzt achieves the lowest latency (0.07 ms), whereas HMM shows noticeably higher delay (3.59 ms) due to its computational complexity. These results highlight a trade-off between alignment accuracy and runtime efficiency,

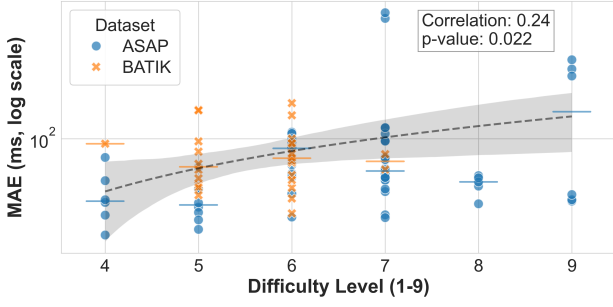


Figure 5. A scatter plot of mean absolute error (MAE) and Henle’s difficulty level in (n)ASAP and Batik dataset. The MAE results are from OLTWARzt.

with LSE and OLTWARzt providing a favorable balance for low-latency use.

The results also show about the characteristics of the datasets. While the overall alignment performance between (n)ASAP and Batik is comparable, Vienna4x22 shows noticeably higher error variance and kurtosis. This reflects the dataset’s unique structure—22 diverse renditions for each of only four pieces—leading to substantial variability in expressive timing, articulation, and interpretation. These variations present additional challenges for score following and result in heavier-tailed error distributions, as seen in the higher kurtosis values.

7. DISCUSSIONS

Figure 5 further illustrates the relationship between musical difficulty and alignment accuracy for (n)ASAP and Batik. We observe a moderate positive correlation ($r = 0.24$, $p = 0.022$) between MAE and the annotated difficulty levels, indicating that technically more demanding pieces tend to produce larger alignment errors. Vienna4x22 was excluded from this analysis due to its use of short excerpts, which makes consistent difficulty grading unreliable.

To further understand how alignment behaviors differ from methods, Figure 6 illustrates an example of alignment result comparing OLTWARzt (left) and HMM (right). Although OLTWARzt smoothly follows the beat events, the HMM warping path shows frequent horizontal segments, indicating the “sticky” tendency to stay near note onsets, reflecting its state-based formulation that emphasizes onset transitions. This leads to cases where it lingers on sustained notes and becomes locally stuck, showing limited forward momentum. The corresponding region (highlighted in yellow) exhibits changes in harmony, note density, and dynamics compared to the preceding passage, which provides sufficient contrast for the score follower to recover.

Lastly, we found that not only the choice of evaluation metrics, but also how alignment errors are computed (Section 5.2) can affect accuracy results to a meaningful extent. Small differences in error calculation sometimes led to noticeable shifts in reported accuracy.

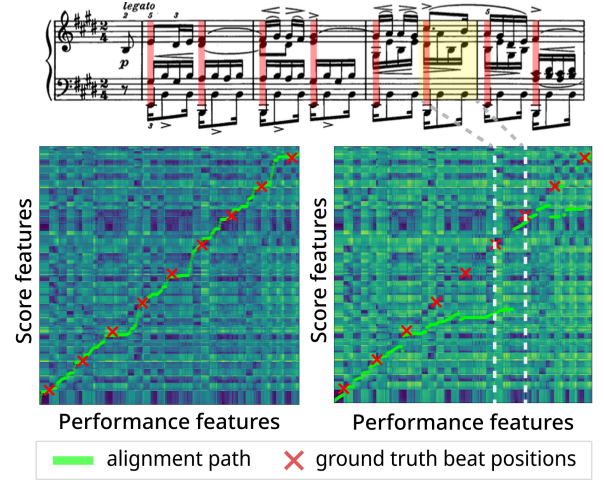


Figure 6. Two examples of alignment path with beat positions: (left) OLTWARzt, (right) HMM.

8. USE CASES AND APPLICATIONS

To demonstrate the practicality of our package, we built a lightweight web application that runs locally with real-time audio input or pre-recorded files. Built with websocket-based communication, the system responds quickly enough to ensure minimal perceptual delay. Our companion website includes a video demonstration and a link to the source code. This application aims to help researchers test their own score following models in an interactive setting. Beyond the web demo, our package is also used as the score following module in the ACCompanion [3], a real-time accompaniment system. These applications demonstrate the versatility of our framework and validate its utility in interactive music scenarios.

9. CONCLUSIONS AND FUTURE WORK

We presented a systematic framework for real-time audio-based score following as the open-source Python package *Matchmaker*. It supports live and simulation-based evaluation with baseline models, enabling reproducible benchmarking across datasets and features. Experiments on three public piano datasets show that the OLTWARzt variant achieves the highest performance and that the onset-sensitive spectral feature (LSE) outperforms chroma in both accuracy and latency. However, the current framework is limited in its support for tempo models commonly integrated with HMM-based score followers which may partly explain the limited performance of the HMM baseline. Also, recent works often include learned features or multimodal input which poses a new challenge to evaluate. Although our evaluation was limited to classical piano, extending *Matchmaker* to other instruments and genres requires only adapting the proper datasets and feature extraction modules. Future work will extend the framework to support a wider variety of instruments and musical styles, and include additional feature representations, advanced tempo modeling, and multimodal inputs.

10. ACKNOWLEDGMENTS

This work has been supported by the Austrian Science Fund (FWF), grant agreement PAT 8820923 (“*Rach3: A Computational Approach to Study Piano Rehearsals*”). Additionally, this work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) under Grant RS-2023-NR077289.

11. REFERENCES

- [1] R. B. Dannenberg, “An On-Line Algorithm for Real-Time Accompaniment,” in *Proceedings of the International Computer Music Conference (ICMC '84)*, Paris, France, 1984.
- [2] B. Vercoe, “The Synthetic Performer in the Context of Live Performance,” in *Proceedings of the International Computer Music Conference (ICMC '84)*, Paris, France, 1984.
- [3] C. Cancino-Chacon, S. Peter, P. Hu, E. Karystinaios, F. Henkel, F. Foscarin, N. Varga, and G. Widmer, “The ACCompanion: Combining Reactivity, Robustness, and Musical Expressivity in an Automatic Piano Accompanist,” in *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI-23)*, Macao, SAR, China, May 2023, arXiv:2304.12939 [cs, eess]. [Online]. Available: <http://arxiv.org/abs/2304.12939>
- [4] K. Armstrong, T.-C. Hung, J.-X. Huang, and Y.-W. Liu, “Real-time piano accompaniment model trained on and evaluated according to human ensemble characteristics,” in *Proceedings of the Sound and Music Computing (SMC)*, Porto, Portugal, 2024.
- [5] C. Raphael, “Music Plus One and Machine Learning,” in *Proceedings of the 27th International Conference on Machine Learning (ICML 2010)*, Haifa, Israel, 2010.
- [6] A. Maezawa, “I Got Rhythm, so Follow Me More: Modeling Score-Dependent Timing Synchronization in a Piano Duet,” in *Proceedings of the Sound and Music Computing Conference (SMC 2024)*, Porto, Portugal, 2024.
- [7] A. Arzt, G. Widmer, and S. Dixon, “Automatic Page Turning for Musicians via Real-Time Machine Listening,” in *Proceedings of the European Conference on Artificial Intelligence (ECAI)*, 2008.
- [8] F. Henkel, S. Schwaiger, and G. Widmer, “Fully Automatic Page Turning on Real Scores,” in *Extended Abstracts for the Late-Breaking Demo Session of the 22nd International Society for Music Information Retrieval Conference (ISMIR 2021)*, Online, 2021, arXiv:2111.06643 [cs]. [Online]. Available: <http://arxiv.org/abs/2111.06643>
- [9] C. Brazier and G. Widmer, “Towards Reliable Real-time Opera Tracking: Combining Alignment with Audio Event Detectors to Increase Robustness,” in *Proceedings of the Sound and Music Computing Conference*, Online, 2020, arXiv:2006.11033 [cs, eess]. [Online]. Available: <http://arxiv.org/abs/2006.11033>
- [10] J. Park, S. Yong, T. Kwon, and J. Nam, “A Real-Time Lyrics Alignment System Using Chroma and Phonetic Features for Classical Vocal Performance,” in *ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. Seoul, Korea, Republic of: IEEE, Apr. 2024, pp. 1371–1375. [Online]. Available: <https://ieeexplore.ieee.org/document/10445926/>
- [11] R. Gong, P. Cuvillier, N. Obin, and A. Cont, “Real-time audio-to-score alignment of singing voice based on melody and lyric information,” in *Interspeech 2015*. ISCA, Sep. 2015, pp. 3312–3316. [Online]. Available: https://www.isca-archive.org/interspeech_2015/gong15_interspeech.html
- [12] T. Otsuka, K. Nakadai, T. Takahashi, T. Ogata, and H. G. Okuno, “Real-Time Audio-to-Score Alignment Using Particle Filter for Coplayer Music Robots,” *EURASIP Journal on Advances in Signal Processing*, vol. 2011, no. 1, p. 384651, Dec. 2011. [Online]. Available: <https://asp-eurasipjournals.springeropen.com/articles/10.1155/2011/384651>
- [13] O. Lartillot, C. Cancino-Chacon, and C. Brazier, “Real-Time Visualisation of Fugue Played by a String Quartet,” in *Proceedings of the Sound and Music Computing Conference (SMC 2020)*, Online, 2020.
- [14] R. Dannenberg and B. Mont-Reynaud, “Following an Improvisation in Real Time,” in *Proceedings of the International Computer Music Conference*, Champaign/Urbana, Illinois, USA, 1987.
- [15] R. B. Dannenberg and H. Mukaino, “New Techniques for Enhanced Quality of Computer Accompaniment,” in *Proceedings of the International Computer Music Conference*, Cologne, Germany, 1988.
- [16] B. Vercoe and M. Puckette, “Synthetic Rehearsal: Training the Synthetic Performer,” in *Proceedings of the International Computer Music Conference (ICMC '85)*, Vancouver, BC, Canada, 1985.
- [17] M. Puckette, “Score following using the sung voice,” in *Proceedings of the International Computer Music Conference (ICMC '95)*, Banff, AB, Canada, 1995.
- [18] P. Desain, H. Honing, and H. Heijink, “Robust Score-Performance Matching: Taking Advantage of Structural Information,” in *Proceedings of the International Computer Music Conference (ICMC)*, Thessalonki, Greece, 1997.
- [19] C. Raphael and Y. Gu, “Orchestral Accompaniment for a Reproducing Piano,” in *Proceedings of the International Computer Music Conference (ICMC 2009)*, Montreal, Canada, 2009.

- [20] M. Prockup, D. Grunberg, A. Hrybyk, and Y. E. Kim, "Orchestral Performance Companion: Using Real-Time Audio to Score Alignment," *IEEE MultiMedia*, vol. 20, no. 2, pp. 52–60, Apr. 2013. [Online]. Available: <http://ieeexplore.ieee.org/document/6530591/>
- [21] A. Arzt and G. Widmer, "Real-Time Music Tracking Using Multiple Performances as a Reference," in *Proceedings of the 16th International Society for Music Information Retrieval Conference (ISMIR 2015)*, Malaga, Spain, 2015.
- [22] F. Henkel, S. Balke, M. Dorfer, and G. Widmer, "Score Following as a Multi-Modal Reinforcement Learning Problem," *Transactions of the International Society for Music Information Retrieval*, vol. 2, no. 1, pp. 67–81, Nov. 2019. [Online]. Available: <http://transactions.ismir.net/articles/10.5334/tismir.31/>
- [23] A. Cont, D. Schwarz, N. Schnell, and C. Raphael, "Evaluation of Real-Time Audio-to-Score Alignment," in *Music Information Retrieval Evaluation eXchange (MIREX 2007)*, Vienna, Austria, 2007.
- [24] A. Cont, "Antescofo: Anticipatory Synchronization and Control of Interactive Parameters in Computer Music," in *Proceedings of the International Computer Music Conference (ICMC '08)*, Belfast, Ireland, 2008.
- [25] J. Echeveste, P. Cuvillier, and A. Cont, "Improved Synchronization of a Pre-Recorded Music Accompaniment on a User's Music Playing," U.S. Patent US 2023/0 082 086 A1, Mar., 2023.
- [26] S. Dixon and G. Widmer, "MATCH: A Music Alignment Tool Chest," in *Proceedings of the 6th International Conference on Music Information Retrieval (ISMIR 2005)*, London, UK, 2005.
- [27] C. Joder, S. Essid, and G. Richard, "Learning Optimal Features for Polyphonic Audio-to-Score Alignment," *IEEE Transactions on Audio, Speech, and Language Processing*, vol. 21, no. 10, pp. 2118–2128, Oct. 2013. [Online]. Available: <http://ieeexplore.ieee.org/document/6525340/>
- [28] M. Muller, F. Kurth, and T. Roder, "Towards an Efficient Algorithm for Automatic Score-to-Audio Synchronization," in *Proceedings of the 5th International Conference on Music Information Retrieval (ISMIR 2004)*, Barcelona, Spain, 2004.
- [29] Z. Duan and B. Pardo, "A state space model for online polyphonic audio-score alignment," in *2011 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. Prague, Czech Republic: IEEE, May 2011, pp. 197–200. [Online]. Available: <http://ieeexplore.ieee.org/document/5946374/>
- [30] P.-W. Chou, F.-N. Lin, K.-N. Chang, and H.-Y. Chen, "A Simple Score Following System for Music Ensembles Using Chroma and Dynamic Time Warping," in *Proceedings of the 2018 ACM on International Conference on Multimedia Retrieval*. Yokohama Japan: ACM, Jun. 2018, pp. 529–532. [Online]. Available: <https://dl.acm.org/doi/10.1145/3206025.3206090>
- [31] M. Muller, "Music Synchronization," in *Fundamentals of Music Processing*. Cham: Springer International Publishing, 2021, pp. 119–170. [Online]. Available: https://link.springer.com/10.1007/978-3-030-69808-9_3
- [32] M. Pérez Fernández, H. Kirchhoff, and X. Serra, "A comparison of pitch chroma extraction algorithms," in *Proceedings of the 19th Sound and Music Computing Conference (SMC/JIM/IFC 2022)*. Saint-Étienne, France: SMC Network, 2022. [Online]. Available: <https://doi.org/10.5281/zenodo.6573082>
- [33] C.-T. Chen, J.-S. R. Jang, W.-S. Liu, and C.-Y. Weng, "An efficient method for polyphonic audio-to-score alignment using onset detection and constant Q transform," in *2016 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. Shanghai: IEEE, Mar. 2016, pp. 2802–2806. [Online]. Available: <http://ieeexplore.ieee.org/document/7472188/>
- [34] J. J. Carabias-Orti, F. J. Rodriguez-Serrano, P. Vera-Candeas, and N. Ruiz-Reyes, "An Audio to Score Alignment Framework Using Spectral Factorization and Dynamic Time Warping," in *Proceedings of the 16th International Society for Music Information Retrieval Conference (ISMIR 2015)*, Malaga, Spain, 2015.
- [35] F. Korzeniowski and G. Widmer, "Refined Spectral Template Models for Score Following," in *Proceedings of the Sound and Music Computing Conference (SMC 2013)*, Stockholm, Sweden, 2013.
- [36] S. Ewert, M. Muller, and P. Grosche, "High resolution audio synchronization using chroma onset features," in *2009 IEEE International Conference on Acoustics, Speech and Signal Processing*. IEEE, 2009, pp. 1869–1872.
- [37] A. Pillay, "A Neural Score Follower for Computer Accompaniment of Polyphonic Musical Instruments," Master's thesis, Carnegie Mellon University, Pittsburgh, PA, USA, 2024.
- [38] S. Dixon, "An On-Line Time Warping Algorithm for Tracking Musical Performances," in *Proceedings of the 19th International Joint Conference on Artificial Intelligence (IJCAI 05)*, Edinburgh, Scotland, 2005.
- [39] R. Macrae and S. Dixon, "Polyphonic Score Following Using on-Line Time Warping," in *Music Information Retrieval Evaluation eXchange (MIREX 2008)*, Philadelphia, USA, 2008.

- [40] F. J. Rodriguez-Serrano, P. Vera-Candeas, and J. J. Carabias-Orti, "A Real-Time Score Follower for MIREX 2015," in *Music Information Retrieval Evaluation eXchange (MIREX 2015)*, Malaga, Spain, 2015.
- [41] J. J. Carabias, F. J. Rodriguez, and P. Vera, "A Real-Time Nmf-Based Score Follower for MIREX 2012," in *Music Information Retrieval Evaluation eXchange (MIREX 2012)*, Porto, Portugal, 2012.
- [42] A. Arzt, "Real-Time Music Tracking Using Tempo-Aware on-Line Dynamic Time Warping," in *Proceedings of the Vienna Talk on Musical Acoustics (VITA)*, Vienna, Austria, 2010.
- [43] P. Cano, A. Loscos, and J. Bonada, "Score-Performance Matching using HMMs," in *Proceedings of the International Computer Music Conference*, Beijing, China, 1999.
- [44] E. Nakamura, P. Cuvillier, and A. Cont, "Autoregressive Hidden Semi-Markov Model of Symbolic Music Performance for Score Following," in *Proceedings of the 16th International Society for Music Information Retrieval Conference (ISMIR 2015)*, Malaga, Spain, 2015. [Online]. Available: <https://archives.ismir.net/ismir2015/paper/000015.pdf>
- [45] C. Raphael, "A Bayesian Network for Real-Time Musical Accompaniment," in *Proceedings of the 14th International Conference on Neural Information Processing Systems*, Vancouver, BC, Canada, 2001, pp. 1433–1439. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2001/file/2b0f658cbffd284984fb11d90254081f-Paper.pdf
- [46] R. Yamamoto, S. Sako, and T. Kitamura, "Real-Time Audio to Score Alignment Using Segmental Conditional Random Fields and Linear Dynamical System," in *Music Information Retrieval Evaluation eXchange (MIREX 2012)*, Porto, Portugal, 2012.
- [47] Y. Jiang and C. Raphael, "Score Following with Hidden Tempo Using a Switching State-Space Model," in *Proceedings of the 21st International Society for Music Information Retrieval Conference (ISMIR 2020)*, Online, 2020.
- [48] M. Dorfer, F. Henkel, and G. Widmer, "Learning to Listen, Read, and Follow: Score Following as a Reinforcement Learning Game," in *Proceedings of the 19th International Society for Music Information Retrieval Conference (ISMIR 2018)*, Paris, France, 2018.
- [49] S. Dixon, "Live Tracking of Musical Performances using On-Line Time Warping," in *Proceedings of the 8th International Conference on Digital Audio Effects (DAFx'05)*, Madrid, Spain, 2005.
- [50] A. Arzt and G. Widmer, "Towards Effective Any-Time Music Tracking," in *Proceedings of the Starting AI Researchers Symposium (STAIRS), held at ECAI 2010*, Lisbon, Portugal, 2010.
- [51] S. Behnel, R. Bradshaw, C. Citro, L. Dalcin, D. S. Seljebotn, and K. Smith, "Cython: The best of both worlds," *Computing in Science & Engineering*, vol. 13, no. 2, pp. 31–39, 2011.
- [52] S. D. Peter, C. E. Cancino-Chacón, F. Foscarin, A. P. McLeod, F. Henkel, E. Karystinaios, and G. Widmer, "Automatic Note-Level Score-to-Performance Alignments in the ASAP Dataset," *Transactions of the International Society for Music Information Retrieval*, vol. 6, no. 1, pp. 27–42, Jun. 2023. [Online]. Available: <http://transactions.ismir.net/articles/10.5334/tismir.149/>
- [53] P. Hu and G. Widmer, "The Batik-plays-Mozart Corpus: Linking Performance to Score to Musicological Annotations," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2023.
- [54] W. Goebel, "The vienna 4x22 piano corpus," 1999. [Online]. Available: <http://dx.doi.org/10.21939/4X22>
- [55] F. Foscarin, E. Karystinaios, S. D. Peter, C. Cancino-Chacón, M. Grachten, and G. Widmer, "The match file format: Encoding alignments between scores and performances," in *Proceedings of the Music Encoding Conference (MEC 2022)*, Halifax, Canada.
- [56] C. Brazier and G. Widmer, "Addressing the Recitative Problem in Real-time Opera Tracking," in *Proceedings of Frontiers of Research in Speech and Music FRSM 2020*, Online, Oct. 2020, arXiv:2010.11013 [eess].
- [57] A. Morsi and X. Serra, "Bottlenecks and solutions for audio to score alignment research," in *Proceedings of the 23rd International Society for Music Information Retrieval Conference (ISMIR 2022)*, 2022, pp. 272–279.

MATCHMAKER: AN OPEN-SOURCE LIBRARY FOR REAL-TIME PIANO SCORE FOLLOWING AND SYSTEMATIC EVALUATION

Jiyun Park^{1*} Carlos Cancino-Chacón^{2*}

Suhit Chiruthapudi² Juhan Nam¹

¹ Graduate School of Culture Technology, KAIST, South Korea

² Institute of Computational Perception, Johannes Kepler University Linz, Austria

{june, juhan.nam}@kaist.ac.kr, {carlos.cancino_chacon, suhit.chiruthapudi}@jku.at

ABSTRACT

Real-time music alignment, also known as *score following*, is a fundamental MIR task with a long history and is essential for many interactive applications. Despite its importance, there has not been a unified open framework for comparing models, largely due to the inherent complexity of real-time processing and the language- or system-dependent implementations. In addition, low compatibility with the existing MIR environment has made it difficult to develop benchmarks using large datasets available in recent years. While new studies based on established methods (e.g., dynamic programming, probabilistic models) have emerged, most evaluations compare models only within the same family or on small sets of test data. This paper introduces *Matchmaker*, an open-source Python library for real-time music alignment that is easy to use and compatible with modern MIR libraries. Using this, we systematically compare methods along two dimensions: music representations and alignment methods. We evaluated our approach on a large test set of solo piano music from the (n)ASAP, Batik, and Vienna4x22 datasets with a comprehensive set of metrics to ensure robust assessment. Our work aims to establish a benchmark framework for score-following research while providing a practical tool that developers can easily integrate into their applications.

1. INTRODUCTION

Real-time music alignment, also known as *score following*, is the task of aligning performance data to the corresponding position in the musical score in real-time. Ever since it was first introduced independently by Roger Dannenberg [1] and Barry Vercoe [2] over 40 years ago, music alignment has become one of the fundamental MIR tasks. Score following is a necessary component of many inter-

active applications (e.g., automatic accompaniment systems [3–6], automatic page turning [7, 8], lyrics alignment or tracking singing voice [9–11], audiovisual/multimodal [6, 12] and visualizations [13]. Music alignment began as real-time score following [1, 2, 14–17] but, by the mid-90s, had diverged into online and offline methods (see, e.g., early offline work by Desain et al. [18]).

From its early use on monophonic sources like voice [17] and wind instruments, score following has grown to support polyphonic instruments such as piano, ensemble, and even full orchestral performances [17, 19–21]. Research has also expanded across input modalities of the performance, with systems operating on audio or MIDI, and score representations including string format, symbolic score, and sheet image [22].

The score following challenge [23] in MIREX laid the foundation to formalize the evaluation framework, introducing important metrics that include considerations in real-time. However, many subsequent studies have been developed in different environments—ranging from system-dependent [24, 25] to language-dependent [26, 27] implementations—often tailored to specific use cases and without publicly shared source code. As a result, implementations became fragmented across platforms, making it difficult to extend, reproduce, or compare methods in a unified setting. This has hindered the development of a unified evaluation framework and comparison over methods or features on shared datasets remain rare, limiting the generalizability and reproducibility.

In this paper, we address these challenges by proposing a unified, open framework for the evaluation and benchmarking of real-time audio-based score following. Considering public datasets that offer a range of difficulty levels, multiple renditions, and precise beat-level annotations, we base our evaluation on three representative piano performance datasets. We implement this framework as an open-source Python package called *Matchmaker*,¹ that allows real-time execution of representative baselines of score following algorithms. In addition to benchmarking, it supports audio device input and has been validated in application contexts through a standalone demo system.

*Equal contribution.



© J. Park, C. Cancino-Chacón, S. Chiruthapudi and J. Nam. Licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0). **Attribution:** J. Park, C. Cancino-Chacón, S. Chiruthapudi and J. Nam, “Matchmaker: An Open-Source Library for Real-Time Piano Score Following and Systematic Evaluation”, in *Proc. of the 26th Int. Society for Music Information Retrieval Conf.*, Daejeon, South Korea, 2025.

¹ <https://github.com/pymatchmaker/matchmaker>

2. A CONCEPTUAL FRAMEWORK FOR SCORE FOLLOWING

As a way to organize and compare the components of systems for score following, we follow the structure proposed by Müller [28]. This framework consists of three core components: (1) input music representations, (2) features, and (3) online alignment algorithms.

2.1 Music Representation

Score following aligns a fixed reference derived from musical scores with a time-evolving input from a performance. The score can take various symbolic formats (e.g., MIDI, MusicXML) or sheet images, and is typically converted into an intermediate representation such as synthesized audio or event sequences. The performance input may be given as either audio or MIDI, each with distinct representational and computational characteristics. Audio input is continuous and latency-sensitive, while MIDI is discrete and event-based. Instrumental factors also affect alignment design: polyphonic or discrete-pitch instruments (e.g., piano) differ from continuous-pitch sources (e.g., violin, voice). Multi-instrument recordings pose further challenges due to timbral overlap and source ambiguity.

2.2 Features

Chroma features are the most commonly used in music synchronization, with many variants for their computation [29–32]. Other works also use various spectral features such as constant-Q transforms (CQT) [27, 33], non-negative matrix factorization (NMF)-based [34] or spectral template [35] for improved polyphonic alignment. Beyond spectral representations, context-aware features such as onset-based feature [36] or beat-synchronous frames have been introduced to capture temporally salient events useful for alignment. Later work explored learned features, including feedforward mappings [27], semi-supervised decompositions like NMF, and more recent neural approaches [37]. While these offer richer contextual information, they often rely on fixed-length inputs and introduce latency, making real-time usage more challenging.

2.3 Alignment Algorithms

Two major families of alignment algorithms have been used in score following: dynamic programming and probabilistic models.

The dynamic programming approach, especially dynamic time warping (DTW), aligns two sequences by minimizing cumulative cost. Its online variant, On-Line Time Warping (OLTW) [38], enables causal alignment within a fixed-size of window. Variants include windowed [39], parallel [40], and constrained DTW [40, 41], as well as tempo-aware extensions [21, 42].

Probabilistic state-space models offer an alternative by treating alignment as latent state inference under uncertainty [24, 29, 43]. HMM-based systems model each note as a sequence of states (e.g., attack–steady–release), with

extensions including semi-Markov [44], hybrid [19], and Bayesian variants [45]. Kalman filter models and switching state-space systems [46, 47] further incorporate tempo dynamics, while particle filters [12, 29] handle multimodal uncertainty in real time.

Other paradigms include early string-matching algorithms [1] and reinforcement learning-based approaches for multimodal or visual score alignment [48].

3. IMPLEMENTATION

3.1 Python Package Structure

Matchmaker is an open source Python package that implements representative real-time music alignment algorithms within a modular, extensible framework. Figure 1 illustrates the overview of the package and the whole pipeline. The current version of *Matchmaker* provides two types of algorithms: 1) online time warping, with two variants: OLTWDixon, based on the methods proposed in [38, 49], and OLTWArzt, based on [21, 50]; and 2) an HMM-based algorithm, similar to the one used in [3, 47]. A full description of the algorithms and their parameters can be found in the supplementary Appendix.²

Matchmaker supports two main usage scenarios: (1) live streaming mode using the audio device and (2) simulation mode, which processes a performance file as input. Figure 2 shows an example of running live streaming mode with the default setting. The `AudioStream` object handles the input stream by chunking the audio with overlapping windows to avoid padding artifacts. Both the synthesized score audio and the performance audio are passed to a `Processor` object that performs feature extraction. The extracted features are pushed into a queue and consumed by the `OnlineAlignment` object, which runs the alignment methods in real time. *Matchmaker* takes a musical score with all symbolic music formats (MusicXML, MIDI, MEI, etc.) available by *partitura*.³ The returned output is the current position in the score, represented in beats as a musical unit according to the time signature in the piece. More detailed description and API documentation of the package are available here.⁴

3.2 Design and Implementation Details

We provide a simple and user-friendly interface to run the score following with minimal setup. As shown in Figure 2, users can instantiate a `Matchmaker` object with a score file and execute a run that iterates over the estimated score position for each step. To streamline real-time processing, the `AudioStream` class is implemented as a context manager that automatically handles stream initialization and teardown. Furthermore, the alignment process is designed as a generator, enabling users to receive score positions concurrently while the alignment is in progress.

² https://pymatchmaker.github.io/ismir2025_supplementary_materials/

³ <https://github.com/CPJKU/partitura>

⁴ <https://pymatchmaker.readthedocs.io/>

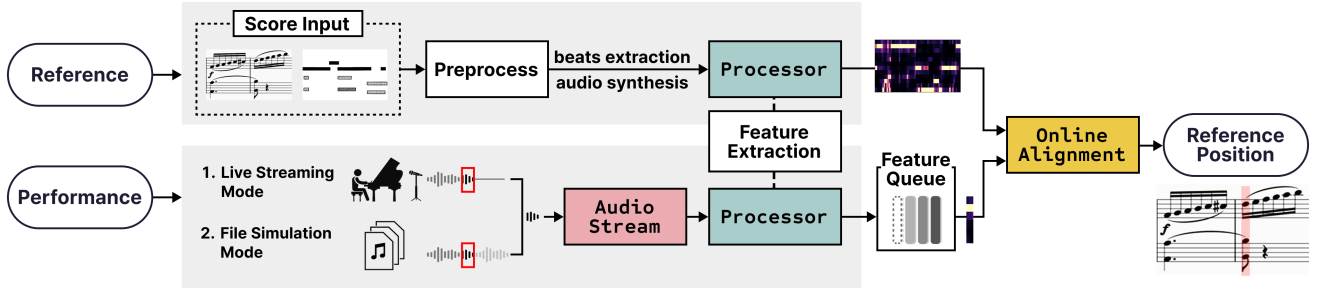


Figure 1. Overview of the score following package

```

1 from matchmaker import Matchmaker
2
3 mm = Matchmaker(
4     score_file="path/to/score.musicxml",
5     input_type="audio",
6 )
7 for current_position in mm.run():
8     print(current_position)

```

Figure 2. A code example for running the *Matchmaker* in a live streaming mode.

This design allows for efficient real-time integration without requiring users to manage multiple threads, buffers, or callbacks explicitly.

While the online mode uses a multi-threaded queue for asynchronous audio buffering, the simulation mode processes audio chunks in advance within a single-threaded setup. By decoupling real-time I/O concerns from core alignment evaluation, it is intended to avoid variability from Python version, OS-level threading, or queuing delays, ensuring a consistent and reproducible benchmarking environment. In addition, OLTWARzt is implemented in Cython [51] for efficiency, a superset of Python designed for C-like performance by incorporating C data types and optimizing the execution of Python code.

4. EXPERIMENTS

4.1 Datasets

We use three public piano performance datasets: (n)ASAP [52], Batik [53] and Vienna 4x22 [54], each of them offering complementary characteristics for benchmarking score following. (n)ASAP, a subset of the MAESTRO dataset including note-level score alignments, includes expressive performances of technically demanding solo piano pieces, offering high difficulty and stylistic diversity. We use only the pieces in the MAESTRO v2 test split. Vienna4x22 provides 22 distinct renditions for each of four relatively easy pieces, which is suitable to test robustness to interpretive variation. Batik dataset contains recordings of 12 Mozart sonatas by a single pianist with the longest average piece duration among the three datasets, enabling evaluation across long-form classical repertoire.

We use ground-truth beat-level annotations provided with the (n)ASAP dataset, and extract equivalent annota-

Dataset	#Pieces	#Perf	#Beats	#Notes	Dur (h)	Difficulty
(n)ASAP	43	59	26,329	100,958	2.65	6.53
Batik	30	30	18,789	102,421	2.85	5.67
Vienna	4	88	13,728	43,656	2.24	4.88
Total	77	177	58,846	247,035	7.74	6.11

Table 1. Datasets used in the evaluation.

tions for Batik and Vienna4x22 from the *.match* files [55], which contain note-wise score-performance alignments. In addition, we incorporate the difficulty levels of each piece based on G. Henle Publishers,⁵ which provides a 1-to-9 grading scale. The pieces used in our experiments span levels 4 through 9, representing a diverse set of works above intermediate level. Table 1 provides the detailed statistics of the datasets.

We only included performances in the experiment that recorded an MAE of less than 100 ms in the offline test, using the *synctoolbox*⁶ with Chroma & DLNCO features. The evaluation was conducted on 184 performances across 93 pieces, totaling over 58,000 beats and 247,000 notes, with an overall duration of 7.74 hours of performances and a piece-wise average difficulty of 6.11.

4.2 Experiment Settings

We conducted all evaluations under simulation-based conditions to ensure reproducibility. Live testing was avoided due to variability introduced by room acoustics and hardware setup, which complicates fair comparison across systems. The accuracy tests were carried out on an Intel i9-9900K CPU (16 cores @ 3.6 GHz), Python 3.9, with a sample rate of 44.1 kHz and a frame rate of 30, chosen to balance latency and alignment accuracy. We tested chromagram, mel-spectrogram, constant-Q transform (CQT), mel-frequency cepstral coefficients (MFCCs) [56] and a simple STFT-based onset-sensitive representation similar to the one used in Dixon [38], which we name log-spectral energy (LSE). While results for all features were evaluated, we report detailed latency and accuracy metrics for the best-performing configuration of each model. To account for hardware variability, latency was measured in multiple setups: an Intel i9-9900K, an Apple M4 MacMini, and an

⁵ <https://www.henle.de/Levels-of-Difficulty/>

⁶ <https://github.com/meinardmueller/synctoolbox>

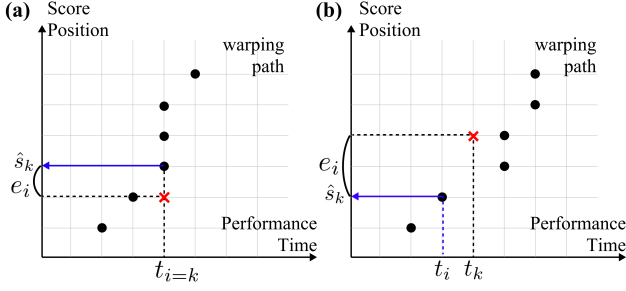


Figure 3. Two examples of error calculation using the mapping function. (a) shows a one-to-many alignment at the evaluation point, while (b) illustrates a skipped alignment.

Apple M2 Pro MacBook, with the reported latency values averaged across these devices.

4.3 Preprocessing

In the preprocessing step (see Fig. 1), the symbolic scores are synthesized to audio using FluidSynth, provided by *partitura*. Since MusicXML often lacks tempo markings, we set the synthesis tempo to each performance’s average—rounded to the nearest 20 BPM—assuming performers follow approximate tempo indications.

To generate beat annotations for the synthesized score audio, we computed beat positions using the synthesis tempo and the score’s time signature. For compound meters (e.g., 6/8, 9/8, and 12/8), we adopted (n)ASAP’s beat annotation rules—counting them as two, three, and four beats per measure, respectively—across all datasets to align score-side annotations with performance annotations. Based on the synthesized audio, we then extract the feature using the same `Processor` used in the online phase, but precompute them offline for the entire score sequence.

5. EVALUATION

Evaluating score following is challenging due to causality, timing precision, and output latency. Since the MIREX challenge [23] provided foundational metrics, later studies introduced alternative evaluation strategies including beat-level evaluations or asynchrony [3], reflecting the task’s frequent integration with automatic accompaniment systems.

In this work, we adopt two complementary evaluation perspectives. First, we evaluate in the performance domain, where errors are measured in milliseconds based on ground-truth annotations aligned to the audio. This approach is commonly used in audio-to-score alignment research and enables precise, frame-level evaluation, since the annotations directly reflect the actual timing of the performance. Second, we also evaluate in the score domain measured in beat units as suggested in [29, 57], which better reflects the nature of score following as a task of predicting the corresponding score position at each moment of the performance.

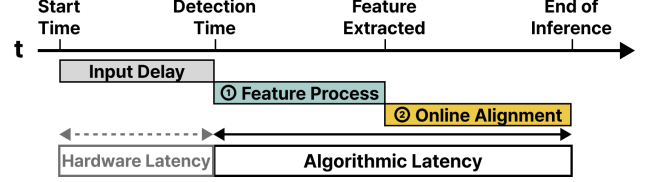


Figure 4. Defined delay types of the system. Only system delay is considered in the experiment.

5.1 Evaluation Metrics

We select evaluation metrics mostly adapted from score following MIREX benchmark [23] and audio-to-score alignment (ASA) metrics [57]. We use **Alignment Rate** (AR) within a tolerance range of $|\theta_e|$, varying from 50 ms to 2000 ms. We also compute **Absolute Errors** (AE), both in milliseconds and in beats, from which we derive the **Average Absolute Error** (AAE) and **Median Absolute Error** (MAE), along with the standard deviation σ_e . To further characterize the distribution of errors, we report **kurtosis** and **skewness** which capture the peakedness and asymmetry of the non-absolute error distribution, respectively. In addition, we report the **average latency** μ_{lat} , defined as the *system delay* from the detection time to the end of inference. Unlike total latency, this excludes *hardware latency* and is composed of two parts: (i) feature processing and (ii) execution of the online alignment algorithm for each frame step (see Fig. 4). Errors exceeding 2 seconds (or 2 beats in the score domain) are excluded from AE calculations, including both AAE and MAE, to avoid distortion from unbounded tracking failures. We report AR in two ways. The averaged piece-wise AR is a common measure, while the total AR reflects the proportion of successfully aligned beat events across the entire dataset. The latter avoids overrepresentation of shorter pieces and provides a more balanced view of overall performance.

To evaluate runtime latency under simulation, we measure two components: the average duration for extracting features from incoming audio frames, and the time taken by the alignment process to consume features and predict score positions. Specifically, the latency was computed from the moment audio was read to the time the score position was predicted—excluding hardware I/O delays. This two-step measurement allows for standardized latency reporting independent of the hardware setup.

5.2 Alignment Mapping Function

Given the alignment path, the alignment mapping function is applied to transfer the beat positions on one axis (either performance or score) to another axis to compute the alignment error. Due to the local, stepwise nature of real-time alignment, the resulting path is not necessarily monotonic and may contain multiple correspondents or skipped positions, depending on the implementation and purpose of the methods. Unlike linear interpolation methods commonly used in offline audio-to-score alignment, which assume continuous mappings, our evaluation relies only on

Dataset	Method	AAE(ms)↓ ± σ	MAE(ms)↓	Skew.	Kurt.	Piece-wise AR (%) ↑					Total AR↑ (≤2000ms, %)
						≤50ms	≤100ms	≤500ms	≤1000ms	≤2000ms	
(n)ASAP	OLTWDixon	189.55 ± 281.55	97.09	3.20	17.97	40.3	58.5	82.5	88.3	92.0	89.4
	OLTWARzt	183.56 ± 263.95	91.18	0.75	11.79	44.1	58.3	84.8	92.0	95.1	92.8
	HMM	487.73 ± 423.27	346.01	0.18	3.33	15.6	22.2	37.5	43.8	43.8	43.8
Batik	OLTWDixon	186.97 ± 262.55	<u>104.40</u>	3.75	24.70	28.2	51.7	82.1	85.2	87.6	<u>89.4</u>
	OLTWARzt	193.36 ± 269.13	<u>107.15</u>	1.00	12.63	35.9	53.0	82.2	87.4	90.3	<u>89.7</u>
	HMM	693.63 ± 376.58	641.77	0.11	0.98	4.5	10.8	34.0	46.2	64.2	61.9
Vienna4x22	OLTWDixon	285.43 ± 390.82	132.73	1.57	5.90	26.6	43.2	72.4	80.0	85.5	82.5
	OLTWARzt	300.41 ± 368.70	152.51	0.50	3.93	33.2	44.5	73.3	84.3	86.7	86.7
	HMM	439.64 ± 427.02	319.13	0.15	3.79	23.5	33.3	51.1	57.1	63.0	75.9

Table 2. Evaluation results on three datasets using different score-following methods. The piece-wise alignment rate (AR) is measured as the average over pieces, while the total AR indicates the global proportion of aligned beat events across the entire dataset. All tests were conducted with STFT-based Chroma as features.

predictions made prior to or at each evaluation time point. To reflect this, we define the mapping function as follows:

$$\hat{u}_k = \min\{u_i \mid (u_i, v_i) \in \mathcal{W}, v_i = \max\{v_j \mid v_j \leq k\}\},$$

where $\mathcal{W} = \{(u_i, v_i)\}$ is the warping path expressed in the frame indices: u_i is the score-rendered-audio frame index and v_i is the performance-audio frame index. The inner max finds the latest performance frame v_i not exceeding the current frame k , and the outer min selects the smallest score frame u_i among those alignments. This mapping relies solely on past or current frames to maintain causality. It handles skipped or one-to-many mappings and avoids any interpolation methods that depend on future frames.

6. RESULTS

Table 2 presents a comparison of alignment methods based on performance-domain evaluation, measured in milliseconds. All methods exhibit positive skewness in error distribution, reflecting the expected lag of the beat estimates in real-time alignment. The overall results show that the OLTW-based method outperforms the HMM baseline across all datasets in both alignment accuracy and coverage. While OLTWDixon and OLTWARzt show comparable MAE depending on the dataset, OLTWARzt consistently achieves higher coverage (*Total AR*), suggesting that it is more robust against overall failures. The difference likely stems from OLTWDixon skipping uncertain regions, while OLTWARzt’s “backward-forward” strategy corrects early misalignments and enhances coverage. Despite having the lowest AR, the HMM shows the lowest skewness and kurtosis primarily because significant errors (>2 s) are excluded from the summary statistics and its “sticky” behavior to linger in the same state in local regions tends to narrow the error distribution.

Table 3 presents an evaluation comparison in beat units, offering a tempo-normalized perspective. The overall trend mirrors the performance-domain results in milliseconds, but these results are standardized across tempi. AAE remains around 0.3 beats, with median values typically below 0.2. Total AR is consistently lower than the 2000 ms-

Dataset	Method	AAE↓(beats) ± σ	MAE↓(beats)	AR↑ (%)
(n)ASAP	OLTWDixon	0.22 ± 0.27	0.13	83.4
	OLTWARzt	0.27 ± 0.30	0.16	85.2
	HMM	0.80 ± 0.54	0.66	76.9
Batik	OLTWDixon	0.20 ± 0.27	0.11	88.9
	OLTWARzt	0.29 ± 0.34	0.18	88.8
	HMM	0.80 ± 0.38	0.67	59.3
Vienna4x22	OLTWDixon	0.31 ± 0.33	0.19	78.3
	OLTWARzt	0.37 ± 0.38	0.24	84.0
	HMM	0.76 ± 0.78	0.51	70.3

Table 3. Beat-level evaluation results including total alignment rate (AR) (%).

Feature Process			Online Alignment	
Type	MAE (ms)	Latency (ms)	Method	Latency (ms)
Chroma	265.50	3.05	OLTWDixon	1.22
mel	297.92	3.40	OLTWARzt	0.07
CQT	341.25	42.58	HMM	3.59
LSE	241.85	0.91		
MFCC	931.81	2.58		

Table 4. Comparison of feature types and alignment methods in terms of alignment error (MAE) and latency. LSE is log-spectral energy feature that was adopted in [38]. Latency values are averaged over the hardware setups evaluated in Section 4.

based metric, reflecting that most pieces have tempi above 60BPM, where two beats span less than two seconds.

In addition, a comparison of various feature types and latencies of the alignment methods are reported in Table 4. Among the features, log-spectral energy (LSE) shows the lowest MAE (241.85 ms) and delay (0.91 ms), indicating strong performance with minimal overhead. In contrast, CQT and MFCC yield higher MAE, with CQT also requiring considerable extraction time (42.58 ms), which limits its real-time suitability. For alignment methods, OLTWARzt achieves the lowest latency (0.07 ms), whereas HMM shows noticeably higher delay (3.59 ms) due to its computational complexity. These results highlight a trade-off between alignment accuracy and runtime efficiency,

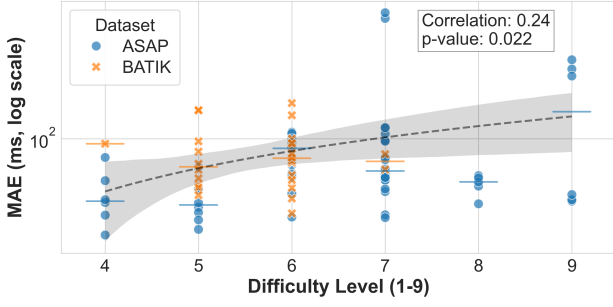


Figure 5. A scatter plot of mean absolute error (MAE) and Henle’s difficulty level in (n)ASAP and Batik dataset. The MAE results are from OLTWARzt.

with LSE and OLTWARzt providing a favorable balance for low-latency use.

The results also show about the characteristics of the datasets. While the overall alignment performance between (n)ASAP and Batik is comparable, Vienna4x22 shows noticeably higher error variance and kurtosis. This reflects the dataset’s unique structure—22 diverse renditions for each of only four pieces—leading to substantial variability in expressive timing, articulation, and interpretation. These variations present additional challenges for score following and result in heavier-tailed error distributions, as seen in the higher kurtosis values.

7. DISCUSSIONS

Figure 5 further illustrates the relationship between musical difficulty and alignment accuracy for (n)ASAP and Batik. We observe a moderate positive correlation ($r = 0.24$, $p = 0.022$) between MAE and the annotated difficulty levels, indicating that technically more demanding pieces tend to produce larger alignment errors. Vienna4x22 was excluded from this analysis due to its use of short excerpts, which makes consistent difficulty grading unreliable.

To further understand how alignment behaviors differ from methods, Figure 6 illustrates an example of alignment result comparing OLTWARzt (left) and HMM (right). Although OLTWARzt smoothly follows the beat events, the HMM warping path shows frequent horizontal segments, indicating the “sticky” tendency to stay near note onsets, reflecting its state-based formulation that emphasizes onset transitions. This leads to cases where it lingers on sustained notes and becomes locally stuck, showing limited forward momentum. The corresponding region (highlighted in yellow) exhibits changes in harmony, note density, and dynamics compared to the preceding passage, which provides sufficient contrast for the score follower to recover.

Lastly, we found that not only the choice of evaluation metrics, but also how alignment errors are computed (Section 5.2) can affect accuracy results to a meaningful extent. Small differences in error calculation sometimes led to noticeable shifts in reported accuracy.

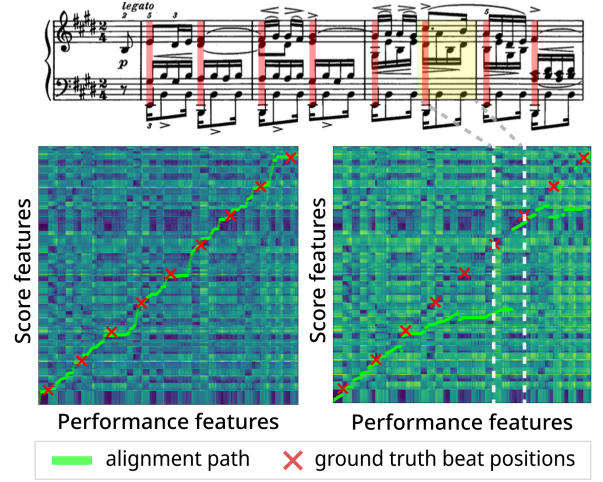


Figure 6. Two examples of alignment path with beat positions: (left) OLTWARzt, (right) HMM.

8. USE CASES AND APPLICATIONS

To demonstrate the practicality of our package, we built a lightweight web application that runs locally with real-time audio input or pre-recorded files. Built with websocket-based communication, the system responds quickly enough to ensure minimal perceptual delay. Our companion website includes a video demonstration and a link to the source code. This application aims to help researchers test their own score following models in an interactive setting. Beyond the web demo, our package is also used as the score following module in the ACCompanion [3], a real-time accompaniment system. These applications demonstrate the versatility of our framework and validate its utility in interactive music scenarios.

9. CONCLUSIONS AND FUTURE WORK

We presented a systematic framework for real-time audio-based score following as the open-source Python package *Matchmaker*. It supports live and simulation-based evaluation with baseline models, enabling reproducible benchmarking across datasets and features. Experiments on three public piano datasets show that the OLTWARzt variant achieves the highest performance and that the onset-sensitive spectral feature (LSE) outperforms chroma in both accuracy and latency. However, the current framework is limited in its support for tempo models commonly integrated with HMM-based score followers which may partly explain the limited performance of the HMM baseline. Also, recent works often include learned features or multimodal input which poses a new challenge to evaluate. Although our evaluation was limited to classical piano, extending *Matchmaker* to other instruments and genres requires only adapting the proper datasets and feature extraction modules. Future work will extend the framework to support a wider variety of instruments and musical styles, and include additional feature representations, advanced tempo modeling, and multimodal inputs.

10. ACKNOWLEDGMENTS

This work has been supported by the Austrian Science Fund (FWF), grant agreement PAT 8820923 (“*Rach3: A Computational Approach to Study Piano Rehearsals*”). Additionally, this work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) under Grant RS-2023-NR077289.

11. REFERENCES

- [1] R. B. Dannenberg, “An On-Line Algorithm for Real-Time Accompaniment,” in *Proceedings of the International Computer Music Conference (ICMC '84)*, Paris, France, 1984.
- [2] B. Vercoe, “The Synthetic Performer in the Context of Live Performance,” in *Proceedings of the International Computer Music Conference (ICMC '84)*, Paris, France, 1984.
- [3] C. Cancino-Chacon, S. Peter, P. Hu, E. Karystinaios, F. Henkel, F. Foscarin, N. Varga, and G. Widmer, “The ACCompanion: Combining Reactivity, Robustness, and Musical Expressivity in an Automatic Piano Accompanist,” in *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI-23)*, Macao, SAR, China, May 2023, arXiv:2304.12939 [cs, eess]. [Online]. Available: <http://arxiv.org/abs/2304.12939>
- [4] K. Armstrong, T.-C. Hung, J.-X. Huang, and Y.-W. Liu, “Real-time piano accompaniment model trained on and evaluated according to human ensemble characteristics,” in *Proceedings of the Sound and Music Computing (SMC)*, Porto, Portugal, 2024.
- [5] C. Raphael, “Music Plus One and Machine Learning,” in *Proceedings of the 27th International Conference on Machine Learning (ICML 2010)*, Haifa, Israel, 2010.
- [6] A. Maezawa, “I Got Rhythm, so Follow Me More: Modeling Score-Dependent Timing Synchronization in a Piano Duet,” in *Proceedings of the Sound and Music Computing Conference (SMC 2024)*, Porto, Portugal, 2024.
- [7] A. Arzt, G. Widmer, and S. Dixon, “Automatic Page Turning for Musicians via Real-Time Machine Listening,” in *Proceedings of the European Conference on Artificial Intelligence (ECAI)*, 2008.
- [8] F. Henkel, S. Schwaiger, and G. Widmer, “Fully Automatic Page Turning on Real Scores,” in *Extended Abstracts for the Late-Breaking Demo Session of the 22nd International Society for Music Information Retrieval Conference (ISMIR 2021)*, Online, 2021, arXiv:2111.06643 [cs]. [Online]. Available: <http://arxiv.org/abs/2111.06643>
- [9] C. Brazier and G. Widmer, “Towards Reliable Real-time Opera Tracking: Combining Alignment with Audio Event Detectors to Increase Robustness,” in *Proceedings of the Sound and Music Computing Conference*, Online, 2020, arXiv:2006.11033 [cs, eess]. [Online]. Available: <http://arxiv.org/abs/2006.11033>
- [10] J. Park, S. Yong, T. Kwon, and J. Nam, “A Real-Time Lyrics Alignment System Using Chroma and Phonetic Features for Classical Vocal Performance,” in *ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. Seoul, Korea, Republic of: IEEE, Apr. 2024, pp. 1371–1375. [Online]. Available: <https://ieeexplore.ieee.org/document/10445926/>
- [11] R. Gong, P. Cuvillier, N. Obin, and A. Cont, “Real-time audio-to-score alignment of singing voice based on melody and lyric information,” in *Interspeech 2015*. ISCA, Sep. 2015, pp. 3312–3316. [Online]. Available: https://www.isca-archive.org/interspeech_2015/gong15_interspeech.html
- [12] T. Otsuka, K. Nakadai, T. Takahashi, T. Ogata, and H. G. Okuno, “Real-Time Audio-to-Score Alignment Using Particle Filter for Coplayer Music Robots,” *EURASIP Journal on Advances in Signal Processing*, vol. 2011, no. 1, p. 384651, Dec. 2011. [Online]. Available: <https://asp-eurasipjournals.springeropen.com/articles/10.1155/2011/384651>
- [13] O. Lartillot, C. Cancino-Chacon, and C. Brazier, “Real-Time Visualisation of Fugue Played by a String Quartet,” in *Proceedings of the Sound and Music Computing Conference (SMC 2020)*, Online, 2020.
- [14] R. Dannenberg and B. Mont-Reynaud, “Following an Improvisation in Real Time,” in *Proceedings of the International Computer Music Conference*, Champaign/Urbana, Illinois, USA, 1987.
- [15] R. B. Dannenberg and H. Mukaino, “New Techniques for Enhanced Quality of Computer Accompaniment,” in *Proceedings of the International Computer Music Conference*, Cologne, Germany, 1988.
- [16] B. Vercoe and M. Puckette, “Synthetic Rehearsal: Training the Synthetic Performer,” in *Proceedings of the International Computer Music Conference (ICMC '85)*, Vancouver, BC, Canada, 1985.
- [17] M. Puckette, “Score following using the sung voice,” in *Proceedings of the International Computer Music Conference (ICMC '95)*, Banff, AB, Canada, 1995.
- [18] P. Desain, H. Honing, and H. Heijink, “Robust Score-Performance Matching: Taking Advantage of Structural Information,” in *Proceedings of the International Computer Music Conference (ICMC)*, Thessalonki, Greece, 1997.
- [19] C. Raphael and Y. Gu, “Orchestral Accompaniment for a Reproducing Piano,” in *Proceedings of the International Computer Music Conference (ICMC 2009)*, Montreal, Canada, 2009.

- [20] M. Prockup, D. Grunberg, A. Hrybyk, and Y. E. Kim, "Orchestral Performance Companion: Using Real-Time Audio to Score Alignment," *IEEE MultiMedia*, vol. 20, no. 2, pp. 52–60, Apr. 2013. [Online]. Available: <http://ieeexplore.ieee.org/document/6530591/>
- [21] A. Arzt and G. Widmer, "Real-Time Music Tracking Using Multiple Performances as a Reference," in *Proceedings of the 16th International Society for Music Information Retrieval Conference (ISMIR 2015)*, Malaga, Spain, 2015.
- [22] F. Henkel, S. Balke, M. Dorfer, and G. Widmer, "Score Following as a Multi-Modal Reinforcement Learning Problem," *Transactions of the International Society for Music Information Retrieval*, vol. 2, no. 1, pp. 67–81, Nov. 2019. [Online]. Available: <http://transactions.ismir.net/articles/10.5334/tismir.31/>
- [23] A. Cont, D. Schwarz, N. Schnell, and C. Raphael, "Evaluation of Real-Time Audio-to-Score Alignment," in *Music Information Retrieval Evaluation eXchange (MIREX 2007)*, Vienna, Austria, 2007.
- [24] A. Cont, "Antescofo: Anticipatory Synchronization and Control of Interactive Parameters in Computer Music," in *Proceedings of the International Computer Music Conference (ICMC '08)*, Belfast, Ireland, 2008.
- [25] J. Echeveste, P. Cuvillier, and A. Cont, "Improved Synchronization of a Pre-Recorded Music Accompaniment on a User's Music Playing," U.S. Patent US 2023/0 082 086 A1, Mar., 2023.
- [26] S. Dixon and G. Widmer, "MATCH: A Music Alignment Tool Chest," in *Proceedings of the 6th International Conference on Music Information Retrieval (ISMIR 2005)*, London, UK, 2005.
- [27] C. Joder, S. Essid, and G. Richard, "Learning Optimal Features for Polyphonic Audio-to-Score Alignment," *IEEE Transactions on Audio, Speech, and Language Processing*, vol. 21, no. 10, pp. 2118–2128, Oct. 2013. [Online]. Available: <http://ieeexplore.ieee.org/document/6525340/>
- [28] M. Muller, F. Kurth, and T. Roder, "Towards an Efficient Algorithm for Automatic Score-to-Audio Synchronization," in *Proceedings of the 5th International Conference on Music Information Retrieval (ISMIR 2004)*, Barcelona, Spain, 2004.
- [29] Z. Duan and B. Pardo, "A state space model for online polyphonic audio-score alignment," in *2011 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. Prague, Czech Republic: IEEE, May 2011, pp. 197–200. [Online]. Available: <http://ieeexplore.ieee.org/document/5946374/>
- [30] P.-W. Chou, F.-N. Lin, K.-N. Chang, and H.-Y. Chen, "A Simple Score Following System for Music Ensembles Using Chroma and Dynamic Time Warping," in *Proceedings of the 2018 ACM on International Conference on Multimedia Retrieval*. Yokohama Japan: ACM, Jun. 2018, pp. 529–532. [Online]. Available: <https://dl.acm.org/doi/10.1145/3206025.3206090>
- [31] M. Muller, "Music Synchronization," in *Fundamentals of Music Processing*. Cham: Springer International Publishing, 2021, pp. 119–170. [Online]. Available: https://link.springer.com/10.1007/978-3-030-69808-9_3
- [32] M. Pérez Fernández, H. Kirchhoff, and X. Serra, "A comparison of pitch chroma extraction algorithms," in *Proceedings of the 19th Sound and Music Computing Conference (SMC/JIM/IFC 2022)*. Saint-Étienne, France: SMC Network, 2022. [Online]. Available: <https://doi.org/10.5281/zenodo.6573082>
- [33] C.-T. Chen, J.-S. R. Jang, W.-S. Liu, and C.-Y. Weng, "An efficient method for polyphonic audio-to-score alignment using onset detection and constant Q transform," in *2016 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. Shanghai: IEEE, Mar. 2016, pp. 2802–2806. [Online]. Available: <http://ieeexplore.ieee.org/document/7472188/>
- [34] J. J. Carabias-Orti, F. J. Rodriguez-Serrano, P. Vera-Candeas, and N. Ruiz-Reyes, "An Audio to Score Alignment Framework Using Spectral Factorization and Dynamic Time Warping," in *Proceedings of the 16th International Society for Music Information Retrieval Conference (ISMIR 2015)*, Malaga, Spain, 2015.
- [35] F. Korzeniowski and G. Widmer, "Refined Spectral Template Models for Score Following," in *Proceedings of the Sound and Music Computing Conference (SMC 2013)*, Stockholm, Sweden, 2013.
- [36] S. Ewert, M. Muller, and P. Grosche, "High resolution audio synchronization using chroma onset features," in *2009 IEEE International Conference on Acoustics, Speech and Signal Processing*. IEEE, 2009, pp. 1869–1872.
- [37] A. Pillay, "A Neural Score Follower for Computer Accompaniment of Polyphonic Musical Instruments," Master's thesis, Carnegie Mellon University, Pittsburgh, PA, USA, 2024.
- [38] S. Dixon, "An On-Line Time Warping Algorithm for Tracking Musical Performances," in *Proceedings of the 19th International Joint Conference on Artificial Intelligence (IJCAI 05)*, Edinburgh, Scotland, 2005.
- [39] R. Macrae and S. Dixon, "Polyphonic Score Following Using on-Line Time Warping," in *Music Information Retrieval Evaluation eXchange (MIREX 2008)*, Philadelphia, USA, 2008.

- [40] F. J. Rodriguez-Serrano, P. Vera-Candeas, and J. J. Carabias-Orti, "A Real-Time Score Follower for MIREX 2015," in *Music Information Retrieval Evaluation eXchange (MIREX 2015)*, Malaga, Spain, 2015.
- [41] J. J. Carabias, F. J. Rodriguez, and P. Vera, "A Real-Time Nmf-Based Score Follower for MIREX 2012," in *Music Information Retrieval Evaluation eXchange (MIREX 2012)*, Porto, Portugal, 2012.
- [42] A. Arzt, "Real-Time Music Tracking Using Tempo-Aware on-Line Dynamic Time Warping," in *Proceedings of the Vienna Talk on Musical Acoustics (VITA)*, Vienna, Austria, 2010.
- [43] P. Cano, A. Loscos, and J. Bonada, "Score-Performance Matching using HMMs," in *Proceedings of the International Computer Music Conference*, Beijing, China, 1999.
- [44] E. Nakamura, P. Cuvillier, and A. Cont, "Autoregressive Hidden Semi-Markov Model of Symbolic Music Performance for Score Following," in *Proceedings of the 16th International Society for Music Information Retrieval Conference (ISMIR 2015)*, Malaga, Spain, 2015. [Online]. Available: <https://archives.ismir.net/ismir2015/paper/000015.pdf>
- [45] C. Raphael, "A Bayesian Network for Real-Time Musical Accompaniment," in *Proceedings of the 14th International Conference on Neural Information Processing Systems*, Vancouver, BC, Canada, 2001, pp. 1433–1439. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2001/file/2b0f658cbffd284984fb11d90254081f-Paper.pdf
- [46] R. Yamamoto, S. Sako, and T. Kitamura, "Real-Time Audio to Score Alignment Using Segmental Conditional Random Fields and Linear Dynamical System," in *Music Information Retrieval Evaluation eXchange (MIREX 2012)*, Porto, Portugal, 2012.
- [47] Y. Jiang and C. Raphael, "Score Following with Hidden Tempo Using a Switching State-Space Model," in *Proceedings of the 21st International Society for Music Information Retrieval Conference (ISMIR 2020)*, Online, 2020.
- [48] M. Dorfer, F. Henkel, and G. Widmer, "Learning to Listen, Read, and Follow: Score Following as a Reinforcement Learning Game," in *Proceedings of the 19th International Society for Music Information Retrieval Conference (ISMIR 2018)*, Paris, France, 2018.
- [49] S. Dixon, "Live Tracking of Musical Performances using On-Line Time Warping," in *Proceedings of the 8th International Conference on Digital Audio Effects (DAFx'05)*, Madrid, Spain, 2005.
- [50] A. Arzt and G. Widmer, "Towards Effective Any-Time Music Tracking," in *Proceedings of the Starting AI Researchers Symposium (STAIRS), held at ECAI 2010*, Lisbon, Portugal, 2010.
- [51] S. Behnel, R. Bradshaw, C. Citro, L. Dalcin, D. S. Seljebotn, and K. Smith, "Cython: The best of both worlds," *Computing in Science & Engineering*, vol. 13, no. 2, pp. 31–39, 2011.
- [52] S. D. Peter, C. E. Cancino-Chacón, F. Foscarin, A. P. McLeod, F. Henkel, E. Karystinaios, and G. Widmer, "Automatic Note-Level Score-to-Performance Alignments in the ASAP Dataset," *Transactions of the International Society for Music Information Retrieval*, vol. 6, no. 1, pp. 27–42, Jun. 2023. [Online]. Available: <http://transactions.ismir.net/articles/10.5334/tismir.149/>
- [53] P. Hu and G. Widmer, "The Batik-plays-Mozart Corpus: Linking Performance to Score to Musicological Annotations," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2023.
- [54] W. Goebel, "The vienna 4x22 piano corpus," 1999. [Online]. Available: <http://dx.doi.org/10.21939/4X22>
- [55] F. Foscarin, E. Karystinaios, S. D. Peter, C. Cancino-Chacón, M. Grachten, and G. Widmer, "The match file format: Encoding alignments between scores and performances," in *Proceedings of the Music Encoding Conference (MEC 2022)*, Halifax, Canada.
- [56] C. Brazier and G. Widmer, "Addressing the Recitative Problem in Real-time Opera Tracking," in *Proceedings of Frontiers of Research in Speech and Music FRSM 2020*, Online, Oct. 2020, arXiv:2010.11013 [eess].
- [57] A. Morsi and X. Serra, "Bottlenecks and solutions for audio to score alignment research," in *Proceedings of the 23rd International Society for Music Information Retrieval Conference (ISMIR 2022)*, 2022, pp. 272–279.

PRECISE AND SIMPLE AUDIO-TO-SCORE ALIGNMENT

Silvan D. PETER^{1,2}, Patricia HU (胡紫漪)¹, and Gerhard WIDMER^{1,2}

¹Institute of Computational Perception, Johannes Kepler University, Linz, Austria

²LIT AI Lab, Linz Institute of Technology, Linz, Austria

1. INTRODUCTION

Audio-to-score alignment is a long-standing challenge in music information retrieval and arguably the most widely applicable alignment task for music research. Alignment algorithms match two versions of a piece of music, and for this to work these versions need to be in comparable formats. Audio-to-audio alignment matches audio features; when matching audio files to scores, they must either synthesize the score or derive audio-like features by means of piano rolls or similar feature sequences [1–3]. Symbolic alignment, by contrast, matches symbolically encoded notes; in an audio-to-score scenario these would be obtained by a transcription of the audio file [4, 5]. In this article, we present an algorithm that bridges audio-like and symbol-level features directly. Sequential audio features encoding onset and spectral activation (see Figure 1) are matched to score positions by a bespoke dynamic programming-based matching algorithm derived from symbolic alignment methods. The resulting method is both precise - surpassing widely used audio-to-audio approaches based on synthesized scores -, and remains flexible in its digital signal processing components, i.e., the method is adaptable to diverse timbral characteristics without requiring a separate transcription model. Furthermore it inherits some of the symbolic alignment runtime advantages with an algorithmic complexity that is at worst linear in the length of the (typically short) symbolic score and (typically long) audio feature sequence. In the following sections, we provide a detailed algorithm description and evaluate its alignment quality on a large-scale dataset of solo piano recordings.

2. ALIGNMENT METHOD

2.1 Signal Processing

The audio signal is processed into two feature sequences, one for onset (time) information, the other for spectral (pitch) information. As a first step, the stereo signal is summed to mono and then sent through an IIR filterbank of second-order Butterworth filters. The filterbank consists of 88 filters centered at the key frequencies of an equally tempered 440 Hz Chamber pitch piano. The passband limits are set to the quarter tone middle points between adjacent

pitch frequencies. The default filterbank is set up for repertoire of this temperament, tuning, and register. Different setups are possible if the musical material is known to differ. The 88 filtered signals are aggregated by window-wise maximal values with window width and hop size being set for 50 Hz frame sequences and stacked as a spectrogram. The onset features are derived by a superflux algorithm [6] from the framed signal: a maximum filtered frame (across three vertically adjacent frequency bins) is subtracted from the subsequent frame and halfwave rectified. The resulting feature is bin-wise normalised to one. The durational feature is directly given by the original filtered and framed signal, again normalized to one for each frequency bin. While this procedure results in usable pitch-wise activation features for piano music, onset activation, normalization, and framerate can be adapted to suit the needs of different types of audio. The score representation consists of a list of score chords (for simplicity just coinciding notes irrespective of durations or voices) given by their onset beat position and the MIDI pitches of the notes encoded at this position.

2.2 Dynamic Programming

The alignment algorithm treats the pitch-wise onset and spectral activations as a proto-transcription and relates it to score information. For any position in the score and each pitch expected to be played at it, all positions in a given temporal window of the framed signal are given a cost related to the best fitting onset position with subsequent spectral activation. In typical dynamic programming fashion it starts at the beginning of both sequences and works through all score and signal window positions while keeping track of previously best aligned subsequences. Pseudocode 1 outlines the algorithm structure.

The cost function relating score positions to onset times in the spectrogram combines three components: an onset term (strong onset activation), a spectral term (continued spectral energy for several frames), and a stretch term (favoring low time warping or tempo variation). The latter is influenced by a path-wise beat period estimate that is continuously updated, associating lower costs to less locally variable tempo estimates. The algorithm affords several tuning parameters like the stretch limits and the weights to sum the cost from its stretch, onset, and spectral components. Each cost component is normalized to fall between zero and one.

Algorithm 1 Score-to-Audio Alignment via Dynamic Programming

Require: `score_onset_times[i]` ▷ score times in beats
Require: `pitch_sets[i]` ▷ notated pitches at score onset
Require: `onsets[p, t]` ▷ onset activation for pitch p at frame t
Require: `spec[p, t]` ▷ spectral presence for pitch p at frame t
Require: `stretch_limits, cost_weights`
1: $D[i, j] \leftarrow \infty$ ▷ accumulated cost
2: $B[i, j] \leftarrow -1$ ▷ backpointer
3: $BP[i, j] \leftarrow bp_{init}$ ▷ beat period estimate
4: $D[0, 0] \leftarrow 0$
5: **for** $i = 0$ to $M - 1$ **do** ▷ loop over score onsets
6: **for** $j = 0$ to $N - 1$ **do** ▷ loop over audio frames
7: **if** $D[i, j] = \infty$ **then**
8: **continue**
9: **end if**
10: $bp \leftarrow BP[i, j]$ ▷ get local beat period estimate
11: $\Delta_{score} \leftarrow \text{score_onset_times}[i + 1] - \text{score_onset_times}[i]$
12: $\text{candidate_frames} \leftarrow \text{compute_frame_window}(j, bp, \Delta_{score}, \text{stretch_limits})$
13: **for all** $p \in \text{pitch_sets}[i + 1]$ **do**
14: **for all** $j' \in \text{candidate_frames}$ **do**
15: $\text{stretch_term} \leftarrow \text{stretch_cost}(j' - j, bp, \Delta_{score})$
16: $\text{onset_term} \leftarrow \text{onsets}[p, j']$
17: $\text{spec_term} \leftarrow \min_k (\text{spec}[p, j' + k])$
18: $\text{transition_cost} \leftarrow D[i, j]$
19: $+ w_{onset} \cdot \text{onset_term}$
20: $+ w_{stretch} \cdot \text{stretch_term}$
21: $+ w_{spec} \cdot \text{spec_term}$
22: **if** $\text{transition_cost} < D[i + 1, j']$ **then**
23: $D[i + 1, j'] \leftarrow \text{transition_cost}$
24: $B[i + 1, j'] \leftarrow j$ ▷ backtracking pointer
25: $BP[i + 1, j'] \leftarrow \text{update_beat_period}(j' - j, \Delta_{score}, bp)$
26: **end if**
27: **end for**
28: **end for**
29: $D[i + 1, \text{mask_cost_above_reset_threshold}] \leftarrow \infty$
30: **end for**
31: $\text{alignment} \leftarrow \text{backtrack_through_pointers}(B)$
32: **return** alignment

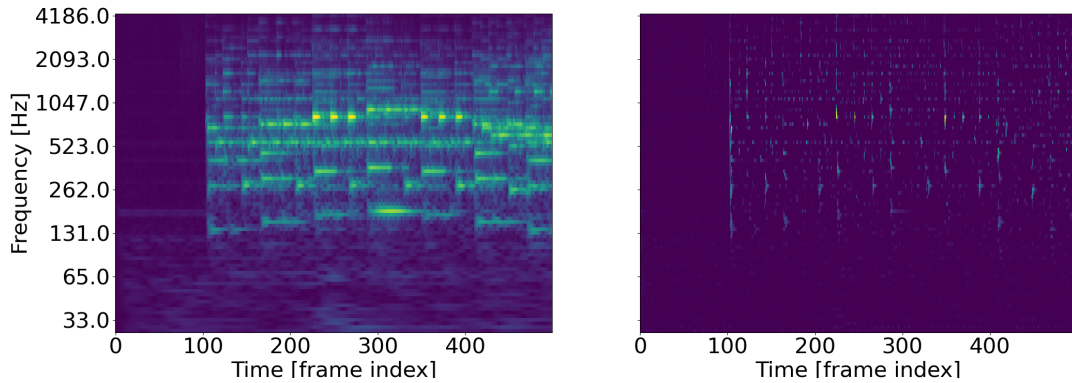


Figure 1. Spectral (left) and onset (right) activation features on the first ten seconds of a piano recording. There are 88 frequency bins (rows) logarithmically spaced and centered on the piano key frequencies. The temporal frame rate is 50 Hz.

Method	Mean (ms)	Median (ms)	< 50 ms (%)	< 100 ms (%)	< 200 ms (%)	< 500 ms (%)
Audio-to-Audio	135	49	53.2	74.4	87.7	91.7
Audio-to-Score (ours)	86	21	83.7	91.7	95.2	97.9
MIDI-to-Score	6	0	98.1	98.5	99.2	99.7

Table 1. Alignment results across a baseline audio-to-audio method using a synthesized score and onset as well as chroma features (first row), our proposed mixed audio-symbolic method (second row), and a MIDI-based symbolic alignment method (third row). Mean and median values are given in milliseconds, lower values are better. The remaining columns show the percentage of absolute errors below given thresholds, higher values are better. All values are averaged performance-wise across the dataset.

3. EVALUATION

We evaluate our algorithm on over 300 piano performances from the (n)ASAP Dataset [7]. We compare it to an audio-to-audio alignment baseline which uses Dynamic Time Warping on both onset-related and spectral features. The implementation is given by the synctoolbox library [8]. Audio-to-audio alignment based on a mix of features and synthesized audio is a common baseline for audio-to-score alignment. When high-quality transcriptions are available, symbolic alignment becomes a more precise baseline. To give an estimated upper bound for the quality of transcription-based symbolic alignment, we assess a MIDI-to-score alignment method (“DualDTWMatcher”) from the parangonar library [7] using the recorded MIDI performances of the (n)ASAP Dataset as proxies for perfect transcriptions. Table 1 shows the results in terms of mean and median errors as well as percentages of errors below four different thresholds.

Our method surpasses the baseline audio-to-audio method at all levels of precision, yet unsurprisingly falls short of the precision of a symbolic alignment model. Notably, several alignments were excluded from the audio-to-audio version where an obviously spurious alignment was computed, while both our method and the MIDI-to-score alignment worked robustly across the entire dataset. There is a runtime to precision tradeoff in the setting of the window size, frame rate, and threshold for cost rest. The higher these values, the more precise the alignment becomes, and the longer it takes to compute it. The values shown stem from a parameter set on the precise yet slow side (no threshold, medium window, 50 Hz). However, we did not optimize the parameter settings.

4. CONCLUSION

We introduce an audio-to-score algorithm which uses both onset and spectral audio features in a note-based matching procedure typically found in symbolic alignment. Our method leverages dynamic beat period estimates and score-informed pitch-wise onset and spectral processing to produce highly precise alignments. It relies on standard digital signal processing and dynamic programming techniques without the need for external processing through transcription, neural features, or synthesis. We hope that our implementation provides a simple and directly accessible tool for the community.

Our implementation is available online: <https://github.com/sildater/parangonar>

Acknowledgments

This research acknowledges support by the Linz Institute of Technology Artificial Intelligence Lab and the by the European Research Council (ERC), under the European Union’s Horizon 2020 research and innovation programme, grant agreement No. 101019375 *Whither Music?*.

5. REFERENCES

- [1] T. Kwon, D. Jeong, and J. Nam, “Audio-to-score alignment of piano music using rnn-based automatic music transcription,” in *Proceedings of the 14th Sound and Music Computing Conference (SMC)*, 2017.
- [2] S. Murgul, M. Reiser, M. Heizmann, and C. Seibert, “Fine-tuning midi-to-audio alignment using a neural network on piano roll and cqt representations,” *arXiv preprint arXiv:2506.22237*, 2025.
- [3] J. Zeitler, B. Maman, and M. Müller, “Robust and accurate audio synchronization using raw features from transcription models,” in *Proceedings of the International Society of Music Information Retrieval Conference (ISMIR)*, 2024, pp. 120–127.
- [4] F. Simonetta, S. Ntalampiras, and F. Avanzini, “Audio-to-score alignment using deep automatic music transcription,” in *23rd International Workshop on Multimedia Signal Processing (MMSP)*, 2021.
- [5] S. D. Peter, P. Hu, and G. Widmer, “Pairing real-time piano transcription with symbol-level tracking for precise and robust score following,” in *Proceedings of the Sound and Music Computing Conference (SMC)*, 2025.
- [6] S. Böck and G. Widmer, “Maximum filter vibrato suppression for onset detection,” in *Proceedings of the 16th International Conference on Digital Audio Effects (DAFx-13)*, Maynooth, Ireland, September 2013.
- [7] S. D. Peter, C. E. Cancino-Chacón, F. Foscarin, A. P. McLeod, F. Henkel, E. Karystinaios, and G. Widmer, “Automatic note-level score-to-performance alignments in the asap dataset,” *Transactions of the International Society for Music Information Retrieval (TISMIR)*, 2023.
- [8] M. Müller, Y. Özer, M. Krause, T. Prätzlich, and J. Driedger, “Sync toolbox: A python package for efficient, robust, and accurate music synchronization,” *Journal of Open Source Software*, vol. 6, no. 64, p. 3434, 2021.